

3. Chapter 2 — Variational Autoencoders (VAEs)

Chapter 2 is about why a normal autoencoder is not enough for generation, how VAEs turn autoencoders into probabilistic latent-variable models, why the ELBO is needed, how Jensen’s inequality appears, why the reparameterization trick is essential, and how VAE variants such as β -VAE and CVAE deal with latent-space structure and entanglement. The chapter learning objectives explicitly include explaining VAEs, why they are needed, why ELBO is used, the role of Jensen’s inequality, reparameterization, why sampling can fail in deterministic AEs, and entanglement/disentanglement in latent-variable models.

4. 1. The Big Idea of a VAE

A **Variational Autoencoder** is a generative model that combines:

1. an **encoder** that infers a latent distribution from data,
2. a **decoder** that generates data from latent variables,
3. a **prior** that defines where latent samples should come from,
4. an **ELBO objective** that balances reconstruction and latent-space regularization.

A normal autoencoder learns:

$$x \rightarrow z \rightarrow \hat{x}$$

A VAE learns:

$$x \rightarrow q_{\phi}(z | x) \rightarrow z \rightarrow p_{\theta}(x | z)$$

The difference is crucial:

Deterministic Autoencoder	Variational Autoencoder
maps x to one fixed code z	maps x to a distribution over z
decoder outputs reconstruction $\hat{x}$	decoder defines likelihood $p_{\theta}(x z)$
good for compression	good for probabilistic generation
latent space can have holes	latent space is regularized toward prior
random sampling may fail	random sampling is meaningful if trained well

The professor's notes state this directly: a deterministic AE maps $x \rightarrow z \rightarrow \hat{x}$, while a VAE maps x to $(\mu(x), \log \sigma^2(x))$, samples $z \sim q_\phi(z | x)$, and decodes with a probabilistic decoder.

5. 2. Why Normal Autoencoders Are Not Enough

A deterministic autoencoder learns:

$$\begin{aligned} z &= f_\phi(x) \\ \hat{x} &= g_\theta(z) \end{aligned}$$

and usually minimizes:

$$\min_{\phi, \theta} \mathbb{E}_{x \sim p_{data}} [\|x - g_\theta(f_\phi(x))\|^2]$$

This is useful for:

- compression,
- reconstruction,
- denoising,
- feature learning.

But it is not a principled generative model.

6. 2.1 The Main Problem: No Probability Model

A generative model should define something like:

$$p_\theta(x)$$

That means:

How likely is a data point x under the model?

A deterministic AE does not define this. It only defines a reconstruction mapping.

So it cannot properly answer:

- How likely is this image?
- Where should we sample z from?
- Which latent regions decode to realistic data?
- What is the probability distribution of generated samples?

The Chapter 2 summary says a deterministic autoencoder can reconstruct training-like examples, but it does not tell us what probability model generated the data.

7. 2.2 The Latent Space Can Have Holes

A deterministic AE is free to put latent codes wherever it wants.

For example, it may encode training samples into disconnected clusters:

$$z_1, z_2, z_3, \dots$$

The space between clusters may not correspond to valid data.

These empty or invalid regions are called **holes**.

If we randomly sample:

$$z \sim \mathcal{N}(0, I)$$

we may land in a hole.

Then decoding:

$$\hat{x} = g_{\theta}(z)$$

may produce nonsense.

The lecture notes say deterministic AE latent codes can be irregular, have holes, and random sampling from something like $\mathcal{N}(0, I)$ often decodes to nonsense.

8. 2.3 Reconstruction Is Not the Same as Generation

A deterministic AE can reconstruct because it sees an input first:

$$x \rightarrow z \rightarrow \hat{x}$$

But generation requires:

$$\begin{aligned} z &\sim p(z) \\ x &\sim p_{\theta}(x | z) \end{aligned}$$

The normal AE does not define a proper $p(z)$, so sampling is not principled.

Exam sentence:

A deterministic autoencoder is a reconstruction model, not a principled generative model, because it does not impose a known latent prior or define a likelihood over observed data.

9. 3. How a VAE Fixes the Problem

A VAE introduces a probabilistic latent-variable model.

Instead of assigning each x to one exact latent code, it assigns each x to a distribution:

$$q_{\phi}(z | x)$$

Usually:

$$q_{\phi}(z | x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x)))$$

The encoder outputs:

- $\mu_{\phi}(x)$: latent mean,
- $\log \sigma_{\phi}^2(x)$: latent log-variance.

Then we sample:

$$z \sim q_{\phi}(z | x)$$

and decode through:

$$p_{\theta}(x | z)$$

The professor's summary states that the encoder is usually a Gaussian with diagonal covariance and outputs $\mu_{\phi}(x)$ and $\log \sigma_{\phi}^2(x)$.

10. 4. The Probabilistic Generative Story

The VAE assumes data is generated by a hidden latent variable.

The generative process is:

1. Sample a latent variable from the prior:

$$z \sim p(z)$$

Usually:

$$p(z) = \mathcal{N}(0, I)$$

2. Generate data from the decoder:

$$x \sim p_{\theta}(x | z)$$

So the joint distribution is:

$$p_{\theta}(x, z) = p_{\theta}(x | z)p(z)$$

The marginal likelihood of observed data is:

$$p_{\theta}(x) = \int p_{\theta}(x | z)p(z) dz$$

This is the probability of x after considering all possible latent variables that could generate it. The professor's summary identifies this integral as the central computational challenge because it is generally intractable when the decoder is a neural network.

11. 5. What Each Distribution Means

12. $p(z)$: Prior

$$p(z) = \mathcal{N}(0, I)$$

This is where we sample from during generation.

The prior defines:

What latent codes are allowed before seeing any data?

If the latent space is trained to match this prior, then sampling becomes meaningful.

13. $p_{\theta}(x | z)$: Decoder / Likelihood

This is the probability of data x given latent code z .

It answers:

If the hidden cause is z , how likely is this observed data x ?

The decoder is probabilistic. Depending on the data type:

Data type	Possible decoder likelihood
binary pixels	Bernoulli
continuous pixels	Gaussian
categorical data	Categorical
text tokens	Categorical over vocabulary

14. $p_{\theta}(x, z)$: Joint Model

$$p_{\theta}(x, z) = p_{\theta}(x | z)p(z)$$

This describes the full generative story:

first choose latent z , then generate observation x .

15. $p_{\theta}(x)$: Marginal Likelihood / Evidence

$$p_{\theta}(x) = \int p_{\theta}(x | z)p(z)dz$$

This is what we ideally want to maximize.

But it is usually intractable.

16. $p_{\theta}(z | x)$: True Posterior

$$p_{\theta}(z | x) = \frac{p_{\theta}(x | z)p(z)}{p_{\theta}(x)}$$

This answers:

Given observed x , which latent z likely generated it?

But because $p_{\theta}(x)$ is hard to compute, the posterior is also hard to compute.

17. $q_{\phi}(z | x)$: Approximate Posterior / Encoder

Since the true posterior is hard, the VAE introduces:

$$q_{\phi}(z | x)$$

This approximates:

$$p_{\theta}(z | x)$$

This is why VAEs are connected to **variational inference**.

The summary emphasizes that $q_\phi(z | x)$ is not the true posterior; it is a tractable family optimized jointly with the decoder.

18. 6. Why It Is Called “Variational”

The word **variational** comes from **variational inference**.

The problem:

$$p_\theta(z | x)$$

is intractable.

So instead of computing the true posterior exactly, we choose a simpler family:

$$q_\phi(z | x)$$

and optimize it so that it becomes close to the true posterior.

This is variational inference:

approximate a hard posterior with a simpler distribution and optimize the approximation.

In VAEs, the approximate posterior is parameterized by a neural network, so inference becomes amortized.

19. 7. Amortized Inference

In traditional variational inference, you might optimize a separate approximate posterior for every data point.

In a VAE, one encoder network learns to output approximate posteriors for all data points:

$$x \mapsto (\mu_\phi(x), \log \sigma_\phi^2(x))$$

This is called **amortized inference**.

Meaning:

Instead of solving a new inference problem for each input, the encoder learns a reusable inference function.

This is efficient because after training, given a new x , the encoder immediately predicts a distribution over z .

20. 8. The True Objective: Maximize Data Likelihood

Ideally, we want:

$$p_{\theta}(x) \approx p_{data}(x)$$

Equivalently, we want generated samples:

$$z \sim p(z), x \sim p_{\theta}(x | z)$$

to be indistinguishable from real data.

The lecture notes summarize the mathematical goal of a VAE as learning:

$$p_{\theta}(x, z) = p_{\theta}(x | z)p(z)$$

such that:

$$p_{\theta}(x) = \int p_{\theta}(x | z)p(z)dz$$

matches the true data distribution.

But directly maximizing:

$$\log p_{\theta}(x)$$

is hard.

So VAEs maximize the ELBO.

21. 9. The ELBO

The ELBO is:

$$\mathcal{L}_{ELBO}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - KL(q_{\phi}(z|x) \parallel p(z))$$

It has two terms:

Term	Meaning
Expected log-likelihood reconstruction / data explanation	
KL divergence	latent regularization

The professor's summary says the first term encourages the latent variable inferred from x to contain enough information so the decoder can explain or reconstruct x , while the second pushes the approximate posterior toward the prior to make the latent space smoother, more organized, and more sampleable.

22. 10. Why the ELBO Is a Lower Bound

The ELBO satisfies:

$$\log p_{\theta}(x) = \mathcal{L}_{ELBO}(x) + KL(q_{\phi}(z|x) \parallel p_{\theta}(z|x))$$

Since:

$$KL(q_{\phi}(z|x) \parallel p_{\theta}(z|x)) \geq 0$$

then:

$$\mathcal{L}_{ELBO}(x) \leq \log p_{\theta}(x)$$

So it is a lower bound on the log marginal likelihood.

The ELBO becomes exact when:

$$q_{\phi}(z | x) = p_{\theta}(z | x)$$

because then the KL gap becomes zero. The professor's sample answer emphasizes that this shows VAE learning is not only about learning a decoder; it is also about learning an inference model that makes the bound tight.

23. 11. Jensen's Inequality and the ELBO

Jensen's inequality is used because log is concave:

$$\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$$

Start with:

$$\log p_{\theta}(x) = \log \int p_{\theta}(x | z)p(z)dz$$

Insert:

$$q_{\phi}(z | x)$$

by multiplying and dividing:

$$\log p_{\theta}(x) = \log \int q_{\phi}(z | x) \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)} dz$$

This becomes:

$$\log \mathbb{E}_{q_{\phi}(z|x)} \left[\frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)} \right]$$

Apply Jensen:

$$\log p_{\theta}(x) \geq \mathbb{E}_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x | z)p(z)}{q_{\phi}(z | x)} \right]$$

Expand:

$$= \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] + \mathbb{E}_{q_{\phi}(z|x)}[\log p(z) - \log q_{\phi}(z | x)]$$

The second part is:

$$-KL(q_{\phi}(z | x) \parallel p(z))$$

So:

$$\mathcal{L}_{ELBO}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] - KL(q_{\phi}(z | x) \parallel p(z))$$

Exam sentence:

Jensen's inequality moves the logarithm inside the expectation and converts the intractable log marginal likelihood into a tractable lower bound.

24. 12. The Reconstruction Term

$$\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)]$$

This is the expected log-likelihood.

It says:

Sample latent variables z from the encoder distribution and reward the decoder if it assigns high probability to the observed x .

In practice, this often behaves like a reconstruction loss.

If the decoder likelihood is Gaussian with fixed variance:

$$p_{\theta}(x | z) = \mathcal{N}(\hat{x}_{\theta}(z), \sigma^2 I)$$

then maximizing log-likelihood is equivalent to minimizing MSE:

$$\|x - \hat{x}\|^2$$

If the decoder likelihood is Bernoulli, it becomes binary cross-entropy.

So reconstruction loss is not arbitrary. It comes from the assumed likelihood model.

25. 13. The KL Term

$$KL(q_\phi(z | x) \| p(z))$$

This measures how far the encoder distribution is from the prior.

If:

$$q_\phi(z | x)$$

is very different from:

$$p(z) = \mathcal{N}(0, I)$$

then the KL is large.

The KL term prevents each data point from being assigned an isolated arbitrary latent region.

It forces the latent space to “speak the same language” as the prior.

The lecture notes summarize this nicely: the prior defines where you are allowed to sample from, the posterior decides where each data point lives, and the KL term forces them to speak the same language.

26. 14. The Core VAE Tradeoff

The VAE must balance:

good reconstruction

against:

organized latent space

If the reconstruction term dominates:

- z stores lots of input-specific information,
- reconstructions improve,
- latent space may become irregular,
- samples may be poor.

If the KL term dominates:

- $q_\phi(z | x)$ becomes too close to $p(z)$,
- latent z carries little information,
- reconstructions degrade,
- posterior collapse may occur.

The professor's sample answer says the VAE must balance fidelity to the data against structure and regularity in latent space.

27. 15. Reparameterization Trick

The encoder outputs:

$$\mu_\phi(x), \sigma_\phi(x)$$

We want to sample:

$$z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$$

But sampling directly creates a problem for backpropagation because the sample depends on parameters.

The reparameterization trick writes:

$$z = \mu_\phi(x) + \sigma_\phi(x)\epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

Now the randomness is isolated in ϵ , which does not depend on learnable parameters.

The sample z becomes a differentiable function of:

$$\mu_\phi(x), \sigma_\phi(x)$$

The professor's summary explains that reparameterization separates randomness from learnable parameters, allowing gradients to flow through the latent sample back into the encoder.

28. Why It Is More Than a Coding Trick

Without reparameterization:

$$z \sim q_\phi(z | x)$$

sampling interrupts ordinary gradient flow.

With reparameterization:

$$z = \mu + \sigma\epsilon$$

the computation graph remains differentiable.

So the conceptual change is:

Instead of differentiating through random sampling, express sampling as a deterministic transformation of fixed noise.

This is one of the core innovations that makes VAEs trainable with standard gradient descent.

29. 16. Training a VAE

For each training sample x :

1. Encoder outputs:

$$\mu_{\phi}(x), \log \sigma_{\phi}^2(x)$$

2. Sample noise:

$$\epsilon \sim \mathcal{N}(0, I)$$

3. Reparameterize:

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x)\epsilon$$

4. Decoder outputs parameters of:

$$p_{\theta}(x | z)$$

5. Compute reconstruction term.
6. Compute KL term.
7. Maximize ELBO or minimize negative ELBO.

In implementation, the loss is often:

$$\mathcal{L}_{loss} = \mathcal{L}_{recon} + \mathcal{L}_{KL}$$

because we minimize loss rather than maximize ELBO.

30. 17. Generation with a VAE

After training, generation is simple:

1. Ignore the encoder/posterior.
2. Sample from the prior:

$$z \sim p(z) = \mathcal{N}(0, I)$$

31. Decode:

$$x \sim p_{\theta}(x | z)$$

The lecture notes explicitly state that at generation time we ignore the posterior, sample from the prior, and decode.

Important distinction:

Stage	Uses encoder?	Uses prior?	Uses decoder?
Training/reconstruction	yes	through KL	yes
Generation/sampling	no	yes	yes

32. 18. Why Sampling Works in VAE but Fails in AE

In a normal AE:

- latent codes can be arbitrary,
- no known prior,
- latent holes exist,
- random z may decode badly.

In a VAE:

- latent distributions are pushed toward $p(z) = \mathcal{N}(0, I)$,
- similar nearby latent points tend to decode similarly,
- random samples from the prior are more likely to land in meaningful regions.

The professor's summary says a well-trained VAE learns a latent space where nearby points tend to decode to similar outputs, making interpolation and random sampling meaningful.

33. 19. Latent Space Geometry

A good VAE latent space should be:

- smooth,
- continuous,
- organized,
- sampleable,

- semantically meaningful.

A smooth latent space means:

$$z_1 \approx z_2$$

should decode to similar outputs:

$$p_{\theta}(x | z_1) \approx p_{\theta}(x | z_2)$$

This allows:

- interpolation,
 - random sampling,
 - controlled manipulation,
 - representation learning.
-

34. Interpolation

Suppose two latent codes:

$$z_A$$

and:

$$z_B$$

We can interpolate:

$$z_{\alpha} = (1 - \alpha)z_A + \alpha z_B$$

where:

$$0 \leq \alpha \leq 1$$

If the latent space is smooth, decoding each z_{α} should gradually morph one sample into another.

35. 20. Entanglement

Entanglement means latent dimensions mix multiple factors together.

Example:

Suppose we train a model on digit images.

A latent dimension might control:

- digit identity,
- stroke thickness,
- slant,

all at once.

Then changing one latent coordinate may change several semantic factors together.

This is entanglement.

36. Why Entanglement Is a Problem

If latent variables are entangled:

- hard to interpret,
- hard to control generation,
- changing one factor changes others,
- representation is less useful.

Example:

You want to change stroke thickness but accidentally change the digit from 7 to 1.

37. 21. Disentanglement

Disentanglement means separate latent dimensions correspond to separate meaningful factors.

Example:

Latent coordinate	Factor
z_1	stroke thickness
z_2	slant
z_3	digit style
z_4	rotation

Disentanglement makes the model more interpretable and controllable.

But a standard VAE does not automatically guarantee disentanglement. The professor's sample answer says the standard VAE objective encourages reconstruction and posterior regularity, but these pressures alone do not force separate latent coordinates to align with separate semantic factors.

38. 22. β -VAE

A β -VAE modifies the VAE objective:

$$\mathcal{L}_{\beta\text{-VAE}}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \beta KL(q_\phi(z|x) \parallel p(z))$$

where:

$$\beta > 1$$

usually increases regularization.

39. What β Does

If $\beta = 1$:

standard VAE

If $\beta > 1$:

stronger KL pressure

This pushes the latent representation to be:

- simpler,
- more compressed,
- more factorized,
- closer to the prior.

This can encourage disentanglement.

The professor's summary says β -VAE weights the KL term more strongly; when $\beta > 1$, the model is encouraged to use a more factorized and compressed latent code, which can promote disentanglement, but too large β harms reconstruction.

40. β -VAE Tradeoff

Higher β helps	Higher β hurts
disentanglement	reconstruction
latent simplicity	detail preservation
compression	sample fidelity
interpretability	input-specific information

Exam sentence:

β -VAE promotes disentanglement by increasing KL pressure, but this often comes at the cost of blurrier or less accurate reconstructions.

41. 23. Posterior Collapse

Posterior collapse occurs when:

$$q_{\phi}(z \mid x) \approx p(z)$$

for many or all x .

This means:

The latent variable carries little or no information about the input.

The decoder ignores z .

The professor's summary states that posterior collapse happens when $q_\phi(z | x)$ becomes almost identical to the prior, so z carries little information about x , and the decoder effectively ignores the latent variable.

42. Why Posterior Collapse Happens

Common causes:

1. Decoder too powerful.
 2. KL pressure too strong.
 3. KL term introduced too early.
 4. Latent dimension too small or too constrained.
 5. Autoregressive decoder can model data without z .
-

43. Symptoms

- KL term near zero,
- latent codes look like random noise,
- reconstructions may become generic,
- changing z has little effect,
- samples lack diversity or input-specific detail.

The lecture notes use an analogy: healthy VAE means all houses are inside the city but each house has a different address; posterior collapse means every house is at the same address.

44. Important Distinction

We want:

$$q_\phi(z | x)$$

to be compatible with:

$$p(z)$$

but not identical for every x .

A healthy VAE has:

$$0 < I(X; Z) < \infty$$

Meaning z contains some information about x , but not unlimited information.

The notes say: “We want the posterior to be compatible with the prior, not identical to it.”

45. 24. ELBO Gap

The ELBO gap is:

$$\log p_{\theta}(x) - \mathcal{L}_{ELBO}(x) = KL(q_{\phi}(z | x) \parallel p_{\theta}(z | x))$$

This measures how far the approximate posterior is from the true posterior.

If $q_{\phi}(z | x)$ is too simple, for example a diagonal Gaussian, but the true posterior is:

- multimodal,
- curved,
- correlated,
- complex,

then the approximate posterior cannot match it.

The lecture notes say the ELBO gap appears when the posterior family is too weak to make the bound tight; the score improves, but the model does not necessarily become a better generator.

46. ELBO Gap vs Posterior Collapse

These are different.

Posterior collapse	ELBO gap
latent z is unused ($q(z$	latent z is used but inaccurately modeled $x) \approx p(z)$)

Posterior collapse**ELBO gap**

KL to prior near zero KL to true posterior large
decoder ignores latent posterior approximation is poor

The notes explicitly warn that posterior collapse and ELBO gap are different failure modes.

47. 25. Why Reconstructions Can Be Good but Samples Bad

Reconstructions use:

$$z \sim q_{\phi}(z \mid x)$$

Sampling uses:

$$z \sim p(z)$$

If the regions used by $q_{\phi}(z \mid x)$ during reconstruction do not overlap well with the prior $p(z)$, then:

- reconstructions can look good,
- random samples can look bad.

The lecture notes state this directly: reconstructions use $z \sim q_{\phi}(z \mid x)$, sampling uses $z \sim p(z)$, so if the posterior approximation is poor and the aggregate posterior does not match the prior well, samples may come from regions the decoder never learned well.

48. 26. CVAE — Conditional Variational Autoencoder

A **Conditional VAE** models:

$$p_{\theta}(x \mid y)$$

instead of only:

$$p_{\theta}(x)$$

where y is a known condition, such as:

- class label,
- attribute,
- text prompt,
- metadata.

For MNIST, y could be the digit label:

$$y \in \{0, 1, \dots, 9\}$$

A CVAE lets us generate a specific digit on command.

49. CVAE Encoder

$$q_{\phi}(z \mid x, y)$$

The encoder sees both:

- input x ,
- condition y .

This means the encoder can focus on variation within the class, rather than encoding the class itself.

50. CVAE Decoder

$$p_{\theta}(x \mid z, y)$$

The decoder receives:

- latent z ,
- condition y .

So the decoder can generate samples consistent with the condition.

51. CVAE Prior

The simplest CVAE keeps:

$$p(z) = \mathcal{N}(0, I)$$

Some variants use:

$$p(z | y)$$

as a conditional prior, but this is not required for the basic CVAE. The lecture notes mention that the simplest CVAE keeps the same prior, though conditional priors are possible extensions.

52. CVAE ELBO

$$\mathcal{L}_{CVAE}(x, y) = \mathbb{E}_{q_{\phi}(z|x, y)}[\log p_{\theta}(x | z, y)] - KL(q_{\phi}(z | x, y) \parallel p(z))$$

What changes compared with a standard VAE?

Standard VAE	CVAE
$q_{\phi}(z x)$	$q_{\phi}(z x, y)$
$p_{\theta}(x z)$	$p_{\theta}(x z, y)$
models $p(x)$	models $p(x y)$
less direct control	controllable generation

The lecture notes provide this CVAE structure: condition both encoder and decoder on y , with encoder $q_{\phi}(z | x, y)$, decoder $p_{\theta}(x | z, y)$, and often prior $p(z) = \mathcal{N}(0, I)$.

53. 27. Why CVAE Helps Disentanglement

In a standard VAE, z must encode everything:

- digit identity,
- stroke thickness,
- slant,
- rotation,
- style.

In a CVAE, the known condition y handles some factors.

For MNIST:

- y = digit identity,
- z = style, thickness, slant, handwriting variation.

So z does not need to encode the digit label again.

The notes explain this as: with conditioning, class information lives in c , and z is free to encode style, variation, and noise; CVAE reduces entanglement by removing known factors from the latent space.

Exam sentence:

A CVAE disentangles by saying: this factor is already known through the condition, so the latent variable should not waste capacity encoding it again.

54. 28. β -VAE vs CVAE

Both can reduce entanglement, but in different ways.

Model	How it reduces entanglement
β -VAE	restricts latent capacity using stronger KL pressure
CVAE	removes known factors by conditioning on labels/attributes

The notes summarize this clearly: β -VAE reduces entanglement by restricting capacity, while CVAE reduces entanglement by removing known factors; they solve the problem from different directions and can be complementary.

55. 29. What CVAEs Do Not Solve

CVAEs are useful, but they do not solve everything.

They do not automatically disentangle unknown factors.

They require labels or side information.

They only disentangle what you explicitly condition on.

Example:

If you condition on digit identity, z may still entangle:

- stroke thickness,
- rotation,
- slant.

So CVAE shifts the burden of representation; it does not eliminate it.

56. 30. VAE vs β -VAE vs CVAE on MNIST

Model	Goal	Latent behavior	Main strength	Main weakness
VAE	reconstruct and sample digits	z encodes identity + style	general generative model	entangled latents
β -VAE	simpler/factorized latent space	z more constrained	better interpretability	worse reconstruction if β too high
CVAE	generate digits controlled by label	y encodes identity, z encodes style	controllable generation	needs labels

The lecture notes compare these MNIST variants and state that standard VAE latents often mix digit identity and style, β -VAE constrains the latent space, and CVAE controls digit identity by conditioning.

57. 31. VAE as Transport

A useful big-picture view from later course material is that a VAE transports samples from a simple latent base distribution into data space.

Sampling:

$$\begin{aligned} z &\sim p(z) \\ x &\sim p_{\theta}(x | z) \end{aligned}$$

So the decoder acts like a transport map from latent space to data space.

Chapter 5's transport synopsis says a VAE learns an encoder posterior and decoder likelihood; sampling uses the decoder as a transport map from a simple latent base into data space.

This is useful because it connects VAEs to:

- diffusion,
- flow matching,
- latent diffusion,
- latent flow matching.

58. 32. VAE Strengths

VAEs are useful because they provide:

- principled latent-variable modeling,
- smooth latent space,
- sampling from known prior,
- interpolation,
- representation learning,
- probabilistic inference,
- foundation for latent generative models.

They are especially useful when latent representations matter.

59. 33. VAE Weaknesses

VAEs also have limitations:

1. Samples can be blurry.
2. Likelihood assumptions may be too simple.
3. Posterior collapse can happen.
4. ELBO gap can make training look good while samples remain poor.
5. Standard VAE does not guarantee disentanglement.
6. Decoder quality limits generation quality.
7. Pixel-level reconstruction losses may not match perceptual quality.

The notes mention that likelihood/reconstruction choices can blur fine details and that VAEs suffer from posterior collapse and ELBO tradeoffs.

60. 34. Why VAE Outputs Can Be Blurry

If the decoder likelihood is Gaussian and the training objective behaves like pixel MSE, then the model is encouraged to predict averages.

If multiple outputs are plausible, averaging them can blur details.

Example:

If the model is unsure whether a stroke should be slightly left or right, MSE may encourage a middle result, producing blur.

Exam sentence:

VAE samples can be blurry because simple likelihood assumptions and pixel-wise losses often reward averaged reconstructions rather than perceptually sharp samples.

61. 35. Important Formula Summary

62. Autoencoder

$$\begin{aligned} x &\rightarrow z \rightarrow \hat{x} \\ \mathcal{L}_{AE} &= \|x - \hat{x}\|^2 \end{aligned}$$

63. VAE Generative Model

$$\begin{aligned} p(z) &= \mathcal{N}(0, I) \\ p_{\theta}(x | z) & \\ p_{\theta}(x, z) &= p_{\theta}(x | z)p(z) \\ p_{\theta}(x) &= \int p_{\theta}(x | z)p(z)dz \end{aligned}$$

64. Approximate Posterior

$$q_{\phi}(z \mid x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x)))$$

65. ELBO

$$\mathcal{L}_{ELBO}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x \mid z)] - KL(q_{\phi}(z \mid x) \parallel p(z))$$

66. ELBO Gap

$$\log p_{\theta}(x) - \mathcal{L}_{ELBO}(x) = KL(q_{\phi}(z \mid x) \parallel p_{\theta}(z \mid x))$$

67. Reparameterization

$$\begin{aligned} z &= \mu_{\phi}(x) + \sigma_{\phi}(x)\epsilon \\ \epsilon &\sim \mathcal{N}(0, I) \end{aligned}$$

68. β -VAE

$$\mathcal{L}_{\beta VAE} = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x \mid z)] - \beta KL(q_{\phi}(z \mid x) \parallel p(z))$$

69. CVAE

$$\begin{aligned} &q_{\phi}(z \mid x, y) \\ &p_{\theta}(x \mid z, y) \\ \mathcal{L}_{CVAE}(x, y) &= \mathbb{E}_{q_{\phi}(z|x,y)}[\log p_{\theta}(x \mid z, y)] - KL(q_{\phi}(z \mid x, y) \parallel p(z)) \end{aligned}$$

70. 36. Most Likely Exam Questions

71. Question 1: Why is a VAE needed instead of a deterministic autoencoder?

A deterministic autoencoder can reconstruct inputs but does not define a proper generative probability model. Its latent codes may occupy arbitrary disconnected regions, so randomly sampling a latent vector can land in holes and decode to unrealistic outputs. A VAE fixes this by introducing a probabilistic latent-variable model: the encoder outputs a distribution $q_\phi(z | x)$, the decoder defines $p_\theta(x | z)$, and the KL term regularizes the latent space toward a known prior, usually $\mathcal{N}(0, I)$. This makes sampling and interpolation more meaningful.

72. Question 2: Explain the two terms of the ELBO.

The ELBO is:

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - KL(q_\phi(z | x) \parallel p(z))$$

The first term is the expected reconstruction or likelihood term. It rewards latent samples that allow the decoder to explain the observed input well. The second term is the KL regularization term. It pushes the approximate posterior toward the prior so that the latent space remains smooth, organized, and sampleable. The tension is that better reconstruction requires storing more information in z , while stronger KL pressure forces z toward a simpler distribution.

73. Question 3: Why is the ELBO a lower bound?

The ELBO is a lower bound because:

$$\log p_\theta(x) = \mathcal{L}_{ELBO}(x) + KL(q_\phi(z | x) \parallel p_\theta(z | x))$$

Since KL divergence is always nonnegative:

$$\mathcal{L}_{ELBO}(x) \leq \log p_\theta(x)$$

It becomes exact when:

$$q_\phi(z | x) = p_\theta(z | x)$$

because the KL gap becomes zero.

74. Question 4: What is the reparameterization trick and why is it important?

The reparameterization trick rewrites latent sampling as:

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x)\epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

This separates randomness from learnable parameters. The random noise ϵ is fixed with respect to the model parameters, while $\mu_{\phi}(x)$ and $\sigma_{\phi}(x)$ remain differentiable. This allows gradients to flow through the latent sample and makes VAEs trainable using ordinary backpropagation.

75. Question 5: What is posterior collapse?

Posterior collapse occurs when:

$$q_{\phi}(z | x) \approx p(z)$$

for many or all inputs. In this case, the latent variable carries little information about the input, and the decoder effectively ignores z . It often happens when the decoder is too powerful or when KL pressure is too strong early in training. The result is a model that may satisfy the KL part of the objective but fails to use the latent space meaningfully.

76. Question 6: Why does a standard VAE not guarantee disentanglement?

A standard VAE encourages good reconstruction and approximate posterior regularity, but these objectives do not force individual latent dimensions to correspond to separate semantic factors. The model may entangle factors such as class, style, thickness, and rotation if that helps optimize the ELBO. Disentanglement usually requires additional inductive bias, such as stronger KL pressure in β -VAE or explicit conditioning in CVAE.

77. Question 7: How does β -VAE encourage disentanglement?

β -VAE modifies the ELBO by multiplying the KL term by β :

$$\mathcal{L} = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - \beta KL(q_{\phi}(z|x) \parallel p(z))$$

When $\beta > 1$, the model is pushed toward a more compressed and factorized latent representation. This can encourage disentanglement, but it often reduces reconstruction quality because the latent bottleneck becomes more restrictive.

78. Question 8: How does CVAE improve controllability?

A CVAE conditions both encoder and decoder on known information y :

$$\begin{aligned} q_{\phi}(z|x, y) \\ p_{\theta}(x|z, y) \end{aligned}$$

This allows generation from a specific condition, such as generating a chosen digit class. Since the condition carries known factors like class identity, z can focus on residual variation such as style, thickness, or slant. This improves controllability and can reduce entanglement of known factors.

79. 37. Common Mistakes to Avoid

80. Mistake 1: Saying the VAE encoder outputs one latent vector

Not exactly. It outputs a distribution, usually represented by μ and $\log \sigma^2$.

Correct:

$$q_{\phi}(z|x) = \mathcal{N}(\mu_{\phi}(x), \text{diag}(\sigma_{\phi}^2(x)))$$

81. Mistake 2: Thinking the decoder is just a reconstruction function

The decoder defines a likelihood:

$$p_{\theta}(x | z)$$

It may output a reconstruction, but theoretically it is a probability model.

82. Mistake 3: Forgetting that generation ignores the encoder

During generation:

$$\begin{aligned} z &\sim p(z) \\ x &\sim p_{\theta}(x | z) \end{aligned}$$

The encoder is not used.

83. Mistake 4: Thinking KL should be zero

KL should not necessarily be zero for every input. If:

$$q_{\phi}(z | x) = p(z)$$

for all x , the latent code contains no input-specific information.

We want compatibility, not total collapse.

Mistake 5: Thinking β -VAE always improves everything

β -VAE can improve disentanglement but often hurts reconstruction.

Mistake 6: Thinking CVAE automatically disentangles all factors

CVAE only removes or controls factors that are explicitly given as conditions.

38. One-Page Open-Note Summary

A Variational Autoencoder is a probabilistic latent-variable generative model. A deterministic autoencoder maps x to one latent code z and reconstructs $\hat{x}$, but it does not define a likelihood or a principled prior, so random latent sampling can fail. A VAE fixes this by assuming a generative story: sample $z \sim p(z) = \mathcal{N}(0, I)$, then generate $x \sim p_\theta(x | z)$. The encoder $q_\phi(z | x)$ approximates the intractable posterior $p_\theta(z | x)$, usually as a diagonal Gaussian with mean and variance predicted by a neural network. Because $\log p_\theta(x) = \log \int p_\theta(x | z)p(z)dz$ is intractable, VAEs maximize the ELBO: expected log-likelihood minus KL divergence to the prior. The reconstruction term encourages z to explain x , while the KL term organizes the latent space and makes sampling meaningful. Jensen's inequality explains why the ELBO is a lower bound. The reparameterization trick $z = \mu + \sigma\epsilon$ allows gradients to pass through stochastic sampling. VAEs can suffer from posterior collapse when $q(z | x) \approx p(z)$, meaning z carries little information. β -VAE increases KL pressure to encourage simpler, more disentangled latents, but may hurt reconstruction. CVAE conditions the encoder and decoder on known information y , allowing controlled generation and reducing the need for z to encode known factors.

39. Very Short Final Revision

Know these:

- AE reconstructs; VAE generates probabilistically.
- AE latent space can have holes.
- VAE encoder outputs $q_\phi(z | x)$, not one fixed z .
- Prior is usually $p(z) = \mathcal{N}(0, I)$.
- Decoder is $p_\theta(x | z)$.
- True goal is maximize $\log p_\theta(x)$.
- ELBO is used because $\log p_\theta(x)$ is intractable.
- ELBO = reconstruction term – KL term.
- Jensen's inequality gives the lower bound.
- Reparameterization: $z = \mu + \sigma\epsilon$.
- Generation: sample $z \sim p(z)$, then decode.
- Posterior collapse: $q(z | x) \approx p(z)$, latent unused.
- β -VAE: stronger KL, better disentanglement, worse recon.

- CVAE: condition on labels/attributes for controlled generation.

Chapter 1. **Chapter 3 — Diffusion Models**

Chapter 3 introduces **diffusion models**, especially the DDPM-style view. The core idea is simple but very powerful:

Start with clean data, gradually add noise until it becomes almost pure Gaussian noise, then train a neural network to reverse that process step by step.

A diffusion model does **not** learn to generate an image in one big jump. Instead, it learns many small denoising steps. This is why diffusion models are stable and powerful: generation becomes a sequence of manageable local predictions rather than one difficult direct mapping from noise to data.

Chapter 2. **1. What Is a Diffusion Model?**

A **diffusion model** is a generative model that learns to generate data by reversing a gradual noising process.

The process has two directions:

Chapter 3. **Forward process: data → noise**

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T$$

where:

- x_0 = clean real data,
- x_t = partially noisy data at timestep t ,
- x_T = almost pure Gaussian noise.

Chapter 4. **Reverse process: noise → data**

$$x_T \rightarrow x_{T-1} \rightarrow x_{T-2} \rightarrow \cdots \rightarrow x_0$$

The reverse process is learned by a neural network.

Simple intuition:

Training teaches the model how to remove a little bit of noise at each step. Generation starts from random noise and repeatedly denoises until a realistic sample appears.

The Chapter 3 notes explain that diffusion models work because we define a tractable noise process that maps data to Gaussian noise, then learn to reverse it step by step to generate new data.

Chapter 5. 2. Why Diffusion Models Make Sense

Direct generation is hard:

$$\mathcal{N}(0, I) \rightarrow \text{realistic image}$$

This asks a model to transform pure noise into a complex image in one step.

Diffusion makes the task easier by breaking it into many small steps:

$$x_t \rightarrow x_{t-1}$$

Each reverse step only removes a small amount of noise.

The notes explain that instead of directly learning a difficult mapping from noise to data, diffusion breaks the problem into many local, approximately Gaussian reverse steps, making the learning problem more stable and well-conditioned.

Chapter 6. 3. Review: Markov Chain

A **Markov chain** is a sequence of random variables where the next state depends only on the current state, not the full past.

$$p(x_t \mid x_{t-1}, x_{t-2}, \dots, x_0) = p(x_t \mid x_{t-1})$$

Meaning:

The current state contains all the information needed to predict the next state.

In diffusion, the forward process is a Markov chain:

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

This means each noisy image x_t is created from the previous noisy image x_{t-1} .

The Chapter 3 notes describe the forward process as a Markov chain where each transition depends only on the previous step.

Chapter 7. 4. The Forward Diffusion Process

The forward process is fixed and known. It is not learned.

It gradually adds Gaussian noise to the data.

In DDPM, a common forward transition is:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t) I)$$

where:

Symbol	Meaning
q	fixed forward noising distribution
x_{t-1}	previous state
x_t	new noisier state
α_t	signal-retention factor
$1 - \alpha_t$	noise variance added at step t
I	identity covariance matrix

The notes give this DDPM transition and explain that as t grows, the distribution becomes close to standard normal, allowing $p(x_T) = \mathcal{N}(0, I)$ to be used as the starting point for generation.

Chapter 8. 5. Noise Schedule

The model does not add the same arbitrary noise at every step. The amount of noise is controlled by a **noise schedule**.

Usually we define:

$$\beta_t \in (0,1)$$

where β_t controls how much noise is added at step t .

Then:

$$\alpha_t = 1 - \beta_t$$

and:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

where:

Term	Meaning
β_t	amount of noise added at step t
α_t	amount of signal kept at step t
$\bar{\alpha}_t$	cumulative signal kept after t steps

The noise schedule controls how fast information is destroyed. If noise is added too aggressively, the reverse problem becomes too difficult. If noise is added too slowly, training can become inefficient or require too many nearly redundant steps.

Chapter 9. 6. Closed-Form Marginal $q(x_t \mid x_0)$

A very important property of DDPM is that we do **not** need to simulate every step from x_0 to x_t .

There is a closed-form formula:

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

Equivalent sampling form:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

where:

$$\epsilon \sim \mathcal{N}(0, I)$$

This is one of the most important equations in diffusion.

It says:

A noisy sample at time t is a mixture of clean data and Gaussian noise.

When t is small:

$$\bar{\alpha}_t \approx 1$$

so:

$$x_t \approx x_0$$

The image is almost clean.

When t is large:

$$\bar{\alpha}_t \approx 0$$

so:

$$x_t \approx \epsilon$$

The image is almost pure noise.

The Chapter 3 summary emphasizes that this closed-form marginal is a key practical simplification because training can sample a random timestep directly instead of simulating the entire trajectory.

Chapter 10. **7. Trajectory Distribution vs Marginal Distribution**

This distinction is very important for the exam.

Chapter 11. **Trajectory distribution**

$$q(x_{1:T} \mid x_0)$$

This means the probability distribution over the full noising path:

$$x_1, x_2, \dots, x_T$$

given clean data x_0 .

It describes the entire trajectory.

Chapter 12. **Marginal distribution**

$$q(x_t \mid x_0)$$

This means the distribution of only one noisy state x_t , given x_0 .

The Chapter 3 notes explain that $q(x_{1:T} \mid x_0)$ is the joint probability density of the entire noisy trajectory, while $q(x_t \mid x_0)$ is the probability of one single noisy state at time t .

Chapter 13. **Why This Matters**

For training, we usually do not need the full trajectory.

We can sample:

1. a clean data point x_0 ,
2. a timestep t ,
3. noise ϵ ,
4. construct x_t directly using:

$$x_t = \sqrt{\tilde{\alpha}_t}x_0 + \sqrt{1 - \tilde{\alpha}_t}\epsilon$$

This makes training efficient.

Exam sentence:

The full trajectory is conceptually important for understanding diffusion as a Markov process, but the closed-form marginal $q(x_t | x_0)$ is what makes practical training efficient.

Chapter 14. 8. Why the Reverse Process Is Hard

The forward process is easy because we design it:

$$q(x_t | x_{t-1})$$

But generation requires the reverse:

$$q(x_{t-1} | x_t)$$

In principle, we want:

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$$

However, the true reverse conditional is not directly available.

By Bayes' rule:

$$q(x_{t-1} | x_t) = \frac{q(x_t | x_{t-1})q(x_{t-1})}{q(x_t)}$$

Even though:

$$q(x_t | x_{t-1})$$

is known and Gaussian, the marginal distributions:

$$q(x_{t-1}), q(x_t)$$

depend on the real data distribution, which is unknown and complex.

Therefore:

$$q(x_{t-1} | x_t)$$

is intractable for real images.

The notes clearly state that although the local forward corruption rule is simple and Gaussian, the exact reverse direction depends on the unknown data distribution and is not directly usable in realistic diffusion modeling.

Chapter 15. 9. The Learned Reverse Process

Because the true reverse is hard, we introduce a learned reverse model:

$$p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)$$

Usually:

$$p(x_T) = \mathcal{N}(0, I)$$

and each reverse step is modeled as Gaussian:

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_t)$$

where the neural network predicts information needed to compute the reverse mean.

The Chapter 3 notes say diffusion models solve the intractable reverse problem by introducing a learned reverse chain where each reverse conditional is parameterized by a neural network.

Chapter 16. **10. What Does the Neural Network Predict?**

There are several possible parameterizations.

The network can predict:

Prediction target	Meaning
$\epsilon_{\theta}(x_t, t)$	noise added to x_0
$\mu_{\theta}(x_t, t)$	reverse mean
$x_{0,\theta}(x_t, t)$	clean sample estimate
$s_{\theta}(x_t, t)$	score
$v_{\theta}(x_t, t)$	velocity-like target

In DDPM, the most common choice is to predict the noise:

$$\epsilon_{\theta}(x_t, t) \approx \epsilon$$

where:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

The Chapter 3 summary explains that predicting noise is principled because, once x_t , x_0 , and t are known, the unknown part is precisely the noise realization that corrupted the clean sample; estimating it tells the model how the sample was pushed away from the data structure.

Chapter 17. **11. Training Objective: Noise Prediction MSE**

The standard DDPM training objective is:

$$\mathcal{L}_{simple} = \mathbb{E}_{x_0, t, \epsilon} [\| \epsilon - \epsilon_{\theta}(x_t, t) \|^2]$$

Training procedure:

1. Sample clean data:

$$x_0 \sim q_{data}(x)$$

2. Sample timestep:

$$t \sim \{1, \dots, T\}$$

3. Sample noise:

$$\epsilon \sim \mathcal{N}(0, I)$$

4. Create noisy input:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

5. Train network to predict ϵ :

$$\epsilon_{\theta}(x_t, t) \approx \epsilon$$

6. Minimize MSE:

$$\| \epsilon - \epsilon_{\theta}(x_t, t) \|^2$$

The notes state that in DDPM, the network is trained with a simple MSE loss to predict the Gaussian noise added in the forward marginal.

Chapter 18. **12. Why Noise Prediction Is Not Arbitrary**

At first, it may seem strange:

Why train the model to predict noise instead of directly predicting the clean image?

But predicting noise is mathematically meaningful.

From:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

if the model predicts ϵ , then we can estimate x_0 :

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

So predicting noise tells the model how to denoise.

Also, noise prediction is connected to the **score**, which is the gradient of log-density:

$$\nabla_x \log p_t(x)$$

The Chapter 3 sample answer says predicting noise is tied to denoising and score estimation, since the score of a Gaussian-corrupted sample can be expressed in terms of the noise.

Chapter 4 later makes this link more explicit, explaining that noise prediction is closely tied to score estimation under the denoising-score identity.

Chapter 19. 13. Reverse Sampling Step

At generation time, we start from:

$$x_T \sim \mathcal{N}(0, I)$$

Then for:

$$t = T, T - 1, \dots, 1$$

we predict noise:

$$\epsilon_\theta(x_t, t)$$

and compute a reverse mean.

A common DDPM reverse mean in epsilon parameterization is:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

Then sample:

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z$$

where:

$$z \sim \mathcal{N}(0, I)$$

At the final step, no noise is added.

Chapter 4's code explanation states that the reverse loop performs: predict noise, remove predicted noise, add calibrated stochasticity, and repeat until a clean image is obtained.

Chapter 20. **14. Why the Reverse Process Is Stochastic**

Forward diffusion is stochastic because fresh Gaussian noise is added at every step.

The reverse process is also stochastic because many possible cleaner images could have produced a given noisy image.

So the reverse step cannot simply invert the forward process deterministically.

The Chapter 3 notes emphasize that diffusion is not trying to recover the exact original training example. Instead, it tries to sample a new x_0 from a learned distribution that matches the data distribution.

Important exam sentence:

Reverse diffusion is not exact inversion of one forward trajectory; it is probabilistic sampling from the learned reverse distribution.

Chapter 21. **15. Training vs Sampling**

This is a very important distinction.

Chapter 22. During training

We have real data x_0 . We choose a timestep and add known noise:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

Then the model learns:

$$\epsilon_\theta(x_t, t) \approx \epsilon$$

Chapter 23. During sampling

We do **not** have x_0 .

We start with:

$$x_T \sim \mathcal{N}(0, I)$$

and repeatedly use the learned model to move backward:

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$$

Exam sentence:

Training uses clean data to create supervised noisy examples, while sampling starts from noise and applies the learned reverse transitions without access to the original clean data.

Chapter 24. 16. Why Timestep Conditioning Is Essential

The denoising task changes depending on t .

At small t :

- image is mostly clean,

- only slight noise is present,
- model should preserve detail.

At large t :

- image is almost pure noise,
- little signal remains,
- model must infer global structure.

Therefore the model must know the timestep.

The network is written as:

$$\epsilon_{\theta}(x_t, t)$$

not just:

$$\epsilon_{\theta}(x_t)$$

The Chapter 3 summary explains that without time information, the denoising task becomes ambiguous because similar noisy observations can require different denoising operations depending on their signal-to-noise level.

Chapter 25. 17. Sinusoidal Time Embeddings

The timestep t is usually converted into a vector representation using sinusoidal embeddings.

This is similar to positional encoding in transformers.

The reason is:

A neural network needs a useful continuous/vector representation of time, not just an integer index.

A sinusoidal embedding maps t into multiple sine/cosine features at different frequencies.

Conceptually:

- low-frequency components capture broad time information,
- high-frequency components capture fine timestep differences.

The Chapter 3 summary says sinusoidal embeddings transform the discrete or continuous time index into a vector representation the network can use, and that time conditioning tells the model how much information should still be present.

Chapter 26. **18. Why U-Nets Are Used**

For image diffusion, the denoising network is often a **U-Net**.

A U-Net has:

1. **Downsampling path**
 - captures global context,
 - understands large-scale structure.
2. **Bottleneck**
 - compact deep representation.
3. **Upsampling path**
 - reconstructs image resolution,
 - restores detail.
4. **Skip connections**
 - pass spatial information from encoder to decoder,
 - preserve fine details.

Diffusion denoising needs both:

- global structure: what object/image should this become?
- local detail: edges, textures, pixels.

U-Nets are good because they combine both.

The professor's summary states that U-Nets are effective because downsampling captures large-scale context, upsampling reconstructs fine detail, and skip connections preserve spatial information across scales.

Exam sentence:

A U-Net matches the multiscale nature of image denoising because it combines global context with local spatial detail.

Chapter 27. **19. Why Diffusion Is an Iterative Inverse Problem**

Diffusion can be viewed as an inverse problem.

Forward process:

$$x_0 \rightarrow x_t$$

corrupts the data.

Reverse process:

$$x_t \rightarrow x_0$$

tries to undo the corruption.

But it cannot be solved exactly because information is destroyed by noise.

So the model learns a probabilistic denoising direction.

The Chapter 3 summary says diffusion is not only a probabilistic idea but also an iterative inverse problem, and U-Nets are strong inverse-problem solvers.

Chapter 28. **20. Diffusion Families**

The lecture notes explain that the term “diffusion model” covers several related families.

Chapter 29. **A. DDPM**

Denoising Diffusion Probabilistic Model

- discrete-time Gaussian Markov chain,
- stochastic reverse process,
- many steps,
- canonical diffusion framework,
- ELBO motivates the loss.

Learns:

$$\epsilon_{\theta}(x_t, t)$$

or:

$$\mu_{\theta}(x_t, t)$$

Chapter 30. **B. DDIM**

Denoising Diffusion Implicit Model

DDIM uses the same trained network as DDPM but changes the sampling process.

Training remains:

$$\mathbb{E} \parallel \epsilon - \epsilon_{\theta}(x_t, t) \parallel^2$$

Sampling can be more deterministic and much faster.

The notes state that DDIM is best understood as a sampling method for a model trained like DDPM: training is unchanged, but sampling uses a different reverse update that can be deterministic and faster.

Important:

DDIM shows that training and sampling are not identical. A model trained with a DDPM objective can be sampled in different ways.

Chapter 31. **C. Score-SDE Models**

Continuous-time diffusion models use stochastic differential equations.

Instead of integer timesteps:

$$t = 1, 2, \dots, T$$

they use continuous time:

$$t \in [0, 1]$$

They learn a score:

$$s_{\theta}(x, t) = \nabla_x \log p_t(x)$$

This connects diffusion to score matching.

Chapter 32. **D. Latent Diffusion Models**

Latent diffusion performs diffusion not in pixel space, but in latent space.

Instead of:

$$x_t \in \text{pixel space}$$

we use:

$$z_t \in \text{latent space}$$

This is faster and cheaper because latent space is lower-dimensional.

The notes list latent diffusion as diffusing in latent space and mention Stable Diffusion as a major example.

Chapter 33. **E. Guided Diffusion**

Guided diffusion modifies sampling to control outputs.

Examples:

- class-conditioned generation,
- text-conditioned generation,
- style-conditioned generation.

Guided diffusion is covered more deeply in Chapter 4.

Chapter 34. F. Discrete Diffusion

Discrete diffusion is used for categorical variables such as:

- text tokens,
- discrete image tokens,
- masks.

Instead of Gaussian noise, it uses discrete corruption such as masking or token replacement.

Chapter 35. G. Flow Matching / Rectified Flow

Flow matching is diffusion-adjacent but learns a deterministic vector field rather than a stochastic reverse Markov chain.

It will be Chapter 5.

Chapter 36. 21. DDPM vs DDIM

This is likely to appear in the exam.

Feature	DDPM	DDIM
Training	noise prediction MSE	same as DDPM
Sampling	stochastic reverse chain	deterministic or low-noise trajectory
Speed	many steps	fewer steps possible
Diversity	more stochastic	less stochastic if deterministic
Main idea	probabilistic reverse process	alternative sampler

The Chapter 3 summary says DDPM typically uses a stochastic reverse process, while DDIM changes the sampling procedure while keeping the same training objective and can allow faster sampling with fewer steps.

Exam sentence:

DDIM is not a different training objective; it is mainly a different sampling method for a diffusion model trained like DDPM.

Chapter 37. **22. Variance-Preserving View**

In DDPM, the forward process is often called **variance preserving** because the construction keeps the overall scale controlled.

The closed-form marginal:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

has coefficients where:

$$\bar{\alpha}_t + (1 - \bar{\alpha}_t) = 1$$

So signal decreases while noise increases.

This is why the model gradually transitions from data to noise without exploding variance.

Chapter 38. **23. What Information Exists at Different Timesteps?**

This is conceptually important.

Chapter 39. **Early timesteps**

$$t \approx 0$$

- mostly clean data,
- high signal-to-noise ratio,
- details visible,
- denoising is local and small.

Chapter 40. **Middle timesteps**

- structure still exists,
- details are partially destroyed,
- model must infer missing features.

Chapter 41. Late timesteps

$$t \approx T$$

- mostly noise,
- low signal-to-noise ratio,
- semantic information is weak,
- model must infer global structure.

The timestep tells the model what denoising regime it is in.

Chapter 42. 24. Entropy and Information Destruction

Forward diffusion gradually destroys information.

As noise increases:

- entropy increases,
- data structure becomes smoother,
- mutual information between x_t and x_0 decreases,
- x_T approaches Gaussian noise.

The notes summarize the forward process as progressively adding Gaussian noise, increasing entropy, smoothing the data distribution, and pushing everything toward a simple Gaussian.

Chapter 43. 25. Why Diffusion Models Are Powerful

Diffusion models are powerful because they combine:

1. a simple known forward process,
2. a learned reverse process,
3. tractable noisy training pairs,
4. stable MSE training,
5. flexible neural networks,
6. iterative refinement,
7. strong image architectures like U-Nets.

The Chapter 3 summary states that diffusion models can be understood as latent-variable models over a long trajectory, learned denoisers, approximate reverse-time stochastic processes, and score-based generative models under appropriate parameterization.

Chapter 44. 26. Diffusion Compared with VAE

Concept	VAE	Diffusion
Latent variable	low-dimensional z	full trajectory $x_1, \dots, x_T$
Generation	sample z , decode once	sample x_T , denoise step by step
Training objective	ELBO with reconstruction + KL	noise prediction MSE / ELBO-derived
Main challenge	posterior approximation	reverse process approximation
Sampling speed	usually fast	often slower
Output quality	may be blurry	often high quality

The Chapter 3 summary notes that unlike a VAE, where one introduces a low-dimensional latent variable and decodes from it, diffusion uses a trajectory as its latent structure.

Chapter 45. 27. Diffusion Compared with GANs

Concept	GAN	Diffusion
Training	adversarial	denoising MSE / likelihood-based
Stability	often unstable	usually stable
Sampling	one forward pass	many denoising steps
Quality	sharp but unstable	high quality and diverse
Mode collapse	common risk	less severe
Objective	generator vs discriminator	reverse process learning

Diffusion models became popular partly because they avoid many GAN training instabilities.

Chapter 46. 28. Full Training Algorithm

A practical DDPM training loop:

1. Sample clean data:

$$x_0 \sim q_{data}$$

2. Sample timestep:

$$t \sim \text{Uniform}(1, T)$$

3. Sample noise:

$$\epsilon \sim \mathcal{N}(0, I)$$

4. Create noisy input:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

5. Predict noise:

$$\hat{\epsilon} = \epsilon_{\theta}(x_t, t)$$

6. Compute loss:

$$\mathcal{L} = \| \epsilon - \hat{\epsilon} \|^2$$

7. Backpropagate and update θ .

Chapter 47. **29. Full Sampling Algorithm**

A practical DDPM sampling loop:

1. Start with pure noise:

$$x_T \sim \mathcal{N}(0, I)$$

2. For $t = T, T - 1, \dots, 1$:
 - predict noise:

$$\epsilon_{\theta}(x_t, t)$$

- compute reverse mean:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

- sample:

$$x_{t-1} = \mu_{\theta}(x_t, t) + \sigma_t z$$

- where:

$$z \sim \mathcal{N}(0, I)$$

3. At $t = 0$, output the final sample.

Chapter 4's reverse sampling notes explain the same loop: initialize from pure noise, iterate backwards in time, predict noise, remove predicted noise, add calibrated Gaussian noise, and stop with a clean sample.

Chapter 48. **30. Common Notation**

Chapter 49. x_0

Clean real data.

Example: original MNIST digit or real image.

Chapter 50. x_t

Noisy version of x_0 at timestep t .

Chapter 51. x_T

Almost pure Gaussian noise.

Chapter 52. q

Forward noising process. Fixed, not learned.

Chapter 53. p_θ

Learned reverse generative process.

Chapter 54. ϵ

Actual Gaussian noise added to create x_t .

Chapter 55. $\epsilon_\theta(x_t, t)$

Neural network prediction of the noise.

Chapter 56. β_t

Noise added at step t .

Chapter 57. α_t

Signal retained at step t :

$$\alpha_t = 1 - \beta_t$$

Chapter 58. $\bar{\alpha}_t$

Cumulative product:

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

This measures how much original signal remains at time t .

Chapter 59. **31. Most Important Formula Summary**

Chapter 60. **Forward chain**

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

Chapter 61. **Forward transition**

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I)$$

Chapter 62. **Closed-form marginal**

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

Chapter 63. **Direct noisy sample**

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

Chapter 64. **Learned reverse chain**

$$p_{\theta}(x_{0:T}) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1} | x_t)$$

Chapter 65. Reverse conditional

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \Sigma_t)$$

Chapter 66. Noise prediction objective

$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} [\| \epsilon - \epsilon_{\theta}(x_t, t) \|^2]$$

Chapter 67. Reverse mean in epsilon form

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

Chapter 68. 32. Most Likely Exam Questions

Chapter 69. Question 1: What is the main idea of diffusion models?

A diffusion model learns to generate data by reversing a gradual corruption process. During the forward process, clean data x_0 is progressively corrupted into Gaussian noise x_T . During generation, the model starts from $x_T \sim \mathcal{N}(0, I)$ and learns reverse transitions that gradually denoise the sample until it resembles data. The key intuition is that many small denoising steps are easier to learn than one direct mapping from noise to data.

Chapter 70. Question 2: Why is the forward process easy but the reverse process hard?

The forward process is easy because it is designed by us as a Gaussian Markov chain. Each transition $q(x_t | x_{t-1})$ is known and easy to sample. The reverse process $q(x_{t-1} | x_t)$ is hard

because it depends on the unknown data distribution through the marginals $q(x_t)$ and $q(x_{t-1})$. Therefore, diffusion models learn an approximation $p_\theta(x_{t-1} | x_t)$ using a neural network.

Chapter 71. Question 3: What is the difference between $q(x_{1:T} | x_0)$ and $q(x_t | x_0)$?

$$q(x_{1:T} | x_0)$$

is the distribution over the entire noising trajectory from x_1 to x_T given clean x_0 . By contrast:

$$q(x_t | x_0)$$

is only the marginal distribution of one noisy state at timestep t . This distinction matters because training usually only needs one randomly sampled noisy state x_t , and the closed-form marginal makes that efficient.

Chapter 72. Question 4: Why does predicting noise make sense?

Predicting noise makes sense because the forward marginal can be written as:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

Given x_t , x_0 , and t , the unknown corruption is exactly ϵ . If the model can predict ϵ , it knows how to remove the corruption and estimate the clean structure. Noise prediction is also mathematically connected to denoising and score estimation, so it is not arbitrary.

Chapter 73. Question 5: Why must the neural network know the timestep t ?

The appropriate denoising operation depends on the noise level. At small t , x_t is almost clean, so denoising should be gentle. At large t , x_t may be almost pure noise, so denoising requires more global inference. Without timestep information, the same input pattern could correspond to different denoising regimes. Therefore the timestep is part of the state of the reverse problem.

Chapter 74. Question 6: Why are U-Nets commonly used in image diffusion?

U-Nets are well-suited to image denoising because they combine global and local information. The downsampling path captures large-scale structure, the upsampling path reconstructs fine detail, and skip connections preserve spatial information across resolutions. This matches the multiscale nature of reverse diffusion.

Chapter 75. Question 7: What is the role of the noise schedule?

The noise schedule controls how quickly information is destroyed during the forward process. It shapes the signal-to-noise ratio at each timestep. If noise is added too quickly, reverse denoising becomes difficult. If noise is added too slowly, training and sampling may be inefficient. Different schedules, such as linear and cosine schedules, allocate information destruction differently and affect training stability and sample quality.

Chapter 76. Question 8: What is the difference between DDPM and DDIM?

DDPM uses a stochastic reverse process and usually requires many sampling steps. DDIM keeps the same training objective but changes the sampling procedure, often making it more deterministic and much faster. The key idea is that a model trained with a DDPM noise-prediction objective can be sampled using different reverse procedures.

Chapter 77. Question 9: Why do we not recover the exact original x_0 during reverse sampling?

Because the forward process is stochastic and destroys information. Many possible clean samples could lead to a given noisy state. Reverse diffusion is not exact inversion of one trajectory. It samples a plausible clean data point from the learned data distribution.

Chapter 78. Question 10: How is diffusion different from a VAE?

A VAE uses a low-dimensional latent variable z and generates data by decoding it once. A diffusion model uses a long trajectory of noisy variables $x_1, \dots, x_T$ and generates data by gradually reversing noise. The latent structure in diffusion is the entire noising trajectory rather than a single compact latent code.

Chapter 79. **33. Common Mistakes to Avoid**

Chapter 80. **Mistake 1: Thinking the forward process is learned**

Wrong. The forward process is fixed and designed by us.

The reverse process is learned.

Chapter 81. **Mistake 2: Thinking diffusion directly maps noise to image in one step**

Wrong. Standard diffusion uses many iterative denoising steps.

Chapter 82. **Mistake 3: Confusing x_t and x_T**

x_t is a noisy sample at an arbitrary timestep.

x_T is the final almost pure noise state.

Chapter 83. **Mistake 4: Forgetting timestep conditioning**

A denoiser without t does not know the current noise level.

Chapter 84. **Mistake 5: Thinking noise prediction is just a trick**

Noise prediction is connected to the Gaussian forward process, denoising direction, and score estimation.

Chapter 85. **Mistake 6: Thinking DDIM requires retraining**

DDIM can use the same network trained with DDPM-style noise prediction. It mainly changes sampling.

Chapter 86. **Mistake 7: Thinking reverse diffusion recovers the training image**

Generation produces a new sample from the learned data distribution, not necessarily the original x_0 .

Chapter 87. **34. One-Page Open-Note Summary**

A diffusion model is a generative model that learns to reverse a gradual noising process. The forward process is a fixed Markov chain that corrupts clean data x_0 into noisy states $x_1, \dots, x_T$, where x_T is approximately Gaussian noise. In DDPM, the forward transition is Gaussian, and the closed-form marginal is $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$. This formula makes training efficient because we can sample any timestep directly. The reverse process is hard because the true reverse conditional depends on the unknown data distribution, so a neural network learns $p_\theta(x_{t-1} | x_t)$. In common DDPM training, the network predicts the added noise ϵ from x_t and timestep t , using MSE loss. Timestep conditioning is essential because denoising at low noise and high noise are very different tasks. U-Nets are commonly used for image diffusion because they combine global context, local detail, and skip connections. Generation starts from $x_T \sim \mathcal{N}(0, I)$ and repeatedly applies learned reverse denoising steps. DDPM sampling is stochastic and often slow, while DDIM changes the sampling procedure to allow faster and more deterministic generation without changing the basic training objective.

Chapter 88. **35. Very Short Final Revision**

Know these:

- Diffusion = learn to reverse gradual noise.
- Forward process: $x_0 \rightarrow x_T$, fixed, Gaussian, not learned.
- Reverse process: $x_T \rightarrow x_0$, learned.

- Markov property: each step depends only on previous step.
- $q(x_{1:T} | x_0)$ = full trajectory.
- $q(x_t | x_0)$ = one noisy marginal.
- Closed-form: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$.
- Network predicts $\epsilon_\theta(x_t, t)$.
- Loss: $\mathbb{E} \| \epsilon - \epsilon_\theta(x_t, t) \|^2$.
- Timestep t is essential because noise level changes.
- U-Net is used because denoising needs global structure + local detail.
- DDPM = stochastic many-step sampler.
- DDIM = alternative faster sampler using same trained model.
- Diffusion does not invert one exact trajectory; it samples from the learned data distribution.

Chapter 89. Chapter 4 — Advanced Diffusion Concepts

Chapter 3 explained the basic diffusion process: add noise forward, then learn to denoise backward.

Chapter 4 goes deeper and explains **why the diffusion objective works, what the model is really learning, and how diffusion becomes controllable.**

The main ideas in Chapter 4 are:

1. Diffusion training and the **ELBO**
2. The **score function**
3. Reverse diffusion as **score step + calibrated noise**
4. **Guided diffusion**
5. **Classifier guidance**
6. **Classifier-free guidance**
7. Why we need **noise-aware classifiers**
8. The guidance tradeoff
9. Evaluation using **FID**

Chapter 90. 1. Big Picture of Chapter 4

In Chapter 3, we said the model predicts noise:

$$\epsilon_\theta(x_t, t) \approx \epsilon$$

and trains with:

$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} [\| \epsilon - \epsilon_\theta(x_t, t) \|^2]$$

Chapter 4 explains that this MSE loss is not just a random engineering trick. It comes from the probabilistic structure of diffusion models and can be derived from a variational objective. The chapter summary states that once the forward process is Gaussian and the reverse variance is fixed appropriately, the diffusion KL terms reduce to an error on the reverse mean, which after reparameterization becomes the familiar noise-prediction MSE.

So Chapter 4 answers:

Why does predicting noise work theoretically?

and:

How can we guide diffusion to generate what we want?

Chapter 91. 2. Diffusion as Variational Learning

In a generative model, the ideal goal is to maximize data likelihood:

$$\log p_\theta(x_0)$$

But in diffusion, the model has many hidden noisy states:

$$x_1, x_2, \dots, x_T$$

So the likelihood involves a long latent trajectory. Directly computing it is hard.

Just like in VAEs, diffusion uses a lower bound:

$$\log p_\theta(x_0) \geq \text{ELBO}$$

The diffusion ELBO decomposes into terms such as:

1. reconstruction-like term near x_0 ,
2. intermediate KL terms between true and learned reverse transitions,
3. prior-matching term at x_T .

The Chapter 4 summary says the ELBO decomposes into reconstruction-like, prior-matching, and intermediate consistency terms.

Chapter 92. **2.1 What the Diffusion ELBO Is Trying to Do**

The diffusion ELBO forces the learned reverse process:

$$p_{\theta}(x_{t-1} | x_t)$$

to match the true reverse posterior:

$$q(x_{t-1} | x_t, x_0)$$

At every step, the model should learn:

Given a noisy sample x_t , what is the correct distribution over the slightly cleaner sample x_{t-1} ?

The problem is that the full likelihood is hard, but the ELBO gives us a tractable objective.

Chapter 93. **3. Why the MSE Noise Prediction Loss Appears**

This is a very important exam point.

In DDPM, the true forward posterior is Gaussian:

$$q(x_{t-1} | x_t, x_0)$$

The learned reverse transition is also modeled as Gaussian:

$$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(\mu_{\theta}(x_t, t), \Sigma_t)$$

If the variance is fixed properly, then matching two Gaussians mainly becomes matching their means.

So the KL term becomes roughly:

$$\| \mu_q - \mu_\theta \|^2$$

Then, because the reverse mean can be parameterized using the predicted noise $\epsilon_\theta(x_t, t)$, this becomes:

$$\| \epsilon - \epsilon_\theta(x_t, t) \|^2$$

So the familiar training objective:

$$\mathcal{L}_{simple} = \mathbb{E}_{x_0, t, \epsilon} [\| \epsilon - \epsilon_\theta(x_t, t) \|^2]$$

is connected to the ELBO.

The summary states clearly that the commonly used MSE objective is not arbitrary; it is a tractable surrogate that emerges from the probabilistic structure of the model.

Chapter 94. Exam Explanation

If asked:

“Why does diffusion use MSE noise prediction?”

Answer:

Diffusion training can be derived from the ELBO. The intermediate terms compare the true reverse posterior with the learned reverse transition. Since both are Gaussian and the variance is fixed, matching these distributions reduces mainly to matching their means. When the reverse mean is written in terms of the predicted noise, this becomes an MSE loss between the true added noise ϵ and the predicted noise $\epsilon_\theta(x_t, t)$. Therefore, noise-prediction MSE is a principled simplification of the variational objective.

Chapter 95. 4. What Is a Score?

The **score** of a distribution is:

$$s(x) = \nabla_x \log p(x)$$

For diffusion at time t :

$$s_t(x_t) = \nabla_{x_t} \log p_t(x_t)$$

It is the gradient of the log-density.

Simple meaning:

The score points in the direction where probability density increases fastest.

If you are at a noisy point x_t , the score tells you:

Move this way to become more likely under the noisy data distribution.

The Chapter 4 summary describes the score as the direction of steepest increase in log-density, telling us how to move a noisy sample toward a more likely region of the data distribution at the current noise level.

Chapter 96. 5. Score Intuition with a Gaussian

Suppose:

$$p(x) = \mathcal{N}(0,1)$$

Then:

$$\log p(x) = -\frac{x^2}{2} + C$$

The score is:

$$\nabla_x \log p(x) = -x$$

So:

- if $x > 0$, score is negative $\rightarrow$ pushes left,
- if $x < 0$, score is positive $\rightarrow$ pushes right,
- score always pushes toward the mean 0.

Meaning:

The score pulls samples toward high-density regions.

For a Gaussian, the highest-density region is the mean.

Chapter 97. **6. Score Intuition with Multimodal Data**

Suppose the data has two modes:

- left mode,
- right mode.

The score field points toward the nearest/highest local mode.

So if a sample is between modes, the score tells it where probability density increases.

This matters for diffusion because reverse sampling is basically following a learned probability geometry.

The Chapter 4 summary says that for a unimodal Gaussian, the score points toward the mean, while for a multimodal distribution, it points toward the locally most plausible mode.

Chapter 98. **7. Connection Between Noise Prediction and Score**

In Chapter 3, we trained:

$$\epsilon_{\theta}(x_t, t)$$

to predict noise.

In Chapter 4, we reinterpret this as learning a score-like direction.

For Gaussian noising, there is a relationship:

$$\nabla_{x_t} \log p_t(x_t) \approx -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t)$$

This means:

Predicting noise is closely related to estimating the score.

The model is not just learning “noise.” It is learning how to move a noisy sample toward the data distribution.

The Chapter 4 summary says that in DDPM-style training, the network predicts noise, but this prediction is closely tied to the score through the denoising-score identity; therefore, the model is effectively learning a score field even when trained as a noise predictor.

Chapter 99. Exam Explanation

If asked:

“What is the score perspective in diffusion?”

Answer:

The score perspective views diffusion models as learning the gradient of the log-density at each noise level. This score tells the sampler how to move a noisy sample toward higher-probability regions. Although DDPMs are trained to predict noise, the predicted noise is mathematically related to the score under Gaussian corruption. Therefore, noise prediction can be interpreted as learning a score field for denoising.

Chapter 100. 8. Reverse Diffusion as Drift + Noise

Reverse diffusion has two components:

1. deterministic drift,
2. stochastic noise.

A simplified reverse update looks like:

$$x_{t-1} = \text{drift step} + \text{random noise}$$

The drift step moves the sample toward high probability regions.

The random noise keeps sampling stochastic and prevents collapse.

Chapter 4 emphasizes that reverse diffusion is neither just denoising nor just random sampling. It is a calibrated stochastic transport process.

Chapter 101. Why the Random Term Is Important

Without the random term, reverse diffusion would become like pure gradient ascent toward high-density modes.

This could cause:

- reduced diversity,
- mode collapse,
- overly deterministic outputs,
- samples all looking similar.

The stochastic term preserves sampling behavior.

So reverse diffusion is:

guided motion toward data + controlled randomness.

Chapter 102. 9. Why Guided Diffusion Is Needed

Unconditional diffusion learns:

$$p_{\theta}(x)$$

It can generate plausible samples, but we cannot directly control what we get.

Example:

If trained on MNIST, unconditional diffusion may generate random digits:

$$0, 1, 2, \dots, 9$$

But what if we want only digit 3?

We need:

$$p_{\theta}(x \mid y = 3)$$

For text-to-image generation, we want:

$$p_{\theta}(x \mid \text{text prompt})$$

For style control, we want:

$$p_{\theta}(x \mid \text{style})$$

Guided diffusion modifies the reverse process so samples are not only realistic, but also match the condition.

The Chapter 4 notes summarize this beautifully: unconditional diffusion answers “What images are plausible?” while guided diffusion answers “What images are plausible and match this condition?”

Chapter 103. 10. Why Conditioning Is Hard in Diffusion

In a normal classifier or autoregressive model, conditioning is easy:

$$p(y \mid x, c)$$

You feed condition c into the model.

But in diffusion, generation happens over many reverse steps:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$$

So the condition must influence the whole reverse trajectory.

It is not enough to apply the condition at the end.

The Chapter 4 notes say that in diffusion, generation happens through a stochastic reverse process, so conditional generation must modify the score field itself.

Chapter 104. **11. Guided Diffusion as Modified Score Field**

Unconditional diffusion follows the score:

$$\nabla_x \log p(x)$$

Guided diffusion wants to follow:

$$\nabla_x \log p(x | y)$$

Using Bayes' rule:

$$p(x | y) = \frac{p(y | x)p(x)}{p(y)}$$

Taking log:

$$\log p(x | y) = \log p(x) + \log p(y | x) - \log p(y)$$

Now take gradient with respect to x :

$$\nabla_x \log p(x | y) = \nabla_x \log p(x) + \nabla_x \log p(y | x)$$

because:

$$\nabla_x \log p(y) = 0$$

So:

$$\text{conditional score} = \text{unconditional score} + \text{classifier gradient}$$

This is the theoretical basis of classifier guidance.

The Chapter 4 summary states that classifier guidance follows from Bayes’ rule, where the unconditional score comes from the diffusion model and the classifier gradient encourages condition alignment.

Chapter 105. 12. Classifier Guidance

Classifier guidance uses two models:

- 1. an unconditional diffusion model,
- 2. a classifier.

The diffusion model gives:

$$\nabla_x \log p_t(x_t)$$

The classifier gives:

$$\nabla_{x_t} \log p_\phi(y \mid x_t, t)$$

Then we combine:

$$\nabla_x \log p_t(x_t \mid y) = \nabla_x \log p_t(x_t) + w \nabla_x \log p_\phi(y \mid x_t, t)$$

where:

Symbol	Meaning
w	guidance strength
$p_\phi(y \mid x_t, t)$	
$(\nabla_x \log p_\phi(y \mid x_t, t))$	

The Chapter 4 notes explain that the diffusion score ensures realism, while the classifier gradient ensures condition alignment.

Chapter 106. Simple Intuition

Suppose the diffusion model says:

Move toward realistic MNIST digits.

The classifier says:

Move toward images that look like digit 3.

Together:

Move toward realistic MNIST digits that look like 3.

Chapter 107. **13. Guidance Strength w**

The guidance scale w controls how strongly the condition affects sampling.

If:

$$w = 0$$

then generation is unconditional.

If:

$$w > 0$$

the sample is pushed toward the condition.

If w is too large:

- condition adherence improves,
- diversity decreases,
- artifacts may appear,
- samples may become unnatural.

The chapter summary says increasing guidance strength often improves condition adherence but can reduce diversity, amplify artifacts, and make the sampler brittle.

Chapter 108. 14. Why We Need a Noise-Aware Classifier

In classifier guidance, the classifier is not asked to classify clean images only.

During reverse diffusion, it sees noisy states:

$$x_t$$

At large t , these images may be heavily corrupted.

A normal classifier trained only on clean images:

$$p_\phi(y \mid x_0)$$

may fail on:

$$x_t$$

because noisy images look very different.

Therefore we train a noise-aware classifier:

$$p_\phi(y \mid x_t, t)$$

It receives:

1. noisy image x_t ,
2. timestep t .

The Chapter 4 notes state that the reverse process visits noisy states x_t , not clean images x_0 , so a classifier trained only on clean images may produce unstable or meaningless gradients at large timesteps; a noise-aware classifier learns how class information appears at every noise level.

Chapter 109. What the Noise-Aware Classifier Learns

For each labeled clean image (x_0, y) :

1. sample timestep t ,
2. sample noise ϵ ,
3. create:

$$x_t = \sqrt{\tilde{\alpha}_t}x_0 + \sqrt{1 - \tilde{\alpha}_t}\epsilon$$

4. train classifier to predict:

y

from:

(x_t, t)

Training objective:

$$\mathcal{L}_{clf} = -\log p_\phi(y \mid x_t, t)$$

This teaches the classifier:

Given a noisy diffusion state and timestep, what class is it likely to become?

Chapter 110. **15. Why Timestep Input Matters for the Classifier**

The classifier must know the noise level.

At small t :

- image is almost clean,
- class evidence is visible.

At large t :

- image is very noisy,
- class evidence is weak.

The same pixel pattern can mean different things at different noise levels.

So the classifier should be:

$$p_{\phi}(y \mid x_t, t)$$

not just:

$$p_{\phi}(y \mid x_t)$$

The Chapter 4 notes explicitly say the correct model is time-conditioned because the same noisy image intensity pattern can mean different things at different timesteps.

Chapter 111. **16. Limitations of Classifier Guidance**

Classifier guidance is powerful, but it has problems.

Chapter 112. **Problems**

1. Requires a separate classifier.
2. Classifier must work on noisy inputs.
3. Sampling is slower because each reverse step requires classifier gradient.
4. Classifier gradients can be unstable.
5. Strong guidance can reduce diversity.
6. Strong guidance can create artifacts.
7. Requires labels or conditions for classifier training.

The Chapter 4 notes list these limitations: classifier guidance requires a separate classifier, the classifier must be trained on noisy diffusion states, and sampling is slower because gradients are computed at each reverse step.

Chapter 113. **17. Classifier-Free Guidance — CFG**

Classifier-free guidance avoids using a separate classifier.

Instead, we train one diffusion model to work in two modes:

1. conditional mode:

$$\epsilon_{\theta}(x_t, t, y)$$

2. unconditional mode:

$$\epsilon_{\theta}(x_t, t, \emptyset)$$

where:

$$\emptyset$$

means null condition.

During training, we randomly drop the condition with probability p_{drop} .

The Chapter 4 notes say CFG trains a conditional diffusion model while randomly replacing the condition with a null condition, so the same network learns both conditional and unconditional predictions.

Chapter 114. 18. How CFG Training Works

For labeled data (x_0, y) :

1. sample timestep t ,
2. create noisy input x_t ,
3. randomly decide:

With probability $1 - p_{drop}$:

$$y_{in} = y$$

With probability p_{drop} :

$$y_{in} = \emptyset$$

4. train the model:

$$\epsilon_{\theta}(x_t, t, y_{in}) \approx \epsilon$$

Loss:

$$\mathcal{L} = \mathbb{E}[\| \epsilon - \epsilon_{\theta}(x_t, t, y_{in}) \|^2]$$

The Chapter 4 code explanation states that random condition dropping is the key to CFG because the model learns both conditional and unconditional predictions in one network.

Chapter 115. 19. CFG Sampling Formula

At sampling time, evaluate the model twice:

Conditional prediction:

$$\epsilon_{cond} = \epsilon_{\theta}(x_t, t, y)$$

Unconditional prediction:

$$\epsilon_{uncond} = \epsilon_{\theta}(x_t, t, \emptyset)$$

Combine:

$$\epsilon_{CFG} = (1 + w)\epsilon_{cond} - w\epsilon_{uncond}$$

Equivalent form:

$$\epsilon_{CFG} = \epsilon_{uncond} + (1 + w)(\epsilon_{cond} - \epsilon_{uncond})$$

Different sources use slightly different parameterizations of w , but the idea is the same:

amplify the difference between conditional and unconditional predictions.

The Chapter 4 notes explain that $w = 0$ gives the pure conditional model, while larger w amplifies what makes the conditional prediction different from the unconditional prediction; stronger guidance improves conditioning but reduces diversity.

Chapter 116. **20. CFG Intuition**

Unconditional prediction asks:

What would be a plausible image?

Conditional prediction asks:

What would be a plausible image matching the condition?

The difference:

$$\epsilon_{cond} - \epsilon_{uncond}$$

represents:

the direction added by the condition.

CFG amplifies that direction.

So if the condition is:

digit 3

CFG pushes the sample harder toward digit 3.

If the condition is:

“red fox sitting in snow”

CFG pushes the image toward fox + red + snow.

Chapter 117. **21. Why Random Condition Dropping Is Essential**

If the model is never trained unconditionally, then:

$$\epsilon_{\theta}(x_t, t, \emptyset)$$

is meaningless.

Then the CFG subtraction has no useful reference.

Random condition dropping teaches the model a true unconditional branch.

The Chapter 4 sample answer states that random dropping is essential because without it, the model would never learn a true unconditional branch, and the subtraction-based guidance mechanism would have no meaningful reference point.

Chapter 118. **22. Classifier Guidance vs Classifier-Free Guidance**

Feature	Classifier Guidance	Classifier-Free Guidance
Uses external classifier?	Yes	No
Needs noisy classifier?	Yes	No
Training model	unconditional diffusion + classifier	one conditional model with condition dropout
Sampling cost	classifier gradient each step	two forward passes
Flexibility	can retrofit unconditional model	must train conditionally
Stability	can be brittle	usually more stable
Used in modern text-to-image	less common	very common

The Chapter 4 notes say classifier guidance can retrofit guidance onto an existing unconditional model but needs a noise-robust classifier and slower backprop-based sampling, while CFG requires conditional training but avoids an external classifier and uses forward passes only.

Chapter 119. **23. The Guidance Tradeoff**

Guidance is not a free improvement.

Increasing guidance strength usually gives:

Chapter 120. **Benefits**

- better class alignment,
- better prompt adherence,
- stronger condition control.

Chapter 121. **Costs**

- less diversity,
- more artifacts,
- less natural samples,
- possible oversaturation,
- brittle sampling.

The Chapter 4 summary says stronger guidance shrinks the effective set of plausible outputs because trajectories that do not strongly support the condition are suppressed.

Chapter 122. **Example**

Prompt:

“A red fox in snow.”

Low guidance:

- may generate realistic animal,
- but not clearly red fox or snow.

Medium guidance:

- good fox,
- snow visible,
- realistic image.

Too high guidance:

- oversaturated red,
- unnatural shape,
- repeated artifacts,
- less diversity.

Exam sentence:

Guidance strength controls a realism–control–diversity tradeoff.

Chapter 123. **24. Hyperparameters in Guided Diffusion**

Important hyperparameters include:

1. guidance strength w ,
2. number of sampling steps,
3. noise schedule,
4. classifier robustness,
5. condition dropout probability in CFG,
6. model capacity,
7. conditioning embedding quality,
8. variance schedule.

The Chapter 4 summary notes that guidance scale, number of diffusion steps, noise schedule, classifier robustness, and condition-drop probability all affect the tradeoff among realism, diversity, controllability, and compute cost.

Chapter 124. **25. FID — Fréchet Inception Distance**

FID is an evaluation metric for generative models.

It compares generated samples and real samples in a deep feature space, usually using an Inception network.

Instead of comparing raw pixels, it compares feature distributions.

The real samples produce features with:

$$(\mu_r, \Sigma_r)$$

Generated samples produce features with:

$$(\mu_g, \Sigma_g)$$

FID compares these two Gaussian approximations.

Formula:

$$FID = \| \mu_r - \mu_g \|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

where:

Term	Meaning
μ_r	mean of real features
μ_g	mean of generated features
Σ_r	covariance of real features
Σ_g	covariance of generated features

lower FID generated distribution closer to real distribution

The Chapter 4 summary says FID compares real and generated samples in a deep feature space and captures differences in both mean and covariance of feature distributions.

Chapter 125. **26. What FID Measures Conceptually**

FID asks:

Do generated samples have similar feature statistics to real samples?

A lower FID means:

- generated images look more like real images in feature space,
- feature means are close,
- feature covariances are close.

It measures both:

1. quality,
2. diversity.

Why diversity?

Because covariance captures spread. If generated samples collapse to a few modes, covariance may differ from real data.

Chapter 126. **27. Limitations of FID**

FID is useful but imperfect.

Limitations:

1. depends on feature extractor,
2. assumes Gaussian feature distributions,
3. sensitive to sample size,
4. can be numerically unstable,
5. may not measure prompt alignment,
6. may not capture fine details,
7. may not match human judgment,
8. not sufficient for multimodal alignment.

The Chapter 4 summary says FID is only a proxy and depends on feature extractor, sample size, domain match, and numerical estimation; diffusion evaluation should not rely on a single metric.

Chapter 127. **28. Important Conceptual Map**

Chapter 4 connects several views of diffusion:

View	Meaning
Variational learning	explains why the objective has ELBO/MSE form
Score matching	explains what the model is learning geometrically
Reverse diffusion	explains how sampling transports noise to data
Guidance	explains how control is inserted
Evaluation	explains how generated quality is assessed

The Chapter 4 summary states that the chapter links these viewpoints into one coherent picture: variational learning explains the objectives, score matching explains what the network learns, reverse diffusion explains sampling, guidance explains control, and evaluation clarifies success.

Chapter 128. **29. Important Formula Summary**

Chapter 129. **Diffusion MSE objective**

$$\mathcal{L}_{simple} = \mathbb{E}_{x_0, t, \epsilon} [\| \epsilon - \epsilon_{\theta}(x_t, t) \|^2]$$

Chapter 130. Score

$$s_t(x) = \nabla_x \log p_t(x)$$

Chapter 131. Noise-score relation

$$\nabla_{x_t} \log p_t(x_t) \approx -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t)$$

Chapter 132. Classifier guidance

$$\nabla_x \log p_t(x_t | y) = \nabla_x \log p_t(x_t) + w \nabla_x \log p_\phi(y | x_t, t)$$

Chapter 133. CFG prediction

$$\epsilon_{CFG} = (1 + w)\epsilon_{cond} - w\epsilon_{uncond}$$

Chapter 134. FID

$$FID = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Chapter 135. 30. Most Likely Exam Questions

Chapter 136. Question 1: Why does the diffusion MSE objective appear?

Diffusion training can be derived from an ELBO. The intermediate ELBO terms compare the true reverse posterior with the learned reverse transition. When both are Gaussian and the reverse variance is fixed, matching them reduces to matching their means. Since the reverse mean can be parameterized through predicted noise, this becomes an MSE loss between the true noise and predicted noise. Therefore, noise-prediction MSE is a principled surrogate for variational learning, not an arbitrary trick.

Chapter 137. Question 2: What is the score in diffusion?

The score is:

$$\nabla_x \log p_t(x)$$

It is the direction of steepest increase in log-density at noise level t . Conceptually, it tells a noisy sample how to move toward a more likely region of the data distribution. In DDPM, the network may be trained to predict noise, but this noise prediction is mathematically related to the score, so the model is effectively learning a score-like field.

Chapter 138. Question 3: Why is reverse diffusion both deterministic and stochastic?

The deterministic part, or drift, moves the sample toward higher-probability regions and performs denoising. The stochastic part adds calibrated noise, preserving the sampling nature of the reverse process and preventing collapse into pure mode-seeking behavior. Reverse diffusion is therefore best understood as calibrated stochastic transport, not simple denoising alone.

Chapter 139. Question 4: What is the theoretical basis of classifier guidance?

Classifier guidance follows from Bayes' rule:

$$p(x | y) \propto p(y | x)p(x)$$

Taking gradients:

$$\nabla_x \log p(x | y) = \nabla_x \log p(x) + \nabla_x \log p(y | x)$$

The diffusion model provides the unconditional score $\nabla_x \log p(x)$, while the classifier provides the condition gradient $\nabla_x \log p(y | x)$. Adding them guides samples toward realistic images that also match the condition.

Chapter 140. Question 5: Why is a noise-aware classifier needed?

Classifier-guided diffusion queries the classifier on noisy states x_t , not just clean images x_0 . A classifier trained only on clean images may behave unpredictably on heavily corrupted diffusion states and may produce useless gradients. A noise-aware classifier is trained on noisy examples (x_t, t) so it learns how class information appears at each noise level. This makes guidance more stable and meaningful.

Chapter 141. Question 6: What is classifier-free guidance?

Classifier-free guidance trains one conditional diffusion model to also behave unconditionally by randomly dropping the condition during training. At sampling time, the model is evaluated twice: once with the condition and once with a null condition. The predictions are combined:

$$\epsilon_{CFG} = (1 + w)\epsilon_{cond} - w\epsilon_{uncond}$$

This amplifies the conditional part of the prediction without requiring an external classifier.

Chapter 142. Question 7: Why is condition dropping essential in CFG?

Condition dropping is essential because the model must learn a meaningful unconditional branch. Without training on null conditions, the unconditional prediction $\epsilon_{\theta}(x_t, t, \emptyset)$ would not be reliable, and the CFG subtraction would have no meaningful reference. Random dropping teaches the model both conditional and unconditional behavior.

Chapter 143. Question 8: Why does stronger guidance improve condition adherence but reduce diversity?

Stronger guidance pushes samples more aggressively toward the condition. This improves alignment with the class or prompt, but it suppresses trajectories that are plausible yet less strongly condition-aligned. As a result, the set of possible outputs narrows, diversity decreases, and artifacts may appear if guidance overwhelms the natural data score.

Chapter 144. Question 9: What does FID measure?

FID compares the feature distributions of real and generated samples. It represents each distribution using a mean and covariance in a deep feature space and computes a Fréchet distance between them. Lower FID means generated samples have feature statistics closer to real data. However, FID is only a proxy and does not fully capture prompt alignment, human judgment, or all aspects of generation quality.

Chapter 145. **31. Common Mistakes to Avoid**

Chapter 146. Mistake 1: Thinking MSE is arbitrary

It is not arbitrary. It comes from ELBO simplification under Gaussian assumptions.

Chapter 147. Mistake 2: Thinking the score is a probability

The score is not $p(x)$.

It is:

$$\nabla_x \log p(x)$$

a vector field pointing toward increasing density.

Chapter 148. Mistake 3: Thinking diffusion only removes noise

Reverse diffusion removes noise but also includes calibrated stochasticity to preserve sampling behavior.

Chapter 149. Mistake 4: Thinking classifier guidance uses a clean classifier

A clean-image classifier is usually not enough. The classifier must understand noisy states x_t .

Chapter 150. **Mistake 5: Thinking CFG uses a separate classifier**

CFG does not use a separate classifier. It uses conditional and unconditional predictions from the same diffusion model.

Chapter 151. **Mistake 6: Thinking stronger guidance is always better**

Stronger guidance improves alignment but can reduce diversity and introduce artifacts.

Chapter 152. **Mistake 7: Thinking FID measures everything**

FID measures feature distribution similarity, not prompt alignment, factual correctness, or all human preferences.

Chapter 153. **32. One-Page Open-Note Summary**

Chapter 4 gives a deeper theoretical view of diffusion models. The standard diffusion noise-prediction MSE can be derived from an ELBO: when the forward process and reverse posterior are Gaussian and reverse variance is fixed, the KL terms reduce to mean matching, which can be parameterized as noise prediction. The score is $\nabla_x \log p_t(x)$, the direction that moves a sample toward higher probability at a given noise level. Although DDPMs are often trained to predict noise, this prediction is closely related to score estimation. Reverse diffusion combines a deterministic drift term, which denoises and moves samples toward data, with a stochastic term, which preserves sampling diversity. Guided diffusion modifies the reverse score field so samples become both realistic and condition-aligned. Classifier guidance uses Bayes' rule to add a classifier gradient to the diffusion score. The classifier must be noise-aware because it is queried on noisy states x_t , not clean images. Classifier-free guidance trains one model with random condition dropping so it learns both conditional and unconditional predictions; at sampling, these are combined to amplify condition adherence. Guidance strength controls a tradeoff: stronger guidance improves alignment but reduces diversity and may create artifacts. FID evaluates generated samples by comparing real and generated feature distributions using their means and covariances, but it is only one proxy metric.

Chapter 154. **33. Very Short Final Revision**

Know these:

- Diffusion MSE comes from ELBO simplification.
- Score = $\nabla_x \log p_t(x)$.
- Score points toward higher probability regions.
- Noise prediction is related to score estimation.
- Reverse diffusion = drift + calibrated noise.
- Guided diffusion modifies the score field.
- Classifier guidance = diffusion score + classifier gradient.
- Noise-aware classifier = classifier trained on x_t, t .
- CFG = conditional prediction + unconditional prediction combined.
- Random condition dropping is required for CFG.
- Strong guidance = better control, lower diversity, more artifacts.
- FID compares real/generated feature means and covariances.
- FID is useful but incomplete.

Chapter 155. Chapter 5 — Flow Matching and Similar Techniques

Chapter 5 is about a newer family of generative models called **flow matching**. The main idea is:

Instead of generating by slowly denoising like diffusion, flow matching learns a velocity field that moves samples from a simple distribution, such as Gaussian noise, to the data distribution.

So the mental picture is different:

Diffusion	Flow Matching
learns to remove noise step by step	learns how particles should move
reverse process is often stochastic	sampling is often an ODE
model predicts noise/score	model predicts velocity
generation = denoising	generation = transport

The chapter presents flow matching as a **transport problem**: start from a simple base distribution p_0 , such as Gaussian noise, and move it into the data distribution $p_1 = p_{\text{data}}$.

Chapter 156. 1. The Big Idea of Flow Matching

Flow matching learns a time-dependent velocity field:

$$v_{\theta}(x, t)$$

This velocity field tells each point:

At location x and time t , move in this direction with this speed.

Sampling is done by solving the ordinary differential equation:

$$\frac{dx}{dt} = v_{\theta}(x, t)$$

The chapter's flashcards define this directly: the role of $v_{\theta}(x, t)$ is to define the direction and speed to move particles from a base distribution to a target distribution.

Chapter 157. Simple Intuition

Imagine random noise points as particles floating in space.

At time:

$$t = 0$$

the particles are just noise:

$$x(0) \sim p_0$$

At time:

$$t = 1$$

we want the particles to look like data:

$$x(1) \sim p_1 = p_{\text{data}}$$

The velocity field is like a wind field:

$$v_{\theta}(x, t)$$

It tells each particle where to move.

So generation becomes:

noise particles $\rightarrow$ data particles

The professor's summary uses exactly this "wind-field" intuition: x is the current particle location, t is time, and $v_\theta(x, t)$ tells the particle which direction to move and how fast.

Chapter 158. **2. Base Distribution and Target Distribution**

Flow matching starts with two distributions.

Chapter 159. **Base distribution**

$$p_0$$

This is easy to sample from.

Usually:

$$p_0 = \mathcal{N}(0, I)$$

So:

$$x_0 \sim p_0$$

means:

sample Gaussian noise.

The flashcards say the base distribution p_0 typically represents simple Gaussian noise.

Chapter 160. **Target distribution**

$$p_1$$

This is the data distribution.

$$p_1 = p_{\text{data}}$$

So:

$$x_1 \sim p_1$$

means:

sample a real data example.

For example:

- real image,
- real digit,
- real audio sample,
- real latent representation.

The goal is:

$$p_0 \rightarrow p_1$$

That means:

transform noise into data.

Chapter 161. **3. The ODE Used for Sampling**

The sampling ODE is:

$$\frac{dx}{dt} = v_{\theta}(x, t)$$

This means:

the rate of change of the sample is equal to the learned velocity field.

If the model knows the correct velocity everywhere, then starting from noise and integrating this ODE produces data.

The flashcards list this as the ODE used for sampling in flow matching.

Chapter 162. What Does This Mean Practically?

Start with:

$$x(0) \sim p_0$$

Then update over time:

$$x(0) \rightarrow x(\Delta t) \rightarrow x(2\Delta t) \rightarrow \dots \rightarrow x(1)$$

At the end:

$$x(1) \sim p_{\text{data}}$$

So the generated sample is:

$$x(1)$$

Chapter 163. 4. Why Flow Matching Is Called “Matching”

It is called flow matching because the model is trained to **match a target velocity field**.

During training, we create synthetic paths between noise and data.

Given:

$$x_0 \sim p_0$$

and:

$$x_1 \sim p_1$$

we define an intermediate point:

$$x_t$$

Then we compute the correct velocity:

$$u_t$$

The model is trained so that:

$$v_\theta(x_t, t) \approx u_t$$

using MSE:

$$\mathcal{L}(\theta) = \mathbb{E}[\|v_\theta(x_t, t) - u_t\|^2]$$

The chapter summary describes this as supervised vector-field regression: the model learns a known velocity along a chosen path, instead of estimating densities or latent posteriors.

Chapter 164. **5. The Simplest Path: Linear Interpolation**

The simplest bridge between noise and data is a straight line.

Given:

$$\begin{aligned}x_0 &\sim p_0 \\x_1 &\sim p_1\end{aligned}$$

define:

$$x_t = (1 - t)x_0 + tx_1$$

where:

$$t \in [0,1]$$

At $t = 0$:

$$x_t = x_0$$

At $t = 1$:

$$x_t = x_1$$

So the path starts at noise and ends at data.

The main Chapter 5 notes introduce this as the simplest path: linear interpolation from x_0 to x_1 .

Chapter 165. **6. Target Velocity for Linear Path**

For:

$$x_t = (1 - t)x_0 + tx_1$$

take derivative with respect to t :

$$\frac{dx_t}{dt} = x_1 - x_0$$

So the target velocity is:

$$u_t = x_1 - x_0$$

This means:

Along a straight path, the correct velocity is the displacement from start point to end point.

So training uses:

$$v_{\theta}(x_t, t) \approx x_1 - x_0$$

The Chapter 5 summary states that for the simplest linear bridge, $x_t = (1 - t)x_0 + tx_1$, the target instantaneous velocity is known analytically as $u_t = x_1 - x_0$.

Chapter 166. **7. Simple Numerical Example**

Suppose:

$$\begin{aligned}x_0 &= 2 \\x_1 &= 10\end{aligned}$$

At:

$$\begin{aligned}t &= 0.25 \\x_t &= (1 - 0.25)(2) + 0.25(10) \\x_t &= 0.75(2) + 2.5 \\x_t &= 1.5 + 2.5 = 4\end{aligned}$$

The target velocity is:

$$u_t = x_1 - x_0 = 10 - 2 = 8$$

So the training example says:

At time $t = 0.25$, when the particle is at $x = 4$, the correct velocity is 8.

The model learns:

$$v_{\theta}(4, 0.25) \approx 8$$

Chapter 167. **8. Flow Matching Training Algorithm**

The basic training procedure is:

1. Sample noise:

$$x_0 \sim p_0$$

2. Sample data:

$$x_1 \sim p_1$$

3. Sample time:

$$t \sim \text{Uniform}(0,1)$$

4. Build intermediate point:

$$x_t = (1 - t)x_0 + tx_1$$

5. Compute target velocity:

$$u_t = x_1 - x_0$$

6. Predict velocity:

$$v_\theta(x_t, t)$$

7. Minimize:

$$\| v_\theta(x_t, t) - u_t \|^2$$

The main notes present essentially this training logic: sample x_0 , sample x_1 , build bridge point x_t , build target velocity u_t , compute MSE, update parameters.

Chapter 168. 9. Flow Matching Sampling Algorithm

After training, generation works like this:

1. Sample initial noise:

$$x(0) \sim p_0$$

2. Solve the ODE:

$$\frac{dx}{dt} = v_\theta(x, t)$$

from:

$$t = 0$$

to:

$$t = 1$$

3. Output:

$$x(1)$$

If the learned velocity field is good, then:

$$x(1) \sim p_{\text{data}}$$

The Chapter 5 summary says sampling is conceptually: initialize from noise, then repeatedly apply small velocity-driven updates.

Chapter 169. 10. Euler Integration

Most ODEs cannot be solved exactly, so we approximate.

The simplest method is **Euler integration**:

$$x_{k+1} = x_k + \Delta t v_\theta(x_k, t_k)$$

where:

Symbol	Meaning
x_k	current sample
t_k	current time
Δt	step size
$v_\theta(x_k, t_k)$	predicted velocity
x_{k+1}	updated sample

The chapter summary explains that Euler works because the exact solution satisfies an integral equation, and Euler approximates the integral by a Riemann sum.

Chapter 170. Simple Intuition

If the velocity says:

$$v = 5$$

and step size is:

$$\Delta t = 0.1$$

then move:

$$0.1 \times 5 = 0.5$$

So:

$$x_{new} = x_{old} + 0.5$$

Euler says:

Move a small step in the direction predicted by the velocity field.

Chapter 171. 11. Why Euler Works

The ODE is:

$$\frac{dx}{dt} = v_{\theta}(x, t)$$

For a small step:

$$\Delta t$$

Taylor expansion gives:

$$x(t + \Delta t) \approx x(t) + \Delta t \frac{dx}{dt}$$

Since:

$$\frac{dx}{dt} = v_{\theta}(x, t)$$

we get:

$$x(t + \Delta t) \approx x(t) + \Delta t v_{\theta}(x, t)$$

That is Euler's method.

The main notes derive Euler from a first-order Taylor expansion and explain it as assuming the velocity is roughly constant over a small interval.

Chapter 172. **12. Why Flow Matching Is a Transport Model**

At the surface, the training loss looks like regression:

$$\mathbb{E}[\| v_{\theta}(x_t, t) - u_t \|^2]$$

So someone might say:

It is just vector regression.

But that misses the deeper idea.

The learned velocity field defines a dynamical system:

$$\frac{dx}{dt} = v_{\theta}(x, t)$$

This dynamical system transports an entire distribution:

$$p_0 \rightarrow p_1$$

The professor's sample question emphasizes this: the regression loss is only the training mechanism; the learned object is a transport field that moves probability mass.

Chapter 173. Exam Sentence

Flow matching is best understood as a transport model because the learned velocity field induces distributional dynamics that move probability mass from a base distribution to a data distribution. The MSE loss is just the way we train the vector field.

Chapter 174. 13. Continuity Equation

The continuity equation describes how probability density changes as particles move.

If:

$$p_t(x)$$

is the density at time t , and:

$$v_t(x)$$

is the velocity field, then:

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x)v_t(x)) = 0$$

This equation means:

Probability mass is conserved. It can move around, but it cannot appear from nowhere or vanish.

The Chapter 5 summary calls this a deep conceptual point: flow matching models the evolution of an entire probability distribution over time, and the continuity equation says probability mass is neither created nor destroyed, only transported.

Chapter 175. **14. Understanding the Continuity Equation**

The equation:

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x)v_t(x)) = 0$$

has two parts.

Chapter 176. **First term**

$$\frac{\partial p_t(x)}{\partial t}$$

This means:

How density changes over time at location x .

Chapter 177. **Second term**

$$\nabla \cdot (p_t(x)v_t(x))$$

This means:

Net outflow of probability from that location.

If more probability leaves a region than enters, density decreases.

If more probability enters than leaves, density increases.

The main notes explain that the continuity equation is like mass conservation in fluids, except the “fluid” is probability density.

Chapter 178. **15. Divergence Intuition**

For a vector field $v(x)$, divergence measures net outflow.

$$\nabla \cdot v(x)$$

If:

$$\nabla \cdot v(x) > 0$$

the flow is spreading outward.

Density tends to decrease.

If:

$$\nabla \cdot v(x) < 0$$

the flow is converging inward.

Density tends to increase.

So:

Flow matching moves probability mass like a fluid.

Chapter 179. **16. Difference Between Diffusion and Flow Matching**

This is extremely important.

Concept	Diffusion	Flow Matching
Starting point	noise	noise

Concept	Diffusion	Flow Matching
Final goal	data	data
Learned object	noise/score/denoiser	velocity field
Sampling equation	reverse Markov chain or SDE/ODE	ODE
Typical process	denoise gradually	transport particles
Training target	added noise or score	target velocity
Randomness during sampling	often stochastic	often deterministic after initial noise
Core view	reverse corruption	probability transport

The Chapter 5 summary says flow matching shifts the viewpoint: it avoids explicit score estimation and avoids latent posterior approximation by turning generation into supervised vector-field regression.

Chapter 180. 17. Flow Matching vs VAE

Concept	VAE	Flow Matching
Latent variable	z	usually starts with $x_0 \sim p_0$
Main learned object	encoder + decoder	velocity field
Training challenge	approximate posterior and ELBO	supervised velocity matching
Sampling	sample z , decode once	integrate ODE
Distribution movement	latent prior to data via decoder	base distribution to data via flow

The Chapter 5 summary directly contrasts them: VAE handles an approximate posterior $q_\phi(z | x)$ and decoder likelihood $p_\theta(x | z)$, while flow matching avoids latent posterior approximation by learning a vector field.

Chapter 181. 18. Flow Matching vs Diffusion

Diffusion learns a denoising or score-related mechanism:

$$\epsilon_\theta(x_t, t)$$

or:

$$s_{\theta}(x_t, t)$$

Flow matching learns:

$$v_{\theta}(x, t)$$

Diffusion asks:

What noise should be removed?

Flow matching asks:

In which direction and how fast should this particle move?

The professor's summary says diffusion learns a denoising/score-related mechanism, while flow matching learns a velocity field.

Chapter 182. 19. The Importance of the Bridge

The interpolation path:

$$x_t = (1 - t)x_0 + tx_1$$

is called a **bridge**.

It defines what intermediate states look like between noise and data.

The bridge is not a trivial detail.

It affects:

- what x_t values the model sees during training,
- what velocities it must learn,
- whether paths are straight or curved,
- how complex the vector field becomes,
- sampling stability.

The professor's sample question says the interpolation path expresses a modeling assumption about how probability mass should move through state space, and better bridges can make learning easier and sampling more stable.

Chapter 183. 20. Nonlinear Bridges

Linear interpolation is only one option.

A more general bridge can be:

$$x_t = a(t)x_0 + b(t)x_1 + \sigma(t)\epsilon$$

or another stochastic interpolation.

Then the target velocity becomes:

$$u_t = \frac{dx_t}{dt}$$

Different bridges create different training distributions and velocity targets.

The main notes mention extensions such as nonlinear bridges, variance-preserving stochastic interpolation, and bridges where the path depends on t .

Chapter 184. 21. Why Coupling Matters

To create a training path, we need a pair:

$$(x_0, x_1)$$

where:

- $x_0 \sim p_0$ is noise,
- $x_1 \sim p_1$ is data.

But how do we pair noise samples with data samples?

This is called a **coupling**.

Chapter 185. Independent Coupling

The simplest approach:

- sample x_0 randomly,
- sample x_1 randomly,
- pair them.

This is easy but may create messy paths.

Noise point x_0 might be paired with a faraway data point x_1 , creating long crossing trajectories.

Chapter 186. Optimal Transport Coupling

Optimal transport tries to pair x_0 and x_1 more intelligently.

It seeks pairings that minimize transport cost:

$$\mathbb{E}[\|x_1 - x_0\|^2]$$

The result:

- shorter paths,
- less crossing,
- smoother velocity field,
- easier learning.

The professor's summary says OT-based flow matching changes the geometry of supervision by changing which start and end samples define the training stories.

Chapter 187. 22. Why OT Couplings Matter Even with Linear Paths

Even if the path formula remains:

$$x_t = (1 - t)x_0 + tx_1$$

the pairing still matters.

Random pairing may produce tangled trajectories.

OT pairing produces more coherent trajectories.

So the path formula is only part of the design. The coupling also shapes the difficulty of learning.

The professor's sample answer states that even if the bridge stays linear, the choice of which x_0 is paired with which x_1 determines the geometry of training trajectories.

Chapter 188. **23. Latent Flow Matching**

Latent flow matching applies flow matching in a compressed latent space instead of pixel/data space.

First, use an encoder-decoder:

$$\begin{aligned}x &\rightarrow z \\ z &\rightarrow x\end{aligned}$$

Then learn a velocity field in latent space:

$$v_{\theta}(z, t)$$

The Chapter 5 summary says latent flow matching first defines a representation and then learns transport in lower-dimensional latent space, which may be semantically cleaner and cheaper.

Chapter 189. **Why Latent Flow Matching Helps**

Pixel space is high-dimensional and full of low-level details.

For images, pixel space includes:

- edges,
- textures,
- lighting,
- small local variations,
- noise,

- high-frequency details.

Learning a velocity field directly in pixel space can be hard.

Latent space can be:

- lower-dimensional,
- smoother,
- more semantic,
- easier to transport in,
- cheaper computationally.

The professor's sample answer says latent flow matching can help because dimensionality is reduced, semantic structure is often more regular, trajectories may become simpler, and computation can be cheaper.

Chapter 190. **24. Latent Flow Matching Procedure**

1. Encode real data:

$$z_1 = E(x)$$

2. Sample base latent noise:

$$z_0 \sim \mathcal{N}(0, I)$$

3. Build bridge:

$$z_t = (1 - t)z_0 + tz_1$$

4. Target velocity:

$$u_t = z_1 - z_0$$

5. Train:

$$v_\theta(z_t, t) \approx u_t$$

6. Sampling:

$$z(0) \sim \mathcal{N}(0, I)$$

7. Integrate:

$$\frac{dz}{dt} = v_{\theta}(z, t)$$

8. Decode:

$$x = D(z(1))$$

Chapter 191. **25. Limitation of Latent Flow Matching**

Latent flow matching depends on the quality of the encoder-decoder.

If the autoencoder loses information, the final generated sample cannot recover it.

So the decoder may become a bottleneck.

The professor's sample answer warns that if the autoencoder discards important information, decoder fidelity may cap the final sample quality.

Chapter 192. **26. Conditional Flow Matching**

Flow matching can be conditional by giving the velocity field a condition:

$$v_{\theta}(x, t, y)$$

where y could be:

- class label,
- text prompt,
- image condition,
- attribute,
- style.

Then the ODE becomes:

$$\frac{dx}{dt} = v_{\theta}(x, t, y)$$

Meaning:

Move noise toward data matching condition y .

Chapter 193. **27. Classifier-Free Guidance in Flow Matching**

Just like diffusion, flow matching can use classifier-free guidance.

Train the model with conditions sometimes dropped:

Conditional:

$$v_{\theta}(x, t, y)$$

Unconditional:

$$v_{\theta}(x, t, \emptyset)$$

At sampling time:

$$v_{\text{guided}} = v_{\text{uncond}} + w(v_{\text{cond}} - v_{\text{uncond}})$$

where:

Term	Meaning
v_{cond}	velocity with condition
v_{uncond}	velocity without condition
w	guidance strength
$v_{\text{cond}} - v_{\text{uncond}}$	condition-specific direction

The Chapter 5 summary says CFG in flow matching mirrors CFG in diffusion, but the guided object is a velocity field rather than a score estimate.

Chapter 194. **28. Guidance Tradeoff in Flow Matching**

Increasing guidance strength:

Chapter 195. **Helps**

- stronger condition adherence,
- better class/text alignment.

Chapter 196. **Hurts**

- diversity decreases,
- trajectories may become unstable,
- artifacts may appear,
- generated samples may become less natural.

This is similar to diffusion guidance.

Chapter 197. **29. Classifier Guidance in Flow Matching**

Another option is classifier guidance.

A classifier estimates:

$$p(y \mid x_t, t)$$

Then its gradient pushes the trajectory toward condition y .

Important:

The classifier must work on intermediate states, not only clean data.

The Chapter 5 summary emphasizes a broad principle: guidance only works well if the conditioning signal is valid at the intermediate states actually visited during sampling.

Chapter 198. **30. Consistency Models**

Consistency models are related to diffusion and flow-style methods.

They try to learn a function:

$$f_{\theta}(x_t, t)$$

that maps noisy states to a consistent clean output.

The key idea:

Different noisy versions of the same underlying sample should map to the same clean output.

So if:

$$x_t$$

and:

$$x_s$$

are different noisy states on the same trajectory, then:

$$f_{\theta}(x_t, t) \approx f_{\theta}(x_s, s)$$

The Chapter 5 summary says a consistency model learns a map that should be approximately invariant across noise levels for the same underlying clean sample.

Chapter 199. **31. Why Consistency Models Are Attractive**

They promise very fast generation.

Instead of many denoising steps, maybe:

- one step,
- two steps,
- a few steps.

This is attractive because diffusion models can be slow.

Consistency models aim to jump directly from noisy input to clean output.

The professor's sample answer says consistency models are attractive because they suggest one-step or few-step generation, which is appealing from a sampling-speed perspective.

Chapter 200. 32. Why Consistency Models Are Fragile

Consistency alone does not guarantee correctness.

A trivial constant function could satisfy consistency:

$$f_{\theta}(x, t) = c$$

for all x, t .

Then:

$$f_{\theta}(x_t, t) = f_{\theta}(x_s, s)$$

but the output is useless.

So consistency can be satisfied without meaningful generation.

The professor's sample answer explains that a trivial constant function can satisfy consistency without preserving data structure.

Chapter 201. Other Problems

Consistency models can also suffer from:

- mode averaging,
- collapse,
- instability,

- ambiguity at high noise,
- dependence on a strong teacher.

At high noise, many clean samples may be possible, so the model may average them.

That produces blurry or collapsed outputs.

Chapter 202. **33. Teacher-Student Consistency**

To prevent trivial collapse, consistency models often use a teacher model.

The teacher is usually a strong diffusion model.

The student learns to imitate the teacher's trajectory or outputs.

This gives the consistency model a meaningful target.

The Chapter 5 summary says teacher-student consistency is important because a strong teacher provides an anchor so the student learns a nontrivial canonicalization map rather than collapsing.

Chapter 203. **34. Flow Matching and Consistency Models Compared**

Concept	Flow Matching	Consistency Model
Learns	velocity field	direct map to clean/canonical output
Sampling	integrate ODE	one/few-step mapping
Main appeal	clean transport view	very fast generation
Main risk	poor bridge/coupling	collapse/trivial consistency
Training target	velocity u_t	consistent output across time

Chapter 204. **35. How Chapter 5 Connects to Earlier Chapters**

Chapter 205. Connection to VAE

VAE:

$$z \sim p(z), x \sim p_\theta(x | z)$$

Flow matching:

$$x(0) \sim p_0, \frac{dx}{dt} = v_\theta(x, t), x(1) \sim p_1$$

Both move from simple distribution to data.

But VAE uses a decoder, while flow matching uses an ODE velocity field.

Chapter 206. Connection to Diffusion

Diffusion:

noise $\rightarrow$ data

through reverse denoising.

Flow matching:

noise $\rightarrow$ data

through deterministic transport.

Both are generative paths from simple to complex distributions.

Chapter 207. Connection to Score Models

Score models learn:

$$\nabla_x \log p_t(x)$$

Flow matching learns:

$$v_{\theta}(x, t)$$

The score tells where density increases.

The velocity tells where the particle should move.

Chapter 208. **36. Chapter 5 as a Transport-Based View of Generative Models**

The professor's summary gives a very important big-picture comparison:

Model	What it learns
VAE	encoder-decoder latent mechanism
Diffusion	denoising / score-related mechanism
Flow matching	velocity field
Consistency model	time-consistent canonicalization map

The deeper lesson is that many generative models can be interpreted as ways of moving probability mass from a simple distribution to a complex data distribution.

Chapter 209. **37. Important Formula Summary**

Chapter 210. **Base and target**

$$\begin{aligned} x_0 &\sim p_0 \\ x_1 &\sim p_1 = p_{\text{data}} \end{aligned}$$

Chapter 211. **Flow ODE**

$$\frac{dx}{dt} = v_{\theta}(x, t)$$

Chapter 212. **Linear bridge**

$$x_t = (1 - t)x_0 + tx_1$$

Chapter 213. Target velocity

$$u_t = x_1 - x_0$$

Chapter 214. Flow matching loss

$$\mathcal{L}(\theta) = \mathbb{E}[\|v_\theta(x_t, t) - u_t\|^2]$$

Chapter 215. Euler update

$$x_{k+1} = x_k + \Delta t v_\theta(x_k, t_k)$$

Chapter 216. Continuity equation

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x)v_t(x)) = 0$$

Chapter 217. Conditional velocity

$$v_\theta(x, t, y)$$

Chapter 218. CFG velocity

$$v_{\text{guided}} = v_{\text{uncond}} + w(v_{\text{cond}} - v_{\text{uncond}})$$

Chapter 219. Consistency condition

$$f_{\theta}(x_t, t) \approx f_{\theta}(x_s, s)$$

for noisy states on the same underlying trajectory.

Chapter 220. **38. Most Likely Exam Questions**

Chapter 221. **Question 1: What is the main idea of flow matching?**

Flow matching treats generation as a transport problem. It starts from a simple base distribution p_0 , usually Gaussian noise, and learns a time-dependent velocity field $v_{\theta}(x, t)$ that moves samples toward the target data distribution $p_1 = p_{\text{data}}$. Sampling is performed by solving the ODE $\frac{dx}{dt} = v_{\theta}(x, t)$ from $t = 0$ to $t = 1$.

Chapter 222. **Question 2: Why is flow matching best understood as a transport model rather than just regression?**

Although training uses an MSE regression loss to match predicted velocity to a target velocity, the learned object is not merely a regression output. The learned velocity field defines a dynamical system $\frac{dx}{dt} = v_{\theta}(x, t)$ that transports an entire probability distribution from p_0 to p_1 . The continuity equation describes this movement of probability mass. Therefore, regression is only the training mechanism; the conceptual object is distributional transport.

Chapter 223. **Question 3: Explain the linear bridge and target velocity.**

Given a base sample $x_0 \sim p_0$ and data sample $x_1 \sim p_1$, the simplest bridge is:

$$x_t = (1 - t)x_0 + tx_1$$

This creates a straight path from x_0 at $t = 0$ to x_1 at $t = 1$. The target velocity is the derivative:

$$u_t = \frac{dx_t}{dt} = x_1 - x_0$$

The model is trained so that:

$$v_{\theta}(x_t, t) \approx x_1 - x_0$$

Chapter 224. Question 4: Why is the interpolation path not a trivial design choice?

The interpolation path determines what intermediate states the model sees and what target velocities it must learn. A linear bridge creates straight paths, but other bridges can create different geometries and training distributions. The path affects the complexity of the learned velocity field, the stability of sampling, and the quality of generated samples. Therefore, the bridge expresses a modeling assumption about how probability mass should move.

Chapter 225. Question 5: Why do optimal-transport couplings matter?

Even if the bridge formula is linear, the pairing between x_0 and x_1 determines the geometry of the training paths. Random pairings can produce long, crossing, tangled trajectories. Optimal-transport couplings pair base and data samples more coherently, often minimizing transport cost and producing shorter, smoother paths. This can make the velocity field easier to learn.

Chapter 226. Question 6: Why can latent flow matching be better than pixel-space flow matching?

Pixel space is high-dimensional and contains many low-level details. Learning a velocity field directly in pixel space can be difficult and expensive. Latent flow matching first maps data into a lower-dimensional latent space, then learns the flow there. This can make trajectories simpler, reduce computation, and exploit more semantic representations. However, quality depends on the encoder-decoder; if the autoencoder loses information, the decoder limits final sample fidelity.

Chapter 227. Question 7: How does classifier-free guidance work in flow matching?

The model is trained to predict both conditional and unconditional velocity fields by sometimes dropping the condition. At sampling time, the conditional and unconditional velocities are combined:

$$v_{\text{guided}} = v_{\text{uncond}} + w(v_{\text{cond}} - v_{\text{uncond}})$$

The unconditional velocity gives a realism baseline, while the difference term provides a condition-specific push. The guidance scale controls the tradeoff between condition adherence and diversity/stability.

Chapter 228. **Question 8: Why are consistency models attractive and fragile?**

Consistency models are attractive because they aim for one-step or few-step generation by learning a function that maps different noisy versions of the same sample to the same clean output. They are fragile because consistency alone does not guarantee correctness. A trivial constant function could satisfy the consistency condition without generating meaningful data. High-noise ambiguity can also cause averaging or collapse. Therefore, consistency models often need teacher guidance or anchors.

Chapter 229. **Question 9: Compare diffusion and flow matching.**

Diffusion learns a reverse denoising process, often by predicting noise or score, and generation proceeds through many reverse steps. Flow matching learns a time-dependent velocity field and generation proceeds by integrating an ODE from noise to data. Diffusion is naturally connected to stochastic reverse processes and score estimation, while flow matching has a clearer deterministic transport interpretation.

Chapter 230. **39. Common Mistakes to Avoid**

Chapter 231. **Mistake 1: Thinking flow matching is only MSE regression**

The MSE loss is only how the velocity field is trained. The learned field transports probability mass.

Chapter 232. Mistake 2: Confusing x_0 in flow matching with x_0 in diffusion

In diffusion, x_0 usually means clean data.

In flow matching, many notes use:

$$x_0 \sim p_0$$

to mean base noise, and:

$$x_1 \sim p_1$$

to mean data.

Always check notation.

Chapter 233. Mistake 3: Thinking the velocity is the same as the score

The score is:

$$\nabla_x \log p_t(x)$$

The velocity is:

$$v_\theta(x, t)$$

They are related in some frameworks, but conceptually different.

Chapter 234. Mistake 4: Thinking linear interpolation is always best

Linear interpolation is simple, but bridge choice affects learning and sampling.

Chapter 235. Mistake 5: Ignoring coupling

Even with the same path formula, different pairings between noise and data can change trajectory complexity.

Chapter 236. **Mistake 6: Thinking latent flow matching has no downside**

It depends on the quality of the latent representation and decoder.

Chapter 237. **Mistake 7: Thinking consistency alone guarantees quality**

Consistency can be satisfied trivially. Meaningful generation often requires anchors or teacher guidance.

Chapter 238. **40. One-Page Open-Note Summary**

Flow matching treats generation as a transport problem. It starts with a simple base distribution p_0 , usually Gaussian noise, and aims to move it into the data distribution $p_1 = p_{\text{data}}$. The model learns a time-dependent velocity field $v_\theta(x, t)$, and sampling solves the ODE $\frac{dx}{dt} = v_\theta(x, t)$. In the simplest setup, sample $x_0 \sim p_0$ and $x_1 \sim p_1$, define a linear bridge $x_t = (1 - t)x_0 + tx_1$, and compute the target velocity $u_t = x_1 - x_0$. The model is trained with MSE to match $v_\theta(x_t, t)$ to u_t . Although this looks like regression, the learned object is a transport field moving probability mass, described by the continuity equation. Euler integration approximates sampling with $x_{k+1} = x_k + \Delta t v_\theta(x_k, t_k)$. Bridge design and coupling strongly affect the difficulty of learning; optimal-transport couplings can produce shorter, smoother trajectories than random pairings. Latent flow matching learns the velocity field in a compressed latent space, reducing dimension and often simplifying transport, but it depends on decoder quality. Flow matching can be conditional through $v_\theta(x, t, y)$, and classifier-free guidance can amplify conditional velocity directions. Consistency models are related methods that aim for one-step or few-step generation by mapping different noisy states to a shared clean output, but they are fragile because trivial consistency or high-noise ambiguity can cause collapse.

Chapter 239. **41. Very Short Final Revision**

Know these:

- Flow matching = transport from p_0 to p_1 .
- p_0 = base distribution, usually Gaussian noise.
- $p_1 = p_{\text{data}}$ = target data distribution.
- Learn velocity field $v_\theta(x, t)$.
- Sampling ODE: $\frac{dx}{dt} = v_\theta(x, t)$.
- Linear bridge: $x_t = (1 - t)x_0 + tx_1$.
- Target velocity: $u_t = x_1 - x_0$.
- Loss: $\mathbb{E} \|v_\theta(x_t, t) - u_t\|^2$.
- Euler update: $x_{k+1} = x_k + \Delta t v_\theta(x_k, t_k)$.
- Continuity equation = probability mass conservation.
- Bridge choice matters.
- Coupling matters; OT coupling gives cleaner paths.
- Latent flow matching = flow in compressed latent space.
- CFG in flow matching guides the velocity field.
- Consistency models promise fast generation but can collapse.

Chapter 240. Chapter 6 — Autoregressive Models and Transformers

Chapter 6 explains how modern sequence models work, especially **autoregressive models** and **Transformers**. This chapter is very important because it connects directly to GPT-style models, language generation, VLMs, supervised fine-tuning, DPO, and RLHF.

The main topics are:

1. Autoregressive sequence modeling
2. Training vs sampling
3. RNNs and LSTMs
4. Why attention was needed
5. Self-attention: queries, keys, values
6. Causal masking
7. Multi-head attention
8. Transformer blocks
9. Positional encoding
10. Tokenization and BPE
11. GPT-style decoder-only Transformers

The professor's chapter summary defines autoregressive modeling as factorizing a full sequence probability into a product of next-step conditionals, then explains RNNs, LSTMs, self-attention, Transformer blocks, positional information, causal masking, tokenization, and GPT-style training.

Chapter 241. 1. What Is an Autoregressive Model?

An **autoregressive model** is a model that generates or predicts a sequence one element at a time.

It defines the probability of a sequence as:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{1:t-1})$$

This means:

The probability of the whole sequence equals the product of the probability of each token given the previous tokens.

The Chapter 6 flashcards define an autoregressive model as a model that factorizes a sequence distribution into a product of next-step conditionals.

Chapter 242. 1.1 What Does $x_{1:T}$ Mean?

$$x_{1:T}$$

means the full sequence:

$$x_1, x_2, x_3, \dots, x_T$$

For language:

$$x_t$$

could be a word, subword, byte, or token.

Example:

$$x_{1:5} = [\text{I, love, machine, learning, .}]$$

Chapter 243. 1.2 What Does $p(x_t \mid x_{1:t-1})$ Mean?

$$p(x_t \mid x_{1:t-1})$$

means:

Probability of the current token given all previous tokens.

Example:

Sentence prefix:

“The cat sat on the”

The model predicts:

$$p(\text{mat} \mid \text{The cat sat on the})$$

or:

$$p(\text{floor} \mid \text{The cat sat on the})$$

or other possible next tokens.

Chapter 244. 2. Why Autoregressive Factorization Matters

The full sequence distribution:

$$p(x_1, x_2, \dots, x_T)$$

is very hard to model directly.

Autoregressive factorization turns this hard problem into many smaller next-token prediction problems:

$$\begin{aligned} p(x_1) \\ p(x_2 \mid x_1) \end{aligned}$$

$$\begin{aligned}
& p(x_3 \mid x_1, x_2) \\
& \dots \\
& p(x_T \mid x_1, \dots, x_{T-1})
\end{aligned}$$

The professor's summary says this is both a probabilistic statement and a practical training strategy: modeling a whole sequence becomes training the model to predict the next token at each position.

Chapter 245. Exam Sentence

Autoregressive factorization is important because it converts sequence modeling into next-token prediction while still defining a valid probability distribution over complete sequences.

Chapter 246. 3. Language Modeling with a Categorical Distribution

In language modeling, each next token is selected from a vocabulary.

Let the vocabulary size be:

$$V$$

At timestep t , the model outputs logits:

$$z_t \in \mathbb{R}^V$$

Then softmax converts logits into probabilities:

$$\pi_t = \text{softmax}(z_t)$$

The next token is modeled as:

$$x_t \sim \text{Categorical}(\pi_t)$$

This means:

The model predicts a probability distribution over all vocabulary tokens.

The flashcards state that the categorical distribution with parameter π_t represents the conditional distribution of the next token, where $\pi_t = \text{softmax}(z_t)$.

Chapter 247. 3.1 What Are Logits?

Logits are raw model scores before softmax.

Example:

$$z_t = [2.1, 0.3, -1.5]$$

They are not probabilities yet.

Softmax turns them into:

$$\pi_t = [0.82, 0.14, 0.04]$$

Then the model either samples or chooses from this distribution.

Chapter 248. 4. Training vs Sampling

Autoregressive models have two different stages:

1. **Training**
2. **Sampling / generation**

This distinction is extremely important.

Chapter 249. 4.1 Training: Teacher Forcing

During training, the model is given the correct previous tokens from the dataset.

This is called **teacher forcing**.

Example training sentence:

“The cat sat on the mat.”

To predict “cat,” the model sees:

“The”

To predict “sat,” the model sees:

“The cat”

To predict “on,” the model sees:

“The cat sat”

The model always conditions on the **true previous tokens**, not its own mistakes.

The Chapter 6 notes state that training uses teacher forcing: condition on the ground-truth prefix to predict the next token.

Chapter 250. 4.2 Sampling: Using the Model’s Own Outputs

During generation, there is no ground-truth future text.

The model starts with a prompt, predicts the next token, adds it to the context, then predicts again.

Example:

Prompt:

“The cat”

Model predicts:

“sat”

Now the context becomes:

“The cat sat”

Model predicts:

“on”

Now:

“The cat sat on”

and so on.

During sampling, the model conditions on its own generated tokens.

The Chapter 6 notes clearly distinguish the two stages: training uses ground-truth prefixes, while sampling conditions on the model’s own previously generated samples.

Chapter 251. **4.3 Why This Difference Matters**

During training, the model sees perfect prefixes.

During generation, the model may make a mistake, and that mistake becomes part of the next input.

This can cause error accumulation.

Example:

If the model generates a wrong word, the rest of the sentence may become worse because every future prediction depends on the generated prefix.

Chapter 252. **Exam Sentence**

Teacher forcing trains the model using true previous tokens, while sampling generates tokens autoregressively using the model’s own previous outputs.

Chapter 253. **5. Autoregressive Loss**

Training maximizes the likelihood of the true sequence:

$$\prod_{t=1}^T p_{\theta}(x_t | x_{<t})$$

Usually we maximize log-likelihood:

$$\sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$$

Equivalently, we minimize negative log-likelihood:

$$-\sum_{t=1}^T \log p_{\theta}(x_t | x_{<t})$$

This is the **cross-entropy loss** for next-token prediction.

The professor's summary states that GPT-style models output logits for the next token at every position, softmax converts logits to probabilities, and training minimizes cross-entropy against the true next token.

Chapter 254. 6. Sampling Strategies

At generation time, the model outputs a probability distribution over the next token.

Then we need to choose a token.

Common strategies:

Chapter 255. 6.1 Greedy Decoding

Choose the most likely token:

$$x_t = \arg \max_i p(i | x_{<t})$$

Advantages:

- simple,
- deterministic,
- fast.

Disadvantages:

- can be repetitive,
 - less creative,
 - may get stuck in dull outputs.
-

Chapter 256. 6.2 Sampling

Sample from the probability distribution:

$$x_t \sim \text{Categorical}(\pi_t)$$

Advantages:

- more diverse,
- creative.

Disadvantages:

- may produce errors,
 - less predictable.
-

Chapter 257. 6.3 Temperature

Temperature changes the sharpness of probabilities.

Softmax with temperature:

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

where:

Temperature	Effect
-------------	--------

Temperature	Effect
$\tau < 1$	sharper, more deterministic
$\tau = 1$	normal
$\tau > 1$	flatter, more random

Low temperature makes the model more confident.

High temperature makes the model more creative but riskier.

Chapter 258. 6.4 Top-k Sampling

Only sample from the top k most likely tokens.

Example:

If $k = 5$, the model samples only among the top 5 tokens.

This prevents very unlikely tokens from being chosen.

Chapter 259. 6.5 Top-p / Nucleus Sampling

Choose the smallest set of tokens whose cumulative probability exceeds p .

Example:

$$p = 0.9$$

The model samples from the most likely tokens that together cover 90% probability.

This adapts the number of candidate tokens depending on uncertainty.

Chapter 260. 7. RNNs — Recurrent Neural Networks

Before Transformers, RNNs were the main neural sequence model.

An RNN processes a sequence step by step.

At each timestep:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

where:

Symbol	Meaning
x_t	input token at time t
h_t	hidden state at time t
h_{t-1}	previous hidden state
W_x, W_h, b	learned parameters

The hidden state acts as a memory of the past.

The Chapter 6 summary explains that an RNN updates a hidden state one step at a time, and the hidden state acts as a running summary of the past.

Chapter 261. 7.1 RNN Intuition

Imagine reading a sentence word by word.

At each word, the RNN updates its memory.

Example:

“The cat sat on the mat”

After reading “The cat,” the hidden state should remember that the subject is a cat.

After reading “sat,” it updates with the action.

So the hidden state is like a compressed summary.

Chapter 262. 7.2 RNN Limitation: Bottleneck

The problem is that all previous information must be compressed into one vector:

$$h_t$$

For long sequences, this becomes difficult.

If the sequence is very long, old information may be forgotten.

The professor's summary calls this a bottleneck: all relevant earlier information must be compressed into a single evolving vector.

Chapter 263. 7.3 RNN Limitation: Sequential Computation

RNNs process tokens one at a time.

You cannot easily parallelize over sequence length because:

$$h_t$$

depends on:

$$h_{t-1}$$

This makes training slower.

Chapter 264. 8. Vanishing and Exploding Gradients

When training RNNs, gradients are propagated through many time steps.

This is called **Backpropagation Through Time** or BPTT.

For long sequences, gradients may:

- vanish,
 - explode.
-

Chapter 265. 8.1 Vanishing Gradients

Gradients become very small.

Result:

- model struggles to learn long-range dependencies,
 - early tokens have little effect on training.
-

Chapter 266. **8.2 Exploding Gradients**

Gradients become very large.

Result:

- unstable updates,
 - loss may explode,
 - training can fail.
-

Chapter 267. **8.3 Why This Happens**

Backpropagation through many repeated steps multiplies many derivative terms.

If many derivatives are less than 1, gradients shrink.

If many derivatives are greater than 1, gradients grow.

The professor's summary says BPTT makes optimization sensitive to exploding or vanishing gradients because gradients must propagate through a long chain of dependencies.

Chapter 268. **9. LSTMs — Long Short-Term Memory Networks**

LSTMs were developed to improve RNNs.

They use gates to control information flow.

An LSTM has:

- cell state,
- hidden state,

- input gate,
- forget gate,
- output gate.

The professor's summary says LSTMs introduce gated mechanisms that control what information is written, kept, and exposed.

Chapter 269. 9.1 Main Idea of LSTM

Instead of forcing everything into a simple hidden state, LSTMs maintain a cell state that can preserve information over longer periods.

The gates decide:

Gate	Function
forget gate	what old information to remove
input gate	what new information to write
output gate	what information to expose

Chapter 270. 9.2 Why LSTMs Help

They help with:

- long-range dependencies,
 - vanishing gradients,
 - remembering important information,
 - forgetting irrelevant information.
-

Chapter 271. 9.3 LSTM Limitations

LSTMs are better than plain RNNs, but still limited.

They still process tokens sequentially.

They still have limited parallelism.

They can still struggle with very long context.

The Chapter 6 summary states that LSTMs help with longer dependencies but still process tokens sequentially, limiting parallelism and making very long-range reasoning difficult.

Chapter 272. **10. Why Attention Was Needed**

RNNs and LSTMs compress the past into a hidden state.

Attention changes this.

Instead of forcing everything into one vector, attention lets each token directly look at other tokens.

The central idea:

A token should be able to decide which previous tokens are most relevant.

The professor's summary says the Transformer allows a token to look back at the relevant context and determine which earlier tokens matter most, instead of relying only on a compressed hidden state.

Chapter 273. **11. Self-Attention**

Self-attention takes a sequence of token embeddings and produces contextualized embeddings.

Each token creates three vectors:

1. Query
 2. Key
 3. Value
-

Chapter 274. **11.1 Query Q**

The query represents:

What information am I looking for?

Chapter 275. **11.2 Key K**

The key represents:

What information do I offer for matching?

Chapter 276. **11.3 Value V**

The value represents:

What content should be retrieved if I am attended to?

The professor’s summary explains exactly this: query is what the token is looking for, key is what each token offers, and value contains the content to be aggregated.

Chapter 277. **12. Query, Key, and Value Projections**

Given token embeddings:

$$X \in \mathbb{R}^{T \times d}$$

we compute:

$$\begin{aligned} Q &= XW_Q \\ K &= XW_K \\ V &= XW_V \end{aligned}$$

where:

Symbol	Meaning
X	token embeddings
W_Q	query projection matrix
W_K	key projection matrix
W_V	value projection matrix
Q	queries
K	keys

Symbol	Meaning
V	values

These projections are learned.

Chapter 278. 13. Attention Scores

Attention compares queries and keys.

For token i attending to token j :

$$score_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

where:

Symbol	Meaning
q_i	query of token i
k_j	key of token j
d_k	key/query dimension
$\sqrt{d_k}$	scaling factor

The dot product measures compatibility.

If q_i and k_j align strongly, token i attends strongly to token j .

Chapter 279. 14. Why Divide by $\sqrt{d_k}$?

Without scaling, dot products can become large when d_k is large.

Large scores cause softmax to become too sharp.

Then gradients can become small or unstable.

So we use:

$$\frac{QK^T}{\sqrt{d_k}}$$

This stabilizes attention.

Chapter 280. **15. Softmax Attention Weights**

Scores are converted to attention weights using softmax:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Each row sums to 1.

Meaning:

For each token, the attention weights tell how much it attends to every other token.

Chapter 281. **16. Attention Output**

The output is a weighted sum of values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This is the core attention equation.

The professor's summary says attention scores are normalized with softmax, and the output for a token becomes a weighted combination of value vectors.

Chapter 282. **17. Why Values Are Needed**

A common question:

If attention weights tell us which tokens matter, why do we need values?

Because attention weights only say **where to look**.

Values say **what information to retrieve**.

The professor's sample question says relevance weights indicate which positions matter, but value vectors provide the content that is aggregated after relevance is decided.

Exam sentence:

Queries and keys determine relevance; values provide the information that is actually passed forward.

Chapter 283. **18. Why Separate Q, K, and V?**

Why not use one vector for all?

Because the roles are different:

Vector	Role
Query	what this token wants
Key	how this token can be found
Value	what content this token contributes

Keeping them separate gives the model more flexibility.

The professor's sample answer says separate projections allow the model to learn one geometry for deciding relevance and another for representing information to be passed forward.

Chapter 284. **19. Self-Attention as Learned Retrieval**

Self-attention behaves like learned retrieval.

For each token:

1. create a query,
2. compare it with all keys,
3. get attention weights,

4. retrieve weighted values.

So each token can dynamically collect information from other tokens.

This is why Transformers are powerful for context.

Chapter 285. **20. Causal Self-Attention**

In language generation, token t must not see future tokens.

When predicting:

$$x_t$$

the model can only use:

$$x_{<t}$$

not:

$$x_{>t}$$

So decoder-only Transformers use **causal masking**.

Causal masking prevents attention to future positions.

The professor's summary states that decoder-only language models must obey the autoregressive constraint: token t may only attend to tokens up to position t , not future tokens.

Chapter 286. **20.1 Why Causal Masking Is Necessary**

Without masking, the model could cheat.

Example:

Sentence:

“The cat sat on the mat”

When predicting “sat,” the model might look ahead and see “sat” in the input.

That would make training meaningless.

Causal masking ensures:

The model learns to predict the future from the past, not copy the future.

Chapter 287. **20.2 Causal Mask Matrix**

For a sequence length 4, allowed attention looks like:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Token 1 attends only to token 1.

Token 2 attends to tokens 1 and 2.

Token 3 attends to tokens 1, 2, and 3.

Token 4 attends to tokens 1, 2, 3, and 4.

Future positions are masked out.

Chapter 288. **21. Multi-Head Attention**

Instead of one attention operation, Transformers use multiple attention heads.

Each head has its own:

$$W_Q, W_K, W_V$$

Each head can learn different relationships.

One head may focus on syntax.

Another may focus on long-distance references.

Another may focus on local context.

The professor's summary says multi-head attention allows the model to capture different relationships at the same time, such as short-range syntax, long-range semantic references, or positional patterns.

Chapter 289. **21.1 Why Multi-Head Attention Is More Expressive**

Single-head attention creates one attention pattern.

Multi-head attention creates many attention patterns in parallel.

This allows the model to attend to different aspects of context simultaneously.

Exam sentence:

Multi-head attention is more expressive because different heads can specialize in different types of token relationships.

Chapter 290. **22. Transformer Block**

A standard Transformer block contains:

1. Multi-head self-attention
2. Feed-forward network
3. Residual connections
4. Layer normalization

The professor's summary lists these as the main components of a Transformer block.

Chapter 291. **22.1 Attention Sublayer**

This mixes information across tokens.

Meaning:

Tokens exchange information with each other.

Chapter 292. **22.2 Feed-Forward Network**

The FFN is applied independently to each token.

Usually:

$$FFN(x) = W_2\sigma(W_1x + b_1) + b_2$$

It increases representational capacity.

Attention handles token interaction.

FFN transforms each token representation.

The professor's summary says attention mixes information across tokens, while the FFN transforms information within each token representation.

Chapter 293. **22.3 Residual Connections**

Residual connections add the input back to the output:

$$x_{new} = x + F(x)$$

They help:

- preserve information,
 - improve gradient flow,
 - train deep networks.
-

Chapter 294. **22.4 Layer Normalization**

LayerNorm normalizes features within each token representation.

It stabilizes training.

It is especially important in Transformers.

The professor's summary says residual connections help preserve information and support gradient flow, while LayerNorm stabilizes training by normalizing features.

Chapter 295. **23. Positional Information**

Self-attention alone is permutation-invariant.

That means if we shuffle tokens, attention does not automatically know their order.

Example:

“dog bites man”

vs

“man bites dog”

Same words, different meaning.

The model needs position information.

So Transformers add positional encodings or positional embeddings.

The professor's summary states that self-attention by itself is permutation-invariant, so Transformers need positional information.

Chapter 296. **23.1 Types of Positional Encoding**

Common types:

1. sinusoidal positional encoding,
2. learned absolute positional embeddings,
3. relative positional embeddings,
4. rotary positional embeddings.

For this chapter, the main idea is:

The model needs some way to know token order.

Chapter 297. **24. Encoder-Only, Decoder-Only, and Encoder-Decoder Transformers**

There are three major Transformer families.

Chapter 298. **24.1 Encoder-Only Transformers**

Examples: BERT-style models.

They use bidirectional attention.

A token can attend to tokens on both left and right.

Good for:

- classification,
 - tagging,
 - retrieval,
 - representation learning,
 - understanding tasks.
-

Chapter 299. **24.2 Decoder-Only Transformers**

Examples: GPT-style models.

They use causal self-attention.

A token can only attend to previous tokens.

Good for:

- text generation,
- next-token prediction,
- chat models,
- autoregressive generation.

The professor's summary says decoder-only models are naturally suited for generation because they are trained autoregressively.

Chapter 300. **24.3 Encoder-Decoder Transformers**

Examples: original Transformer, T5-style models.

They have:

- encoder for input sequence,
- decoder for output sequence.

Good for:

- translation,
 - summarization,
 - sequence-to-sequence tasks.
-

Chapter 301. **25. GPT-Style Decoder-Only Transformer**

A GPT-style model is a decoder-only Transformer trained to predict the next token.

Input:

$$x_1, x_2, \dots, x_T$$

Output:

$$p_{\theta}(x_t \mid x_{<t})$$

for each position.

Training loss:

$$-\sum_t \log p_{\theta}(x_t \mid x_{<t})$$

This directly matches autoregressive factorization.

The Chapter 6 summary says GPT-style training is next-token prediction using cross-entropy against the true next token.

Chapter 302. **26. Tokenization**

Language models do not usually operate directly on raw words.

They operate on tokens.

A token can be:

- a character,
- a byte,
- a subword,
- a word,
- punctuation,
- part of a word.

Example:

The word:

“unbelievable”

might be tokenized as:

[un' believ' able]

or something similar.

Chapter 303. **27. Why Tokenization Matters**

Tokenization affects:

- sequence length,
- vocabulary size,
- memory cost,

- computation,
- rare word handling,
- representation granularity.

The professor's summary says tokenization is not minor preprocessing; it changes sequence length, meaning granularity, training efficiency, and what patterns the model can learn.

Chapter 304. **28. BPE — Byte Pair Encoding**

Byte Pair Encoding is a common tokenization method.

The main idea:

1. Start with small units, often bytes or characters.
2. Count frequent adjacent pairs.
3. Merge the most frequent pair.
4. Repeat many times.
5. Build a vocabulary of frequent subword units.

The Chapter 6 learning outcomes mention byte-level BPE tokenization, learning merges from raw UTF-8 bytes.

Chapter 305. **28.1 Why BPE Is Useful**

Word-level tokenization has problems:

- huge vocabulary,
- cannot handle unseen words well.

Character-level tokenization has problems:

- very long sequences,
- inefficient.

BPE is a compromise.

It represents common words compactly and rare words using smaller pieces.

The professor's summary says BPE allows frequent words to be represented compactly while still handling rare or unseen words through smaller pieces.

Chapter 306. 29. Attention Complexity

Self-attention compares every token with every other token.

For sequence length T , attention scores form a matrix:

$$T \times T$$

So attention cost is roughly:

$$O(T^2)$$

This is powerful but expensive for long sequences.

This is why long-context Transformers are computationally challenging.

Chapter 307. 30. Why Transformers Became Dominant

Transformers became dominant because they solve major problems of RNNs/LSTMs:

RNN/LSTM	Transformer
sequential processing	parallel training
compressed hidden state bottleneck	direct attention to context
struggles with long dependencies	better long-range interactions
slow for long sequences	faster training on GPUs
less scalable	highly scalable

The professor's summary says Transformers avoid the single-state bottleneck of RNNs and allow much greater parallelism during training, which is one reason they became dominant.

Chapter 308. 31. Transformer Intuition

A Transformer does two main things repeatedly:

1. **Attention:** let tokens talk to each other.
2. **FFN:** process each token's updated information.

Layer after layer, each token representation becomes more contextual.

For example, the word “bank” changes meaning depending on context:

- “river bank”
- “bank account”

Self-attention lets the word attend to surrounding tokens to form the correct meaning.

Chapter 309. **32. How Chapter 6 Connects to Later Chapters**

Chapter 310. **Connection to ViT**

Vision Transformers use the same Transformer idea, but tokens are image patches instead of text tokens.

Chapter 311. **Connection to VLMs**

Vision-language models use Transformers to process:

- text tokens,
- image tokens,
- multimodal tokens.

Chapter 9 writes VLM generation as:

$$p_{\theta}(y \mid x, v) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x, v)$$

which is exactly autoregressive factorization with text prompt x , visual input v , and output tokens y .

Chapter 312. Connection to SFT and DPO

Supervised fine-tuning trains the same next-token prediction objective, but on instruction-response data.

DPO and preference optimization use log-probabilities of chosen and rejected responses.

These log-probabilities come from the autoregressive model:

$$\sum_t \log p_\theta(y_t \mid y_{<t}, x)$$

Chapter 313. **33. Important Formula Summary**

Chapter 314. Autoregressive factorization

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{1:t-1})$$

Chapter 315. Language model next-token distribution

$$p(x_t \mid x_{<t}) = \text{Categorical}(\pi_t) \\ \pi_t = \text{softmax}(z_t)$$

Chapter 316. Negative log-likelihood loss

$$\mathcal{L} = - \sum_t \log p_\theta(x_t \mid x_{<t})$$

Chapter 317. RNN hidden update

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Chapter 318. Attention score

$$score_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

Chapter 319. Self-attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Chapter 320. Transformer FFN

$$FFN(x) = W_2\sigma(W_1x + b_1) + b_2$$

Chapter 321. 34. Most Likely Exam Questions

Chapter 322. Question 1: What is an autoregressive model?

An autoregressive model defines a probability distribution over a sequence by factorizing it into a product of next-token conditional probabilities:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{<t})$$

This means each token is predicted from the tokens before it. The factorization is both a probabilistic model of the full sequence and a practical training method because learning becomes next-token prediction at every position.

Chapter 323. Question 2: What is teacher forcing?

Teacher forcing is the training procedure where the model predicts the next token using the ground-truth previous tokens from the dataset. This differs from generation, where the model must condition on its own previously generated tokens. Teacher forcing makes training efficient

and stable, but generation may accumulate errors because the model's own mistakes become part of the context.

Chapter 324. Question 3: Why did Transformers replace RNNs and LSTMs?

RNNs and LSTMs process sequences sequentially and compress previous information into a hidden state. This creates a bottleneck and limits parallelism. Transformers use self-attention, allowing tokens to directly attend to relevant tokens across the context. This improves long-range dependency modeling and enables much greater parallelism during training.

Chapter 325. Question 4: What are queries, keys, and values in self-attention?

Queries represent what a token is looking for. Keys represent what each token offers for matching. Values contain the information to be retrieved if a token is attended to. Attention weights are computed from query-key similarity, and the output is a weighted sum of values.

Chapter 326. Question 5: Why are separate Q, K, and V projections useful?

Separate projections allow the model to distinguish between matching and content retrieval. Queries and keys define the relevance geometry, while values define what information is passed forward. If one vector served all roles, the model would have less flexibility. Separate Q, K, and V projections make attention behave like learned retrieval.

Chapter 327. Question 6: Why is causal masking needed in decoder-only Transformers?

Causal masking prevents each token from attending to future tokens. Without it, the model could cheat during training by seeing the token it is supposed to predict. Causal masking enforces the autoregressive constraint that each prediction can depend only on previous tokens.

Chapter 328. Question 7: What is multi-head attention and why is it useful?

Multi-head attention runs several attention mechanisms in parallel with different learned projections. Each head can specialize in different relationships, such as local syntax, long-range references, or semantic associations. This makes the model more expressive than a single attention head.

Chapter 329. Question 8: What is the role of the feed-forward network in a Transformer block?

Attention mixes information across tokens, while the feed-forward network transforms each token representation independently. The FFN increases representational capacity after contextual information has been gathered through attention.

Chapter 330. Question 9: Why do Transformers need positional encodings?

Self-attention alone is permutation-invariant, meaning it does not naturally know token order. Positional encodings or embeddings provide information about where each token appears in the sequence. Without positional information, the model cannot distinguish different word orders such as “dog bites man” and “man bites dog.”

Chapter 331. Question 10: What is BPE tokenization and why is it used?

Byte Pair Encoding builds subword tokens by repeatedly merging frequent symbol pairs. It provides a compromise between word-level and character-level tokenization: common words are represented compactly, while rare or unseen words can still be represented using smaller pieces. This improves vocabulary efficiency and sequence modeling.

Chapter 332. 35. Common Mistakes to Avoid

Chapter 333. Mistake 1: Saying autoregressive models predict all tokens independently

Wrong. Each token depends on previous tokens:

$$p(x_t \mid x_{<t})$$

Chapter 334. Mistake 2: Confusing training and generation

Training uses teacher forcing with ground-truth prefixes.
Generation uses the model's own generated tokens.

Chapter 335. Mistake 3: Thinking attention weights contain the content

Attention weights only decide relevance.
Values contain the content being aggregated.

Chapter 336. Mistake 4: Forgetting causal masking

Decoder-only Transformers must not see future tokens.

Chapter 337. Mistake 5: Thinking Transformers do not need position

Self-attention alone has no order awareness. Positional information is necessary.

Chapter 338. Mistake 6: Thinking BPE is unimportant preprocessing

Tokenization strongly affects sequence length, vocabulary, efficiency, and model behavior.

Chapter 339. 36. One-Page Open-Note Summary

An autoregressive model defines a sequence probability as a product of next-token conditionals: $p(x_{1:T}) = \prod_t p(x_t | x_{<t})$. This is both a probabilistic statement and a training strategy because modeling full sequences becomes next-token prediction. In language modeling, the model outputs logits, softmax converts them to probabilities, and the next token follows a categorical distribution. Training uses teacher forcing, where the model conditions on true previous tokens; generation uses the model's own previous outputs. RNNs and LSTMs were early sequence models that update a hidden state step by step, but they suffer from sequential computation, hidden-state bottlenecks, and long-range dependency difficulties. Transformers solve this using self-attention. Each token produces a query, key, and value. Query-key similarity determines attention weights, and values provide the content that is aggregated. Multi-head attention lets different heads capture different relationships. Decoder-only Transformers use causal masking so tokens cannot attend to the future. Transformer blocks combine multi-head attention, feed-forward networks, residual connections, and LayerNorm. Positional information is necessary because self-attention alone is order-invariant. BPE tokenization converts text into subword units, balancing vocabulary size and sequence length. GPT-style models are decoder-only Transformers trained with next-token cross-entropy.

Chapter 340. 37. Very Short Final Revision

Know these:

- Autoregressive = predict sequence one token at a time.
- Formula: $p(x_{1:T}) = \prod_t p(x_t | x_{<t})$.
- Training = teacher forcing with true previous tokens.
- Sampling = model uses its own generated tokens.
- Next token distribution = softmax logits $\rightarrow$ categorical.
- RNN = hidden state summary of past.
- LSTM = gated RNN for longer memory.
- Transformer replaces hidden-state bottleneck with attention.
- Query = what token wants.
- Key = what token offers for matching.
- Value = content to retrieve.
- Attention = $\text{softmax}(QK^T / \sqrt{d_k})V$.
- Causal mask prevents looking at future tokens.
- Multi-head attention captures multiple relation types.
- FFN transforms each token after attention.
- Positional encoding gives order.
- BPE = subword tokenization.
- GPT = decoder-only Transformer trained by next-token prediction.

Chapter 341. **Chapter 7 — Vision Transformers (ViT)**

Chapter 7 explains how the Transformer idea from language models is applied to images. The central idea is:

A Vision Transformer splits an image into fixed-size patches, treats those patches as visual tokens, and processes them using Transformer self-attention.

This is a major shift from CNNs. CNNs process images using local convolution filters. ViTs process images as token sequences.

The professor's flashcards state that the central architectural idea of a ViT is to split an image into fixed-size patches and treat them as a sequence of visual tokens.

Chapter 342. **1. Why Vision Transformers Matter**

Before ViTs, CNNs were the dominant architecture for image understanding.

CNNs use:

- local filters,
- convolution,
- pooling,
- hierarchical feature extraction,
- built-in locality.

Vision Transformers showed that strong image understanding does not require convolution as the only organizing principle. Instead, images can be processed using the same Transformer machinery used for language.

The Chapter 7 summary says ViTs matter because they showed that strong image understanding does not require convolution to be the only organizing principle, and because they became a bridge to modern vision-language and generative systems.

Chapter 343. **2. CNN View vs ViT View**

Chapter 344. **CNN View**

A CNN sees an image as a grid of pixels and applies local filters.

Example:

$$Image \rightarrow Convolution \rightarrow FeatureMaps \rightarrow Classifier$$

CNNs are good at detecting local patterns such as:

- edges,
 - corners,
 - textures,
 - shapes,
 - object parts.
-

Chapter 345. ViT View

A ViT sees an image as a sequence of patches.

Example:

$$Image \rightarrow Patches \rightarrow PatchTokens \rightarrow TransformerEncoder$$

The image becomes like a sentence, where each patch is like a word/token.

This is why ViTs connect naturally to language models and multimodal systems.

Chapter 346. 3. From Image to Patch Sequence

Suppose an image has size:

$$H \times W \times C$$

where:

Symbol	Meaning
H	height
W	width
C	number of channels, usually 3 for RGB

Now split the image into patches of size:

$$P \times P$$

The number of patches is:

$$N = \frac{H}{P} \times \frac{W}{P}$$

assuming H and W are divisible by P .

The Chapter 7 summary gives this exact idea: if an image has spatial size and is split into non-overlapping patches, the number of patches is determined by the image size divided by the patch size.

Chapter 347. Example

Image size:

$$224 \times 224$$

Patch size:

$$16 \times 16$$

Number of patches along height:

$$224/16 = 14$$

Number of patches along width:

$$224/16 = 14$$

Total patches:

$$N = 14 \times 14 = 196$$

So the image becomes a sequence of 196 visual tokens.

Chapter 348. 4. What Is a Visual Token?

A visual token is the embedding representation of one image patch.

For each patch:

1. take the $P \times P \times C$ pixels,
2. flatten them into a vector,
3. project that vector into an embedding dimension D .

If patch size is:

$$P \times P$$

and image has C channels, each patch contains:

$$P^2 C$$

values.

Flattened patch:

$$x_p \in \mathbb{R}^{P^2 C}$$

Then patch embedding:

$$e_p = x_p W_E + b_E$$

where:

$$e_p \in \mathbb{R}^D$$

So the image becomes:

$$X \in \mathbb{R}^{N \times D}$$

where:

Symbol	Meaning
N	number of patches/tokens
D	embedding dimension

Chapter 349. **5. Patch Embedding**

Patch embedding is the step that converts raw pixels into token vectors.

There are two equivalent views.

Chapter 350. **View 1: Linear Projection**

Each flattened patch is passed through a learned linear layer:

$$e_i = Wx_i + b$$

This converts a raw patch into a D -dimensional token.

Chapter 351. **View 2: Convolution**

Patch embedding can also be implemented as a convolution with:

- kernel size = patch size,
- stride = patch size,
- output channels = embedding dimension.

For example:

$$kernel = 16, stride = 16$$

This extracts non-overlapping patches and projects them at the same time.

The Chapter 7 summary explains that patch embedding can be understood either as a linear projection of flattened patches or as a convolution with kernel size and stride equal to the patch size.

Chapter 352. Important Exam Idea

Patch embedding is not yet full image understanding.

It simply creates the token interface for the Transformer.

The Transformer layers then learn relationships between these tokens.

Chapter 353. 6. Why the Token-Based View Matters

The professor's flashcards say the token-based view of images is important because it allows models to easily condition, scale, and mix image data across modalities such as text.

This is crucial.

Once images are tokens, we can use Transformer-style methods for:

- image classification,
- image generation,
- image-text alignment,
- multimodal reasoning,
- CLIP-style models,
- diffusion transformers,
- vision-language models.

In other words:

Tokenizing images creates a common language between vision and language models.

Text has tokens.

Images can have patch tokens.

Audio can have audio tokens.

Video can have spatiotemporal tokens.

This is why tokenization is a unifying idea in modern AI.

Chapter 354. 7. Positional Embeddings

A Transformer does not naturally know the order or location of tokens.

For text, this means it does not know word order.

For images, this means it does not know where each patch came from.

If we only give patch embeddings, the Transformer sees a set of patches, not a spatial image layout.

Therefore we add positional embeddings:

$$Z_0 = X + E_{pos}$$

where:

Symbol	Meaning
X	patch embeddings
E_{pos}	positional embeddings
Z_0	input sequence to Transformer

The Chapter 7 summary states that positional information is essential because a Transformer is not inherently aware of spatial arrangement.

Chapter 355. Why Positional Embedding Matters in Images

Imagine two images with the same patches but arranged differently.

Example:

- eyes above mouth = face,
- mouth above eyes = strange layout.

Without positional information, the Transformer may not distinguish these layouts well.

So positional embeddings tell the model:

This patch came from the top-left, this one from the center, this one from the bottom-right, etc.

Chapter 356. **8. Types of Positional Information**

Common options include:

Chapter 357. **1D Positional Embedding**

Flatten the image patches into a sequence:

$$1, 2, 3, \dots, N$$

Then assign each patch a sequence position.

This is simple but loses some explicit 2D structure.

Chapter 358. **2D Positional Embedding**

Use row and column information.

This better represents image geometry.

Chapter 359. **Relative Positional Embedding**

Represents relative distance between patches.

This can help model spatial relationships such as:

- nearby patches,
- far patches,
- above/below,
- left/right.

The Chapter 7 summary notes that positional design choices include flattened 1D positions, explicit 2D structure, or relative positional information, and that the choice affects spatial reasoning and resolution handling.

Chapter 360. 9. Class Token [*CLS*]

For image classification, ViT often adds a learnable class token to the beginning of the sequence.

The sequence becomes:

$$[\text{CLS}], e_1, e_2, \dots, e_N$$

The class token is a special token whose job is to collect global information from all patches.

After the Transformer encoder, the final representation of the class token is used for classification:

$$h_{\text{CLS}} \rightarrow \text{classifier} \rightarrow \text{logits}$$

The Chapter 7 summary says the class token gives the network a dedicated location in the sequence for accumulating image-level information through self-attention.

Chapter 361. Why the Class Token Works

Because of self-attention, the class token can attend to all image patches.

Over many layers, it gathers information about:

- objects,
- shapes,
- background,
- global layout,
- class identity.

Then the classifier uses it as a summary of the whole image.

Chapter 362. 10. Is Classification Possible Without a Class Token?

Yes.

The professor's sample question asks this directly.

Without a class token, we can aggregate patch tokens using:

- mean pooling,
- max pooling,
- attention pooling,
- learned pooling.

Then classify from the pooled representation.

The Chapter 7 summary sample answer says classification is still possible without a class token by aggregating patch tokens, but the model loses a dedicated sequence position specialized for accumulating image-level information.

Chapter 363. Class Token vs Mean Pooling

Method	Idea	Advantage	Disadvantage
Class token	special token gathers global info	task-specific summary	extra learned token
Mean pooling	average all patch outputs	simple and symmetric	less specialized
Attention pooling	learn weighted summary	flexible	more complex

Chapter 364. 11. Transformer Encoder in ViT

A ViT uses a stack of Transformer encoder blocks.

Each block contains:

1. Layer normalization
2. Multi-head self-attention
3. Residual connection
4. Layer normalization
5. MLP / feed-forward network
6. Residual connection

The Chapter 7 summary lists exactly these elements as the ViT encoder block structure.

Chapter 365. ViT Block Flow

Simplified:

$$\begin{aligned}Z' &= Z + \text{MSA}(\text{LN}(Z)) \\Z_{\text{next}} &= Z' + \text{MLP}(\text{LN}(Z'))\end{aligned}$$

where:

Symbol	Meaning
Z	token sequence
MSA	multi-head self-attention
MLP	feed-forward network
LN	layer normalization

Chapter 366. 12. Self-Attention in ViT

Self-attention lets every image patch interact with every other patch.

The attention formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

$$\begin{aligned}Q &= XW_Q \\K &= XW_K \\V &= XW_V\end{aligned}$$

The Chapter 7 summary says that attention allows each token to dynamically gather information from other tokens according to learned relevance scores.

Chapter 367. 13. Global Context

This is one of the most important advantages of ViTs.

In self-attention, every patch has a direct path to every other patch.

The flashcards say global context through self-attention means every token has a direct path to every other token, enabling coordination of structure across the whole image.

Chapter 368. Why Global Context Matters

Images often require long-range understanding.

Example:

To recognize a bird, the model may need to relate:

- beak,
- wings,
- tail,
- background,
- body shape.

These parts may be far apart in the image.

In a CNN, information from distant parts is combined gradually through deeper layers.

In ViT, self-attention allows direct interaction across the entire image.

Chapter 369. **14. CNN Locality vs ViT Global Attention**

Chapter 370. CNNs

CNNs focus locally at first.

A convolution kernel may only see:

$$3 \times 3$$

or:

$$5 \times 5$$

neighborhoods.

Global understanding emerges after many layers.

Chapter 371. ViTs

ViTs allow global interactions from the beginning.

Every patch can attend to every other patch in each attention layer.

Chapter 372. Comparison

Feature	CNN	ViT
Basic unit	convolution filter	patch token
Inductive bias	strong locality	weaker locality
Global context	built gradually	direct through attention
Data efficiency	often better with less data	often needs more data/pretraining
Scalability	strong but architecture-specific	scales well with data/compute
Multimodal compatibility	less direct	very natural

Chapter 373. 15. Why ViTs Often Need More Data

CNNs have strong inductive biases:

- local patterns matter,
- nearby pixels are related,
- same object can appear in different places,
- features are hierarchical.

These biases help CNNs learn from fewer examples.

ViTs have weaker built-in image-specific assumptions.

They rely more on data to learn visual structure.

The Chapter 7 summary says ViTs do not build in the same strong local inductive biases as CNNs, so they often depend more heavily on data, augmentation, pretraining, or architectural refinements.

Chapter 374. 16. Attention Complexity in ViT

Self-attention compares every patch with every other patch.

If there are N patches, attention uses an:

$$N \times N$$

attention matrix.

So complexity is roughly:

$$O(N^2)$$

The number of patches is:

$$N = \frac{H}{P} \times \frac{W}{P}$$

If patch size gets smaller, N increases.

Attention becomes expensive.

Chapter 375. Example

Image:

$$224 \times 224$$

Patch size:

$$16 \times 16$$

Number of patches:

$$14 \times 14 = 196$$

Attention matrix:

$$196 \times 196$$

Now if patch size is:

$$8 \times 8$$

Number of patches:

$$28 \times 28 = 784$$

Attention matrix:

$$784 \times 784$$

That is much more expensive.

The Chapter 7 summary notes that self-attention scales quadratically with sequence length, so making patches too small can become expensive because the number of tokens grows rapidly.

Chapter 376. **17. Patch Size Tradeoff**

Patch size is an important design choice.

Chapter 377. **Smaller patches**

Advantages:

- more fine detail,
- better spatial resolution,
- useful for dense tasks.

Disadvantages:

- more tokens,
- more attention cost,
- more memory,
- slower training/inference.

Chapter 378. Larger patches

Advantages:

- fewer tokens,
- cheaper computation,
- faster.

Disadvantages:

- loses fine detail,
 - coarser representation.
-

Chapter 379. 18. ViT for Classification

A standard ViT classification pipeline:

1. Input image:

$$x \in \mathbb{R}^{H \times W \times C}$$

2. Split into patches.
3. Flatten patches.
4. Project patches to embeddings.
5. Add positional embeddings.
6. Add class token.
7. Pass through Transformer encoder.
8. Take final class token representation.
9. Feed to classifier head.
10. Output class logits.

The Chapter 7 summary gives the conceptual flow: image becomes patch tokens, class token is added, encoder contextualizes the sequence, and a task-specific head produces the output.

Chapter 380. 19. Classification Head

The classification head maps the final representation to class logits:

$$\text{logits} = Wh_{\text{CLS}} + b$$

Then softmax gives class probabilities:

$$p(y \mid x) = \text{softmax}(\text{logits})$$

Training uses cross-entropy loss.

Chapter 381. **20. ViT as a General-Purpose Encoder**

A ViT is not only for classification.

The encoder can produce image representations for many tasks.

Examples:

- classification,
- retrieval,
- image-text alignment,
- visual question answering,
- masked image modeling,
- image generation,
- segmentation,
- object detection,
- multimodal reasoning.

The Chapter 7 summary says the ViT encoder can serve different tasks: classification uses a head on the class token, while representation or multimodal learning uses a general image embedding from the class token, mean pooling, or another pooling mechanism.

Chapter 382. **21. ViTs and Multimodal Learning**

ViTs are very important for VLMs.

Why?

Because ViTs turn images into token embeddings.

Then these visual embeddings can be aligned with text embeddings or passed into a language model.

Examples:

- CLIP uses an image encoder, often ViT-based.
- VLMs use visual tokens/features as input to language decoders.
- Multimodal models combine image tokens and text tokens.

This connects directly to Chapter 8.

Chapter 383. **22. ViTs in Generative Models**

ViTs are also important in image generation.

In diffusion models, the denoising network was traditionally U-Net-based.

But later models use Transformer backbones.

One famous example is **Diffusion Transformer (DiT)**.

DiT replaces or competes with U-Net-style architectures by using Transformer blocks over latent/image patches.

The flashcards mention DiT and ask what it demonstrated regarding scaling compared to traditional U-Nets.

Chapter 384. **23. What Did DiT Show?**

The important idea is:

Transformer backbones can scale very effectively for diffusion image generation.

Traditional diffusion models often used U-Nets. DiT showed that diffusion models can use Transformer architectures and benefit strongly from scaling model size and compute.

This is conceptually important because it suggests Transformers are not just for language or classification. They can also be powerful generative backbones.

The Chapter 7 summary says the rise of transformer backbones in generative image models is significant because it suggests the Transformer has become a general representation mechanism for both understanding and generation.

Chapter 385. **24. ViTs Beyond Classification**

Vision Transformers are used in:

Chapter 386. **1. Masked image modeling**

Similar to masked language modeling.

Mask some image patches and train the model to reconstruct or predict them.

Chapter 387. **2. CLIP-style image-text alignment**

Use ViT to encode images into embeddings aligned with text embeddings.

Chapter 388. **3. Diffusion Transformers**

Use Transformer blocks to model denoising or generation in image/latent token space.

Chapter 389. **4. Segmentation and detection**

Use patch/token features for dense prediction.

Chapter 390. **5. Multimodal systems**

Use visual tokens as input to language models.

The Chapter 7 summary emphasizes that ViTs are now central in masked-image modeling, multimodal encoders, and image generation.

Chapter 391. **25. Strengths of Vision Transformers**

ViTs have several strengths.

Chapter 392. **1. Global attention**

Every patch can interact with every other patch.

Chapter 393. **2. Scales well**

With enough data and compute, ViTs scale very effectively.

Chapter 394. **3. Flexible architecture**

The same Transformer block can be used across vision, language, and multimodal models.

Chapter 395. **4. Natural multimodal compatibility**

Images become tokens, just like text.

Chapter 396. **5. Useful representations**

ViT encoders can be reused across many tasks.

The Chapter 7 summary says ViTs provide flexible global interaction, align naturally with transformer-based multimodal systems, and scale effectively with data and compute.

Chapter 397. **26. Limitations of Vision Transformers**

ViTs also have weaknesses.

Chapter 398. **1. Quadratic attention cost**

Self-attention scales as:

$$O(N^2)$$

where N is the number of patches.

Chapter 399. **2. Weak local inductive bias**

Unlike CNNs, ViTs do not strongly assume local structure.

Chapter 400. **3. Data hungry**

ViTs often need more data or pretraining.

Chapter 401. **4. Patch size tradeoff**

Small patches are expensive; large patches lose detail.

Chapter 402. **5. Resolution handling**

Changing image resolution can require positional embedding adaptation.

The Chapter 7 summary identifies these limitations: quadratic scaling, weaker local inductive biases, and heavier dependence on data, augmentation, pretraining, or refinements.

Chapter 403. **27. CNNs vs ViTs — Critical Evaluation**

A common wrong claim is:

ViTs are simply better than CNNs because they are newer.

This is too simplistic.

CNNs are still strong because they have image-specific inductive biases.

ViTs are strong when trained at scale and integrated into multimodal/generative systems.

The professor's sample answer says the better conclusion is that ViTs and CNNs offer different advantages, and performance depends on data scale, compute budget, and task structure.

Chapter 404. **Comparison Table**

Aspect	CNN	ViT
Input representation	pixel grid	patch tokens
Core operation	convolution	self-attention
Local bias	strong	weaker
Global context	gradual	direct
Data efficiency	often better on small data	usually better at scale
Multimodal integration	less direct	natural
Compute scaling	efficient local ops	attention can be expensive
Best use	vision tasks with limited data	large-scale pretraining, multimodal, generative

Chapter 405. **28. Why Residual Connections and LayerNorm Matter**

Residual connections and LayerNorm do not directly add visual knowledge.

But they make deep ViTs trainable.

Chapter 406. **Residual connections**

Help preserve information and gradients.

$$x_{out} = x + F(x)$$

Without residuals, deep models are harder to optimize.

Chapter 407. **LayerNorm**

Stabilizes activation scale.

This makes training more predictable.

The professor's sample question asks why these matter even though they do not add visual knowledge. The answer: they play an optimization and stability role, making deep Transformer stacks trainable.

Chapter 408. **29. ViT Pipeline Summary**

A complete ViT pipeline:

*Image → Patches → Flatten → Linear Projection → Patch Tokens → Add Position
→ Add Class Token → Transformer Encoder → Class Token Output
→ MLP Head → Prediction*

This is the flow you should be able to explain in the exam.

Chapter 409. **30. Important Formula Summary**

Chapter 410. Number of patches

$$N = \frac{H}{P} \times \frac{W}{P}$$

Chapter 411. Patch vector size

$$P^2 C$$

Chapter 412. Patch embedding

$$e_i = W_E x_i + b_E$$

Chapter 413. Token matrix

$$X \in \mathbb{R}^{N \times D}$$

Chapter 414. Add positional embeddings

$$Z_0 = X + E_{pos}$$

or with class token:

$$Z_0 = [x_{CLS}; x_1E; x_2E; \dots; x_NE] + E_{pos}$$

Chapter 415. Self-attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Chapter 416. ViT block

$$\begin{aligned} Z' &= Z + MSA(LN(Z)) \\ Z_{next} &= Z' + MLP(LN(Z')) \end{aligned}$$

Chapter 417. Classification

$$logits = Wh_{CLS} + b$$

Chapter 418. 31. Most Likely Exam Questions

Chapter 419. Question 1: What is the central idea of a Vision Transformer?

A Vision Transformer splits an image into fixed-size patches, embeds each patch into a vector, treats those patch embeddings as a sequence of visual tokens, adds positional information, and processes the sequence using Transformer self-attention. This allows the model to apply the same token-based architecture used in language to image understanding.

Chapter 420. Question 2: Why is the token-based view of images important?

The token-based view allows images to be processed by Transformers and makes vision compatible with language and multimodal models. Once images are represented as tokens, they can be aligned with text tokens, used in vision-language models, processed by generative Transformers, or integrated into architectures originally developed for sequences.

Chapter 421. Question 3: How is an image converted into tokens in ViT?

An image of size $H \times W \times C$ is split into non-overlapping patches of size $P \times P$. The number of patches is $N = (H/P)(W/P)$. Each patch is flattened into a vector of size P^2C , then passed through a learned linear projection to produce a D -dimensional patch embedding. These embeddings become the visual token sequence.

Chapter 422. Question 4: Why are positional embeddings necessary in ViT?

Self-attention by itself does not know the spatial order of patches. Without positional embeddings, the Transformer may treat the image as an unordered set of patch tokens. Positional embeddings tell the model where each patch came from, allowing it to reason about spatial layout and relationships.

Chapter 423. Question 5: What is the role of the class token?

The class token is a learnable token added to the patch sequence. Through self-attention, it gathers information from all patches and becomes a global image representation. For classification, the final class-token representation is passed to a classifier head to produce class logits.

Chapter 424. Question 6: Can ViT classify images without a class token?

Yes. The model can aggregate final patch tokens using mean pooling, max pooling, attention pooling, or another summarization method. The tradeoff is that without a class token, the model loses a dedicated token specialized for collecting global task-specific information.

Chapter 425. Question 7: Why is global context useful in ViTs?

Self-attention allows each patch token to attend directly to every other patch token. This gives the model a direct path for coordinating distant image regions, which is useful for recognizing objects and understanding global structure.

Chapter 426. Question 8: Compare CNNs and ViTs.

CNNs use local convolutional filters and strong image-specific inductive biases such as locality and translation-related structure. This makes them data-efficient and effective for many vision tasks. ViTs use patch tokens and self-attention, giving them flexible global interactions and strong scalability, especially with large datasets and pretraining. Neither is universally better; performance depends on task, data, and compute.

Chapter 427. Question 9: Why can small patches be expensive in ViT?

Smaller patches increase the number of tokens N . Since self-attention scales roughly as $O(N^2)$, reducing patch size can greatly increase memory and computation. Smaller patches preserve more detail but are more expensive.

Chapter 428. Question 10: Why are transformer backbones in generative image models conceptually significant?

They show that Transformers are not only language or classification architectures. They can serve as general-purpose backbones for image generation, such as in diffusion transformers. This suggests that token-based Transformer architectures can unify image understanding, multimodal alignment, and generation.

Chapter 429. 32. Common Mistakes to Avoid

Chapter 430. Mistake 1: Thinking a ViT processes pixels one by one

A ViT usually processes patches, not individual pixels.

Chapter 431. **Mistake 2: Forgetting positional embeddings**

Patch tokens alone do not tell the Transformer where each patch came from.

Chapter 432. **Mistake 3: Saying ViTs are always better than CNNs**

ViTs and CNNs have different strengths. CNNs can be more data-efficient; ViTs often shine at scale and in multimodal systems.

Chapter 433. **Mistake 4: Thinking class token is mandatory**

It is common but not strictly required. Pooling can also be used.

Chapter 434. **Mistake 5: Ignoring patch size**

Patch size affects token count, detail, memory, and compute.

Chapter 435. **Mistake 6: Thinking attention cost is linear**

Self-attention is quadratic in the number of tokens.

Chapter 436. **33. One-Page Open-Note Summary**

A Vision Transformer applies the Transformer architecture to images by splitting an image into fixed-size patches and treating each patch as a visual token. An image of size $H \times W \times C$ with patch size $P \times P$ becomes $N = (H/P)(W/P)$ patches. Each patch is flattened and projected into a D -dimensional embedding. Positional embeddings are added because self-attention does not naturally know spatial location. For classification, a learnable class token is often added to the sequence, and its final representation is used as a global image summary. The ViT encoder

consists of Transformer blocks with multi-head self-attention, MLPs, residual connections, and LayerNorm. Self-attention gives ViTs global context because every patch can directly attend to every other patch. Compared with CNNs, ViTs have weaker local inductive bias but stronger global flexibility and better compatibility with multimodal and generative systems. ViTs can require more data or pretraining and have quadratic attention cost with respect to the number of patches. Beyond classification, ViTs are used in CLIP, vision-language models, masked image modeling, and diffusion transformers.

Chapter 437. **34. Very Short Final Revision**

Know these:

- ViT = image split into patches → patch tokens → Transformer encoder.
- Number of patches: $N = (H/P)(W/P)$.
- Patch embedding = flatten patch + linear projection.
- Positional embeddings are necessary for spatial layout.
- Class token gathers global image information.
- Self-attention gives global context.
- ViT block = attention + MLP + residuals + LayerNorm.
- CNNs have strong locality bias; ViTs have flexible global attention.
- Smaller patches = more detail but much more compute.
- ViTs are important for VLMs and generative image models.
- DiT shows Transformers can be strong diffusion/generative backbones.

Chapter 438. **Chapter 8 — Vision–Language Models and CLIP**

Chapter 8 explains how models connect **images** and **language**. The main idea is:

A Vision–Language Model learns relationships between visual meaning and textual meaning.

So instead of only asking:

“What class is this image?”

a VLM can answer questions like:

“Which image matches this text?”

“What caption describes this image?”

“What object is shown here?”

“Can this image be classified using labels the model was never directly trained on?”

The chapter focuses especially on **CLIP**, which is a dual-encoder contrastive VLM.

Chapter 439. **1. What Is a Vision–Language Model?**

A **Vision–Language Model**, or **VLM**, is a model that connects images and text.

It learns that:

- an image of a dog,
- the phrase “a dog,”
- and the caption “a brown dog running on grass”

all refer to related meanings, even though one is visual and the others are textual.

The Chapter 8 summary says a VLM learns a joint relationship between images and language, connecting visual meaning and linguistic meaning in a shared framework.

Chapter 440. **Why VLMs Matter**

Traditional vision models are usually trained on fixed label sets.

Example:

A classifier trained on:

`{cat, dog, car}`

can only classify into those categories.

A VLM uses language supervision, so it can work with flexible text descriptions.

This allows:

- open-vocabulary recognition,
- image-to-text retrieval,
- text-to-image retrieval,
- zero-shot classification,
- image captioning,
- visual question answering,
- multimodal instruction following,

- grounding language in images.

The Chapter 8 notes say VLMs are important because language enables flexible, compositional supervision instead of fixed-category labels.

Chapter 441. 2. Training Data for VLMs

Most VLMs are trained on paired image-text data:

$$(x_i, t_i)$$

where:

Symbol	Meaning
x_i	image
t_i	associated text/caption/alt-text/metadata

Example pairs:

Image	Text
image of dog	“a dog running in a park”
image of pizza	“a cheese pizza on a table”
image of ship	“a cargo ship at sea”

At web scale, these pairs are often noisy.

For example, the caption may be incomplete, vague, incorrect, or biased.

The Chapter 8 notes say modern VLM performance is shaped by objective design, model capacity, data quality, diversity, and filtering.

Chapter 442. 3. Two Main Families of VLMs

Chapter 8 organizes VLMs into two dominant architecture families:

1. **Dual-encoder contrastive models**
2. **Fusion / encoder–decoder models**

The Chapter 8 learning objectives specifically ask students to explain these two families and why they support different tasks.

Chapter 443. **4. Family 1: Dual-Encoder Contrastive VLMs**

The most famous example is **CLIP**.

A dual-encoder VLM has two separate encoders:

1. image encoder,
2. text encoder.

The image encoder maps an image to a vector:

$$f_I(x)$$

The text encoder maps text to a vector:

$$f_T(t)$$

Then both vectors are placed in the same shared embedding space.

Chapter 444. **4.1 Shared Embedding Space**

The goal is:

matched image-text pairs should be close, and mismatched pairs should be far apart.

Example:

Image:

dog running on grass

Text:

“a brown dog running on grass”

These should have high similarity.

Image:

dog running on grass

Text:

“a red airplane in the sky”

These should have low similarity.

The Chapter 8 summary says CLIP-like models train matched image-text pairs to be close and mismatched pairs to be far apart.

Chapter 445. **5. Why Dual Encoders Are Efficient**

In a dual-encoder model, images and texts can be encoded independently.

This is very useful for retrieval.

For example, suppose you have 1 million images.

You can precompute image embeddings:

$$f_I(x_1), f_I(x_2), \dots, f_I(x_N)$$

Then for a text query:

“a black cat sitting on a sofa”

you compute:

$$f_T(t)$$

and compare it to all stored image embeddings.

This makes large-scale retrieval efficient.

The Chapter 8 notes say dual encoders are extremely efficient for retrieval because embeddings can be cached and searched with nearest-neighbor methods.

Chapter 446. **6. CLIP**

CLIP stands for **Contrastive Language–Image Pretraining**.

The main idea:

Train an image encoder and a text encoder so that correct image-text pairs have high similarity and incorrect pairs have low similarity.

CLIP is not trained as a normal classifier with fixed classes.

Instead, it learns a shared semantic embedding space.

That is why it can do zero-shot classification and retrieval.

Chapter 447. **7. CLIP Architecture**

CLIP has:

Chapter 448. **Image encoder**

Often:

- ResNet, or
- Vision Transformer.

It maps:

$$x_i \rightarrow h_i$$

Chapter 449. **Text encoder**

Usually Transformer-based.

It maps:

$$t_i \rightarrow g_i$$

Then embeddings are normalized:

$$\hat{h}_i = \frac{h_i}{\|h_i\|}$$
$$\hat{g}_i = \frac{g_i}{\|g_i\|}$$

The Chapter 8 notes explain that CLIP-like dual encoders map image and text into a shared embedding space and usually L2-normalize the embeddings.

Chapter 450. **8. Why L2 Normalization Matters**

L2 normalization makes each embedding have unit length.

After normalization:

$$\|\hat{h}_i\| = 1$$
$$\|\hat{g}_i\| = 1$$

Then the dot product becomes cosine similarity:

$$\hat{h}_i^\top \hat{g}_j = \cos(\hat{h}_i, \hat{g}_j)$$

This means similarity depends mainly on direction, not magnitude.

The Chapter 8 summary says normalization makes the score focus on direction in semantic space rather than raw vector magnitude.

Chapter 451. **Simple Intuition**

If embeddings are not normalized, a vector with large magnitude may get high similarity even if its direction is not semantically correct.

Normalization prevents this.

It makes comparison more stable and meaningful.

Exam sentence:

L2 normalization makes CLIP similarity behave like cosine similarity, so matching depends on semantic direction rather than vector magnitude.

Chapter 452. 9. CLIP Similarity Matrix

Suppose a minibatch contains N matched image-text pairs:

$$(x_1, t_1), (x_2, t_2), \dots, (x_N, t_N)$$

The model computes image embeddings:

$$I_1, I_2, \dots, I_N$$

and text embeddings:

$$T_1, T_2, \dots, T_N$$

Then it computes all pairwise similarities:

$$S_{ij} = \frac{I_i^\top T_j}{\tau}$$

where:

Symbol	Meaning
S_{ij}	similarity between image i and text j
I_i	normalized image embedding
T_j	normalized text embedding
τ	temperature
$i = j$	correct matched pair
$i \neq j$	mismatched pair

The Chapter 8 summary describes the CLIP score matrix as computing similarities between every image embedding and every text embedding in the minibatch.

Chapter 453. **10. What the Similarity Matrix Looks Like**

For a batch of 4 pairs:

$$S = \begin{bmatrix} I_1 T_1 & I_1 T_2 & I_1 T_3 & I_1 T_4 \\ I_2 T_1 & I_2 T_2 & I_2 T_3 & I_2 T_4 \\ I_3 T_1 & I_3 T_2 & I_3 T_3 & I_3 T_4 \\ I_4 T_1 & I_4 T_2 & I_4 T_3 & I_4 T_4 \end{bmatrix}$$

The diagonal entries are correct matches:

$$S_{11}, S_{22}, S_{33}, S_{44}$$

The off-diagonal entries are incorrect matches.

Training wants:

- diagonal high,
- off-diagonal low.

Chapter 454. **11. Contrastive Learning**

CLIP uses **contrastive learning**.

Contrastive learning means:

Pull positive pairs together and push negative pairs apart.

Chapter 455. **Positive pair**

Correct image-text pair:

$$(x_i, t_i)$$

Chapter 456. Negative pair

Incorrect image-text pair:

$$(x_i, t_j), i \neq j$$

The Chapter 8 flashcards emphasize that in CLIP, matched image-text pairs are positive pairs and mismatched pairs are negative pairs.

Chapter 457. 12. Why Negative Pairs Matter

If CLIP only increased similarity for correct pairs, it might collapse and make everything similar to everything.

Negative pairs define the geometry of separation.

They teach the model:

This image matches this caption, but not those other captions.

The professor's sample answer says CLIP must learn not only that correct pairs are similar, but also that incorrect pairs are less similar; otherwise the embedding space could collapse into trivial solutions.

Chapter 458. 13. Image-to-Text Loss

For each image i , CLIP treats the correct text i as the label among all texts in the batch.

Probability that image i matches text j :

$$p(j | i) = \frac{\exp(S_{ij})}{\sum_{k=1}^N \exp(S_{ik})}$$

The correct text is $j = i$.

So image-to-text loss:

$$\mathcal{L}_{I \rightarrow T} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii})}{\sum_{j=1}^N \exp(S_{ij})}$$

This says:

For each image, choose the correct text among all batch texts.

Chapter 459. 14. Text-to-Image Loss

Symmetrically, for each text j , CLIP treats the correct image j as the label among all images.

Probability that text j matches image i :

$$p(i | j) = \frac{\exp(S_{ij})}{\sum_{k=1}^N \exp(S_{kj})}$$

Text-to-image loss:

$$\mathcal{L}_{T \rightarrow I} = -\frac{1}{N} \sum_{j=1}^N \log \frac{\exp(S_{jj})}{\sum_{i=1}^N \exp(S_{ij})}$$

This says:

For each text, choose the correct image among all batch images.

Chapter 460. 15. Symmetric CLIP Loss

CLIP combines both directions:

$$\mathcal{L}_{CLIP} = \frac{1}{2} (\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I})$$

This is called **symmetric contrastive loss**.

The Chapter 8 learning objectives specifically include deriving the CLIP-style similarity matrix and the symmetric contrastive loss.

Chapter 461. Why Symmetric?

Because we want both:

1. image retrieves correct text,
2. text retrieves correct image.

If we only train one direction, alignment may be weaker.

Symmetric loss makes the shared space useful for both text-to-image and image-to-text retrieval.

Chapter 462. 16. Temperature / Logit Scale

CLIP divides similarity by a temperature:

$$S_{ij} = \frac{I_i^\top T_j}{\tau}$$

or equivalently multiplies by a logit scale:

$$S_{ij} = \alpha I_i^\top T_j$$

where:

$$\alpha = \frac{1}{\tau}$$

Chapter 463. What Temperature Does

Temperature controls how sharp the softmax distribution is.

Chapter 464. Low temperature

- similarities become sharper,
- model focuses more strongly on hard negatives,
- optimization becomes more aggressive.

Chapter 465. **High temperature**

- probabilities become smoother,
- less emphasis on hard negatives,
- weaker separation.

The Chapter 8 summary says temperature controls how sharply the model distinguishes the best match from the rest and shapes how strongly learning emphasizes ranking precision among candidate pairs.

Chapter 466. **Exam Sentence**

Temperature is not cosmetic; it changes the optimization dynamics by controlling how strongly the contrastive loss focuses on separating the correct pair from competing negatives.

Chapter 467. **17. CLIP Gradient Intuition**

CLIP gradients do two things:

Chapter 468. **Pull positive pairs together**

Correct image and text embeddings are moved closer:

$$I_i \leftrightarrow T_i$$

Chapter 469. **Push negative pairs apart**

Incorrect image-text pairs are moved farther:

$$I_i \leftrightarrow T_j, i \neq j$$

The Chapter 8 learning objectives include interpreting CLIP gradients as positive pull toward matched pairs and negative push away from batch negatives.

Chapter 470. **18. Zero-Shot Classification with CLIP**

CLIP can classify images without training a classifier head for the target classes.

This is called **zero-shot classification**.

Instead of training a classifier, we write text prompts for each class.

Example classes:

{cat, dog, car}

Prompts:

- “a photo of a cat”
- “a photo of a dog”
- “a photo of a car”

Then:

1. embed the image,
2. embed each prompt,
3. compute similarities,
4. choose the class with highest similarity.

The Chapter 8 summary says zero-shot classification works because classification is reframed as language-conditioned similarity matching rather than a fixed classifier head.

Chapter 471. **Why This Works**

CLIP has already learned to align images with language.

So at test time, new labels can be introduced as text.

The model does not need a new output layer.

Exam sentence:

CLIP zero-shot classification works because class labels are converted into text prompts and compared to the image embedding in the shared image-text space.

Chapter 472. **19. Why Prompt Wording Matters**

Different prompts can produce different text embeddings.

Example:

- “cat”
- “a photo of a cat”
- “a blurry photo of a cat”
- “a close-up image of a cat”

These may perform differently.

The Chapter 8 summary notes that prompt wording matters because the text encoder is sensitive to phrasing.

Chapter 473. **Prompt Ensembling**

To improve zero-shot accuracy, we can use multiple prompts per class and average embeddings.

Example for class “dog”:

- “a photo of a dog”
- “an image of a dog”
- “a blurry photo of a dog”
- “a close-up of a dog”

Then average the text embeddings.

This reduces dependence on one prompt wording.

Chapter 474. **20. Retrieval with CLIP**

CLIP is excellent for retrieval.

Chapter 475. **Text-to-image retrieval**

Input:

“a red car on a street”

Find images with embeddings most similar to the text embedding.

Chapter 476. Image-to-text retrieval

Input image.

Find captions/texts with embeddings most similar to image embedding.

Because image and text embeddings are computed independently, retrieval is efficient.

The Chapter 8 summary says dual-encoder models like CLIP are excellent for retrieval because both images and texts can be embedded independently and compared efficiently.

Chapter 477. 21. Retrieval Metrics: Recall@K

For retrieval, a common metric is **Recall@K**.

Recall@K asks:

Is the correct match in the top K retrieved results?

For example:

Chapter 478. Recall@1

Correct match must be ranked first.

Chapter 479. Recall@5

Correct match must be among top 5.

Chapter 480. Recall@10

Correct match must be among top 10.

The Chapter 8 learning objectives include retrieval metrics such as Recall@K as an evaluation axis.

Chapter 481. **22. Why CLIP Is Not a Caption Generator**

CLIP gives a compatibility score between image and text.

It does not directly generate text word by word.

It can answer:

Does this text match this image?

But it does not naturally answer:

Generate a caption for this image.

For captioning, you need a generative decoder or fusion model.

The Chapter 8 summary says CLIP does not directly define an autoregressive distribution over output words, so caption generation usually requires a generative decoder or another model on top.

Chapter 482. **23. Limitations of Dual Encoders**

Dual encoders are powerful but limited.

Chapter 483. **Main limitations**

1. global alignment may miss local details,
2. weak spatial grounding,
3. poor for counting,
4. poor for detailed reasoning,
5. cannot generate text directly,
6. may rely on spurious correlations,
7. sensitive to prompts.

The Chapter 8 notes say dual encoders are efficient and scalable, but global alignment can miss fine-grained spatial grounding and they are harder to use for multi-step reasoning without a generative or fusion component.

Chapter 484. **24. Family 2: Fusion / Encoder–Decoder VLMs**

The second major VLM family is **fusion-based** or **encoder–decoder** models.

These models combine visual and text information inside the model.

Instead of producing one image vector and one text vector separately, they allow tokens from one modality to interact with tokens from the other modality.

The Chapter 8 notes say fusion models fuse visual tokens with text tokens inside a Transformer, often using cross-attention and conditional generation objectives.

Chapter 485. **25. How Fusion Models Work**

A typical fusion VLM has:

1. image encoder,
2. text decoder,
3. cross-attention between text tokens and visual features.

The image encoder creates visual memory:

$$V = Enc_{image}(x)$$

The text decoder generates tokens:

$$y_1, y_2, \dots, y_T$$

using:

- masked self-attention over previous text tokens,
 - cross-attention to visual memory.
-

Chapter 486. 26. Conditional Generation Objective

Fusion models often train with:

$$p_{\theta}(y \mid x) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x)$$

where:

Symbol	Meaning
x	image
y	generated caption/answer
y_t	token at time t
$y_{<t}$	previous generated tokens

The Chapter 8 learning objectives include writing down the conditional generation objective for fusion models.

Chapter 487. Example

Image:

dog running in grass

Caption:

“A brown dog is running through grass.”

The model learns:

$$\begin{aligned} p(A \mid image) \\ p(\text{brown} \mid image, A) \\ p(\text{dog} \mid image, A \text{ brown}) \end{aligned}$$

and so on.

This is autoregressive generation conditioned on image features.

Chapter 488. **27. Self-Attention vs Cross-Attention in Fusion Models**

Chapter 489. **Self-attention**

Text tokens attend to previous text tokens.

This builds language context.

Example:

The word “dog” attends to “A brown.”

Chapter 490. **Cross-attention**

Text tokens attend to image features.

This connects language generation to visual evidence.

Example:

When generating “brown,” the decoder attends to the region/features showing the dog’s color.

The Chapter 8 summary says self-attention builds a contextual representation of the text prefix, while cross-attention allows the text representation to look into the image.

Chapter 491. **28. Why Fusion Models Are Good for Captioning and VQA**

Fusion models are better when the task requires detailed interaction between vision and language.

Examples:

- captioning,
- visual question answering,
- counting objects,
- describing spatial relations,

- grounded reasoning,
- instruction following.

Why?

Because each generated token can attend to image features.

The Chapter 8 summary says fusion models are often better for tasks requiring localized evidence, such as counting, spatial relations, and questions about specific regions.

Chapter 492. **29. Fusion Model Strengths**

Fusion models are strong for:

- caption generation,
- VQA,
- image-conditioned dialogue,
- multimodal instruction following,
- detailed grounding,
- region-sensitive reasoning.

Chapter 493. **30. Fusion Model Limitations**

Fusion models can be:

- more expensive,
- slower because generation is autoregressive,
- harder to scale for retrieval,
- more complex to train,
- more prone to hallucination if not grounded well.

The Chapter 8 summary notes that fusion models are often more expensive, more sequential at inference time, and less naturally suited to large-scale retrieval than pure dual encoders.

Chapter 494. **31. Dual Encoder vs Fusion Model**

Feature	Dual Encoder / CLIP	Fusion / Encoder–Decoder
---------	---------------------	--------------------------

Feature	Dual Encoder / CLIP	Fusion / Encoder–Decoder
Image/text interaction	after separate encoding	inside model through attention
Main output	similarity score	generated text / answer
Best for	retrieval, zero-shot classification	captioning, VQA, instruction following
Efficiency	very efficient retrieval	more expensive
Generation	no direct text generation	yes
Fine-grained grounding	weaker	stronger
Example	CLIP	BLIP, Flamingo, LLaVA-style systems

Chapter 495. 32. VLM Evaluation

VLMs should not be evaluated with one metric only.

Important axes:

1. retrieval performance,
2. zero-shot classification accuracy,
3. caption quality,
4. VQA accuracy,
5. robustness,
6. bias/fairness,
7. grounding,
8. hallucination,
9. efficiency.

The Chapter 8 learning objectives include evaluation axes beyond a single benchmark, such as retrieval Recall@K, zero-shot accuracy, robustness, bias/fairness, latency, memory, and energy.

Chapter 496. 33. Bias and Spurious Correlations

VLMs learn from large image-text datasets, often from the web.

This can introduce:

- social bias,
- demographic bias,
- background shortcuts,
- object-context shortcuts,
- cultural bias,

- harmful stereotypes.

Example:

If most images of “doctor” in training data show one demographic, the model may learn biased associations.

The Chapter 8 summary warns that if training data contain repeated shortcuts, the model may associate concepts with backgrounds, demographics, or stylistic artifacts rather than intended objects or relations.

Chapter 497. **34. Weak Grounding**

A model may correctly identify the overall scene but fail at details.

Example:

Image has:

- two dogs,
- one ball.

Model says:

“Three dogs playing with two balls.”

The caption sounds plausible but is not grounded.

The Chapter 8 summary identifies weak grounding as a VLM failure mode where the model appears correct globally but fails on local detail, counting, or spatial reasoning.

Chapter 498. **35. Hallucination**

Hallucination means the model describes something not actually supported by the image.

Example:

Image: dog on grass.

Model says:

“A dog playing with a red frisbee.”

But there is no frisbee.

The Chapter 8 summary says generative multimodal models may produce plausible descriptions that are not actually supported by the image.

Chapter 499. **36. Distribution Shift**

A VLM trained on web images may fail in special domains such as:

- medical images,
- satellite images,
- underwater images,
- industrial images,
- rare species,
- multilingual contexts.

The Chapter 8 summary says performance may degrade when moving from web images to medical, aerial, underwater, or multilingual settings.

Chapter 500. **37. Distillation for Efficient VLMs**

Large VLMs are powerful but expensive.

Smaller models such as TinyCLIP or MobileCLIP aim to reduce:

- memory,
- latency,
- energy,
- deployment cost.

Distillation trains a small student model to mimic a larger teacher model.

The student may mimic:

- embeddings,
- similarity matrix,
- predictions,
- ranking behavior.

The Chapter 8 notes include distilling a large teacher CLIP into a small student CLIP using embedding regression and similarity-matrix KL distillation.

Chapter 501. Why Distillation Is Important

In CLIP, we care not only about top-1 classification.

We care about preserving the geometry of the image-text embedding space.

The Chapter 8 summary says VLM quality is not only about top-1 accuracy, but also whether the semantic space preserves meaningful relationships across images and texts.

Chapter 502. 38. t-SNE Visualization

t-SNE is used to visualize high-dimensional embeddings in 2D.

For CLIP, we can visualize:

- image embeddings,
- text embeddings,
- prompt embeddings.

If CLIP works well, semantically related images and texts should cluster together.

The Chapter 8 learning objectives include visualizing joint embedding spaces using t-SNE and interpreting where images, captions, and prompts land.

Chapter 503. 39. Important VLM Systems and Directions

Chapter 8 names important systems and directions:

- CLIP,
- ALIGN,
- OpenCLIP,
- DataComp,
- SigLIP,

- CoOp / adapters,
- OWL-ViT,
- Grounding DINO,
- BLIP-2,
- Flamingo,
- LLaVA,
- TinyCLIP,
- MobileCLIP.

The Chapter 8 learning objectives list these systems and directions explicitly.

Chapter 504. What You Need to Know Conceptually

System / direction	Main idea
CLIP	contrastive image-text alignment
ALIGN	large-scale image-text alignment
OpenCLIP	open-source CLIP-style training
DataComp	dataset quality/filtering for CLIP-like models
SigLIP	alternative contrastive objective
CoOp	prompt learning/adaptation
OWL-ViT	open-vocabulary localization/detection
Grounding DINO	language-grounded detection
BLIP-2	connects vision encoders to language models
Flamingo	multimodal few-shot/instruction-style learning
LLaVA	visual instruction tuning
TinyCLIP/MobileCLIP	efficient/small VLMs

Chapter 505. 40. Important Formula Summary

Chapter 506. Image encoder

$$I_i = f_I(x_i)$$

Chapter 507. Text encoder

$$T_i = f_T(t_i)$$

Chapter 508. L2 normalization

$$\hat{I}_i = \frac{I_i}{\|I_i\|}$$
$$\hat{T}_i = \frac{T_i}{\|T_i\|}$$

Chapter 509. Similarity score

$$S_{ij} = \frac{\hat{I}_i^\top \hat{T}_j}{\tau}$$

Chapter 510. Image-to-text loss

$$\mathcal{L}_{I \rightarrow T} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii})}{\sum_{j=1}^N \exp(S_{ij})}$$

Chapter 511. Text-to-image loss

$$\mathcal{L}_{T \rightarrow I} = -\frac{1}{N} \sum_{j=1}^N \log \frac{\exp(S_{jj})}{\sum_{i=1}^N \exp(S_{ij})}$$

Chapter 512. Symmetric CLIP loss

$$\mathcal{L}_{CLIP} = \frac{1}{2} (\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I})$$

Chapter 513. Fusion generation objective

$$p_\theta(y \mid x) = \prod_{t=1}^T p_\theta(y_t \mid y_{<t}, x)$$

Chapter 514. **41. Most Likely Exam Questions**

Chapter 515. **Question 1: What is a Vision–Language Model?**

A Vision–Language Model learns relationships between visual and linguistic information. It connects image semantics and text semantics so that images, captions, prompts, and descriptions can be compared, retrieved, generated, or reasoned over in a shared multimodal framework.

Chapter 516. **Question 2: What are the two main VLM architecture families?**

The two main families are dual-encoder contrastive models and fusion/encoder–decoder models. Dual encoders process image and text separately and align them in a shared embedding space, making them efficient for retrieval and zero-shot classification. Fusion models combine visual and textual information inside the model using cross-attention or joint processing, making them better for captioning, VQA, and instruction following.

Chapter 517. **Question 3: What is the main idea of CLIP?**

CLIP trains an image encoder and text encoder using contrastive learning. Matched image-text pairs are pulled together in embedding space, while mismatched pairs are pushed apart. The result is a shared semantic space where images can be compared directly with text prompts, enabling retrieval and zero-shot classification.

Chapter 518. **Question 4: Why are negative pairs important in CLIP?**

Negative pairs teach the model what should not match. Without negative examples, the model could make many unrelated images and texts similar, causing collapse or poor discrimination. Negative pairs define the geometry of separation in the embedding space.

Chapter 519. **Question 5: What is the CLIP similarity matrix?**

For a batch of N matched image-text pairs, CLIP computes all pairwise similarities between image embeddings and text embeddings:

$$s_{ij} = \frac{I_i^\top T_j}{\tau}$$

The diagonal entries are correct matches, and off-diagonal entries are mismatches. The contrastive loss encourages diagonal similarities to be high and off-diagonal similarities to be low.

Chapter 520. Question 6: Why is the CLIP loss symmetric?

The loss is symmetric because CLIP trains both image-to-text and text-to-image retrieval. For each image, the correct text should rank highest among all texts; for each text, the correct image should rank highest among all images. This improves bidirectional alignment.

Chapter 521. Question 7: What does temperature do in CLIP?

Temperature controls the sharpness of the softmax over similarities. Lower temperature makes the model focus more strongly on distinguishing the best match from hard negatives. Higher temperature smooths the distribution and weakens the contrastive pressure. Therefore, temperature directly affects optimization and embedding geometry.

Chapter 522. Question 8: Why does CLIP support zero-shot classification?

CLIP supports zero-shot classification because class labels can be written as natural-language prompts. The image is embedded, each class prompt is embedded, and the class with highest image-text similarity is selected. No new classifier head is needed because CLIP already learned image-language alignment.

Chapter 523. Question 9: Why is CLIP good for retrieval but not direct caption generation?

CLIP is good for retrieval because images and texts are encoded independently into a shared space and compared efficiently. However, CLIP only scores compatibility between an image and

text; it does not generate text token by token. Caption generation requires a generative decoder or fusion-based model.

Chapter 524. Question 10: Why are fusion models better for VQA and captioning?

Fusion models allow text tokens to attend to visual features through cross-attention. This means each generated word or answer token can depend on specific image information. This makes fusion models better for tasks requiring detailed grounding, spatial reasoning, counting, or step-by-step multimodal generation.

Chapter 525. 42. Common Mistakes to Avoid

Chapter 526. Mistake 1: Thinking CLIP is a caption generator

CLIP scores image-text compatibility. It does not directly generate captions.

Chapter 527. Mistake 2: Forgetting negative pairs

Contrastive learning needs both positives and negatives.

Chapter 528. Mistake 3: Thinking zero-shot means no training at all

CLIP is pretrained. Zero-shot means no additional task-specific classifier training.

Chapter 529. Mistake 4: Ignoring prompt wording

Prompt wording affects text embeddings and zero-shot performance.

Chapter 530. Mistake 5: Confusing dual encoders with fusion models

Dual encoders compare separate embeddings. Fusion models combine modalities internally.

Chapter 531. **Mistake 6: Thinking high retrieval score means perfect grounding**

CLIP may match globally while missing fine spatial or counting details.

Chapter 532. **43. One-Page Open-Note Summary**

A Vision–Language Model connects visual meaning and textual meaning. The two main architecture families are dual-encoder contrastive models and fusion/encoder–decoder models. CLIP is a dual-encoder model with an image encoder and a text encoder. Both produce normalized embeddings in a shared space. CLIP computes a similarity matrix between all image and text embeddings in a minibatch, where diagonal entries are correct matches and off-diagonal entries are negatives. The symmetric contrastive loss trains image-to-text and text-to-image retrieval simultaneously. Positive pairs are pulled together, and negative pairs are pushed apart. Temperature controls how sharply the model separates correct matches from hard negatives. CLIP enables zero-shot classification by comparing an image embedding to text prompt embeddings such as “a photo of a cat.” It is also strong for retrieval because embeddings can be precomputed and compared efficiently. However, CLIP does not directly generate captions and may miss fine-grained grounding. Fusion VLMs combine visual and textual information internally, often through cross-attention, and are better for captioning, VQA, instruction following, and localized reasoning. VLM evaluation should include retrieval, zero-shot accuracy, robustness, fairness, grounding, hallucination, and efficiency.

Chapter 533. **44. Very Short Final Revision**

Know these:

- VLM = connects image and text semantics.
- Two families: dual encoder and fusion model.
- CLIP = contrastive image-text alignment.
- Image encoder + text encoder → shared embedding space.
- L2 normalization → cosine similarity.
- Similarity matrix S_{ij} : diagonal = correct pairs.
- Contrastive loss pulls positives together and pushes negatives apart.
- Symmetric loss trains both image→text and text→image.

- Temperature controls sharpness and hard-negative pressure.
- Zero-shot classification = compare image to class prompts.
- Retrieval = nearest image/text embedding search.
- CLIP is not a caption generator.
- Fusion models use cross-attention and can generate captions/answers.
- VLM failure modes: bias, weak grounding, hallucination, distribution shift.

Chapter 534. Chapter 9 — Training and Adapting Vision–Language Models

Chapter 9 explains how we adapt a pretrained Vision–Language Model after it already knows something about images and text. The chapter’s main idea is that adaptation methods form a **ladder of interventions**. Some methods only change the **context** given to the model, while others change the model’s **parameters**, **preferences**, or **reward behavior**.

The chapter frames a VLM as an autoregressive policy:

$$p_{\theta}(y \mid x, v) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x, v)$$

where:

Symbol	Meaning
x	text instruction / prompt
v	visual input, such as an image
y	generated response
y_t	token at generation step t
$y_{<t}$	previous generated tokens
θ	model parameters

So the model generates an answer token by token, conditioned on the instruction and the image. Chapter 9 uses this fixed autoregressive view to compare adaptation methods by asking: **what exactly is being changed?**

Chapter 535. 1. The Big Idea: Adaptation Ladder

The adaptation ladder includes:

1. **Zero-shot prompting**
2. **Few-shot prompting**
3. **Retrieval-Augmented Generation, RAG**
4. **Supervised Fine-Tuning, SFT**
5. **LoRA / QLoRA**
6. **DPO / preference tuning**
7. **RLHF / reward-based optimization**
8. **RLOO-style reinforcement learning**

These methods are not simply “weak to strong.” They intervene at different levels.

Method	What changes?
Zero-shot	prompt only
Few-shot	prompt + examples in context
RAG	external retrieved evidence
SFT	model behavior through labeled examples
LoRA / QLoRA	trainable parameterization
DPO	preference alignment
RLHF / RLOO	reward-optimized behavior

The professor’s summary emphasizes that the ladder is not merely a ranking from weak to strong, but a map of what is being modified: prompt, memory, labels, parameterization, preferences, or rewards.

Chapter 536. **2. Zero-Shot Prompting**

Zero-shot prompting means asking the model to perform a task without giving examples.

Example:

“Look at this image and describe the main object.”

The model must answer using only:

- its pretrained knowledge,
- the instruction,
- the image.

No training happens.

No weights change.

Mathematically:

$$p_{\theta}(y \mid x, v)$$

where θ is fixed.

Chapter 537. Why Zero-Shot Works

Zero-shot works because pretrained VLMs have already learned many image-text relationships from large datasets.

For example, if the model has seen many image-caption pairs during pretraining, it may already understand:

- objects,
- scenes,
- colors,
- actions,
- spatial relations,
- common visual descriptions.

So a new instruction can activate existing knowledge.

Chapter 538. Strengths of Zero-Shot Prompting

- cheapest method,
 - no training data needed,
 - fast to try,
 - no parameter updates,
 - useful for general tasks.
-

Chapter 539. Weaknesses of Zero-Shot Prompting

- limited for specialized domains,
- sensitive to prompt wording,
- may hallucinate,
- may fail on domain-specific knowledge,
- no persistent improvement.

Exam sentence:

Zero-shot prompting changes only the instruction context; it does not change model parameters.

Chapter 540. **3. Few-Shot Prompting**

Few-shot prompting gives the model a few examples inside the prompt.

Example:

Example 1:

Image: damaged pipe

Answer: This image shows a pipe leakage.

Example 2:

Image: cracked wall

Answer: This image shows structural wall damage.

Now answer for this image:

...

The model uses these examples as temporary guidance.

No weights are updated.

The chapter summary states that few-shot prompting inserts demonstration examples into the context window and changes behavior without gradient descent because the Transformer hidden states depend on the full context.

Chapter 541. **Few-Shot Is Context Adaptation, Not Parameter Learning**

This is very important.

Few-shot examples do not update:

$$\theta$$

They only change:

$$x$$

the input context.

So the model behaves differently because the context is different.

Exam sentence:

Few-shot learning in prompting is not parameter learning; it is context-conditioned temporary adaptation.

Chapter 542. Strengths of Few-Shot Prompting

- no training required,
 - demonstrates desired answer style,
 - helps with formatting,
 - can improve task understanding,
 - useful when only a few examples exist.
-

Chapter 543. Weaknesses of Few-Shot Prompting

- consumes context window,
 - sensitive to example quality,
 - not persistent,
 - may fail for complex domain knowledge,
 - examples can bias the model.
-

Chapter 544. 4. Zero-Shot vs Few-Shot

Feature	Zero-shot	Few-shot
Examples provided?	no	yes
Parameters changed?	no	no
Cost	very low	low
Adaptation type	instruction only	context demonstrations
Persistence	temporary	temporary
Good for	general tasks	format/style/task clarification

Chapter 545. **5. Retrieval-Augmented Generation — RAG**

RAG means the model retrieves external information before answering.

Instead of relying only on internal knowledge, the system:

1. receives query and image,
2. retrieves relevant documents,
3. conditions the answer on those documents.

The chapter summary says RAG adds inference-time adaptation by introducing external memory; the model retrieves documents and conditions generation on them.

Chapter 546. **Why RAG Is Needed**

A VLM may fail because:

- it lacks domain knowledge,
- its knowledge is outdated,
- the task requires specific documents,
- the answer must be grounded in evidence.

Example:

A medical VLM may visually recognize a skin lesion but need current clinical guidelines.

RAG gives it relevant evidence.

Chapter 547. **RAG Formula**

The chapter writes RAG conceptually as:

$$p_{\theta,\eta}(y \mid x, v) = \sum_{z \in Z} p_{\eta}(z \mid q(x, v)) p_{\theta}(y \mid x, v, z)$$

where:

Symbol	Meaning
z	retrieved document/evidence
Z	retrieved document set
$(p_{\eta}(z q(x, v)))$	
$q(x, v)$	query created from text/image input
$(p_{\theta}(y x, v, z))$	

This means:

The final answer is generated by considering possible retrieved documents as evidence.

Chapter 548. RAG as a Latent-Variable Model

RAG can be viewed as a latent-variable model where the retrieved document z is a latent variable.

The model does not only ask:

Which documents are retrieved?

It also asks:

Which document actually explains the correct answer?

The chapter emphasizes posterior responsibility:

$$p(z | x, v, y)$$

Conceptually, this measures:

Which retrieved document helped explain the gold answer?

The summary says this separates retrieval plausibility from explanatory usefulness.

Chapter 549. 6. Retrieval Plausibility vs Explanatory Usefulness

This is a subtle but important exam idea.

A retrieved document may look relevant to the query but not actually help answer correctly.

Example:

Question:

“What safety regulation applies to this equipment?”

Retrieved document A:

- talks about equipment generally,
- seems relevant.

Retrieved document B:

- contains the exact regulation clause,
- actually supports the answer.

RAG should prefer evidence that explains the final answer, not just text that looks similar.

Exam sentence:

RAG is not only about retrieving top-k similar documents; the important issue is whether the retrieved evidence actually supports the generated answer.

Chapter 550. **7. When RAG Is the Best First Choice**

If the problem is missing knowledge, RAG is often better than SFT or DPO.

Example failure:

The model gives fluent answers but confuses closely related species because it lacks domain knowledge.

The chapter’s sample answer says RAG is often the most principled first intervention because it directly addresses missing knowledge by injecting external evidence at inference time.

Chapter 551. **Why Not SFT First?**

SFT teaches the model to imitate answers.

But if the model lacks evidence, SFT may just teach better-looking guesses.

Chapter 552. **Why Not DPO First?**

DPO teaches preference between answers.

But it does not directly provide missing facts.

So if the bottleneck is knowledge, use RAG first.

Chapter 553. **8. RAG Strengths and Weaknesses**

Chapter 554. **Strengths**

- adds external knowledge,
- can update knowledge without retraining,
- improves grounding,
- supports citations/evidence,
- useful for domain-specific tasks.

Chapter 555. **Weaknesses**

- depends on retrieval quality,
 - wrong retrieval causes wrong answers,
 - documents may conflict,
 - model may ignore retrieved evidence,
 - adds system complexity,
 - latency increases.
-

Chapter 556. **9. Supervised Fine-Tuning — SFT**

SFT means training the model on labeled examples.

For VLMs, data may be triples:

$$(x, v, y)$$

where:

- x = instruction,
- v = image,
- y = correct answer.

The model is trained to maximize:

$$p_{\theta}(y \mid x, v)$$

or minimize next-token cross-entropy:

$$\mathcal{L}_{SFT} = - \sum_t m_t \log p_{\theta}(y_t \mid y_{<t}, x, v)$$

where:

Symbol	Meaning
m_t	mask indicating which tokens count in the loss
y_t	target token
x	instruction
v	image

The chapter summary explains that SFT optimizes next-token maximum likelihood on labeled triples and that the mask determines which tokens count toward the loss.

Chapter 557. 10. Why Loss Masking Matters in SFT

In chat-style training, the input may contain:

- system message,
- user instruction,
- image tokens,
- assistant answer.

Usually, we only want the model to learn to predict the assistant's answer, not the user prompt.

So we mask the loss:

$$m_t = 1$$

for assistant response tokens, and:

$$m_t = 0$$

for system/user/image tokens.

The chapter emphasizes that in multimodal instruction tuning, the practical question is not only “what is the target answer?” but also “which token positions define the learning signal?”

Chapter 558. Example

Prompt:

User: What is in the image?

Answer:

Assistant: A red car is parked beside a tree.

The loss should be applied to:

“A red car is parked beside a tree.”

not to:

“User: What is in the image?”

Exam sentence:

In SFT, masking ensures that training optimizes the assistant response rather than forcing the model to predict the user prompt or system text.

Chapter 559. 11. SFT Strengths and Weaknesses

Chapter 560. Strengths

- persistent adaptation,
- teaches task format,

- improves domain behavior,
- can specialize model,
- uses standard cross-entropy training.

Chapter 561. Weaknesses

- requires labeled data,
 - can overfit,
 - can reduce generality,
 - expensive for large models,
 - may teach imitation without factual grounding,
 - quality depends on dataset quality.
-

Chapter 562. 12. LoRA

LoRA stands for **Low-Rank Adaptation**.

LoRA is not a new objective. It is a parameter-efficient way to fine-tune a model.

Instead of updating all model weights, LoRA freezes the original pretrained weight matrix:

$$W$$

and learns a low-rank update:

$$\Delta W = BA$$

So the adapted weight becomes:

$$W' = W + \Delta W$$

$$W' = W + BA$$

where:

Symbol	Meaning
W	frozen pretrained weight
A, B	small trainable matrices
BA	low-rank update

Symbol	Meaning
--------	---------

r	rank of the update
-----	--------------------

The chapter summary states that LoRA keeps the pretrained weight frozen and learns a low-rank update, reducing trainable parameters from $d_{\text{in}}d_{\text{out}}$ to roughly $r(d_{\text{in}} + d_{\text{out}})$.

Chapter 563. Why LoRA Saves Parameters

Suppose:

$$W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$$

Full fine-tuning updates:

$$d_{\text{out}}d_{\text{in}}$$

parameters.

LoRA learns:

$$\begin{aligned} A &\in \mathbb{R}^{r \times d_{\text{in}}} \\ B &\in \mathbb{R}^{d_{\text{out}} \times r} \end{aligned}$$

Trainable parameters:

$$rd_{\text{in}} + rd_{\text{out}} = r(d_{\text{in}} + d_{\text{out}})$$

If:

$$r \ll d$$

this is much smaller.

Chapter 564. **13. LoRA Is Parameterization, Not the Objective**

This is one of the most important points in Chapter 9.

SFT tells us **what objective** to optimize:

$$-\log p_{\theta}(y \mid x, v)$$

LoRA tells us **which parameters are allowed to move**:

$$\Delta W = BA$$

So SFT and LoRA are not competing at the same level.

You can do SFT using LoRA.

The chapter summary says SFT defines what objective is optimized, while LoRA defines which parameters are allowed to move and in what form.

Chapter 565. **Exam Sentence**

LoRA is a parameter-efficient fine-tuning mechanism, not a separate training objective. It can be used to optimize an SFT, DPO, or other objective while updating only low-rank adapter parameters.

Chapter 566. **14. Which Layers Are Adapted with LoRA?**

The chapter emphasizes adapting both attention and MLP projections.

Chapter 567. **Attention projections**

- q_{proj}
- k_{proj}

- *v_proj*
- *o_proj*

These affect information routing.

They determine:

- what is queried,
- what is matched,
- what is attended,
- what is returned to the residual stream.

Chapter 568. MLP projections

- *gate_proj*
- *up_proj*
- *down_proj*

These affect nonlinear transformation.

The chapter sample answer says attention projections affect routing, while MLP projections affect transformation, and many downstream tasks need both.

Chapter 569. 15. QLoRA

QLoRA means Quantized LoRA.

It combines:

1. quantized base model,
2. LoRA adapters.

The base model is stored in low precision, often 4-bit.

The LoRA adapters are trained in higher precision.

This greatly reduces memory cost.

Chapter 570. Why QLoRA Matters

Large VLMs are expensive to fine-tune.

QLoRA allows fine-tuning on smaller hardware because:

- base weights are quantized,
- only adapter weights are trained,
- memory footprint is much lower.

Exam sentence:

QLoRA enables parameter-efficient fine-tuning by keeping the base model quantized while training small LoRA adapter weights.

Chapter 571. 16. Full Fine-Tuning vs LoRA vs QLoRA

Method	What is trained?	Memory cost	Flexibility
Full fine-tuning	all weights	very high	highest
LoRA	low-rank adapters	much lower	strong
QLoRA	adapters + quantized base	lowest	strong but constrained

Chapter 572. 17. DPO — Direct Preference Optimization

SFT trains the model to imitate one gold answer.

DPO trains the model using preferences.

A preference dataset contains:

$$(x, v, y^+, y^-)$$

where:

Symbol	Meaning
y^+	preferred/chosen response
y^-	rejected response
x	instruction

Symbol	Meaning
v	image

DPO encourages the model to assign higher relative probability to the preferred response than the rejected response.

The chapter summary says DPO compares a preferred response with a rejected response and encourages the policy to increase the preferred answer's relative log-probability compared with a frozen reference policy.

Chapter 573. **18. Why DPO Uses a Reference Policy**

DPO compares the trained policy to a frozen reference policy.

The reference policy helps prevent the model from drifting too far away from the original behavior.

Conceptually:

Improve preference alignment, but do not destroy the base model.

Chapter 574. **DPO Intuition**

Given two answers:

Preferred:

“The image shows a red fire truck parked beside a building.”

Rejected:

“The image shows a blue airplane.”

DPO increases probability of the preferred answer relative to the rejected one.

Chapter 575. **19. DPO vs SFT**

Feature	SFT	DPO
Data type	input + gold answer	input + preferred/rejected answers
Objective	imitate target	prefer chosen over rejected
Supervision	absolute answer	relative preference
Good for	task learning	alignment
Risk	learns dataset style	may optimize preference over truth

Chapter 576. **20. DPO Does Not Guarantee Factual Correctness**

This is very important.

Preference is not the same as truth.

If humans prefer polished but incorrect answers, DPO may make the model more polished but less factual.

The chapter summary warns that relative preference is not identical to factual correctness, and if preference pairs reward style more than truth, the model may become better aligned to human taste while becoming worse grounded scientifically.

Exam sentence:

DPO aligns the model with preferences, but preference alignment is not automatically factual grounding.

Chapter 577. **21. RLHF — Reinforcement Learning from Human Feedback**

RLHF treats generation as a reinforcement learning problem.

The model is a policy.

It generates tokens.

A reward model or human feedback scores the final response.

The goal is to maximize expected reward.

Chapter 578. **RL Interpretation of Autoregressive Generation**

The chapter maps VLM generation into RL terms:

RL term	VLM generation
state	prompt, image, generated prefix
action	next token
policy	next-token distribution
trajectory	full generated answer
reward	scalar score for answer

The chapter sample answer says RLHF does not abandon the language-model viewpoint; it reinterprets token generation in RL terms.

Chapter 579. **22. Policy**

The policy is:

$$\pi_{\theta}(y_t \mid y_{<t}, x, v)$$

This is the model's probability distribution over the next token.

It chooses actions token by token.

Chapter 580. **23. Trajectory**

A full generated response is a trajectory:

$$y = (y_1, y_2, \dots, y_T)$$

The reward is often assigned after the full answer is generated.

Example:

$$R(y, x, v)$$

Chapter 581. **24. RL Objective**

The goal is to maximize expected reward:

$$J(\theta) = \mathbb{E}_{y \sim \pi_\theta} [R(y, x, v)]$$

This means:

Generate answers that receive high reward.

Chapter 582. **25. PPO**

PPO stands for **Proximal Policy Optimization**.

It is used in many RLHF pipelines.

Its main idea is:

Improve the policy, but do not let it move too far from the old/reference policy in one update.

This prevents unstable training.

The chapter summary mentions PPO as the standard trust-region intuition in RL-based alignment.

Chapter 583. **26. RLOO — Reinforce Leave-One-Out**

RLOO is a simpler RL method discussed in the chapter.

The idea:

1. For the same prompt/image, generate several completions.
2. Score each completion.
3. Compare each completion to the average reward of the others.

4. Increase probability of completions that score above the baseline.
5. Decrease probability of completions below the baseline.

The chapter summary says RLOO generates several completions for the same prompt, scores them, and compares each completion to the average reward of the others.

Chapter 584. Why RLOO Is Useful

The leave-one-out baseline reduces variance.

For completion i , define baseline:

$$b_i = \frac{1}{K-1} \sum_{j \neq i} R_j$$

Advantage:

$$A_i = R_i - b_i$$

If:

$$A_i > 0$$

the completion is better than the others, so increase its probability.

If:

$$A_i < 0$$

the completion is worse, so decrease its probability.

The chapter says RLOO is attractive because it uses a relative baseline for the same prompt and avoids some heavier PPO-style machinery.

Chapter 585. **27. Adaptation Layers: RAG, LoRA/SFT, RL**

The chapter asks us to understand adaptation as operating on different layers.

Method	Layer
RAG	evidence layer
LoRA/SFT	policy-parameter layer
RL alignment	behavioral objective layer

The sample answer states:

- RAG changes what the model sees,
 - LoRA/SFT changes how the model maps seen information to outputs,
 - RL alignment changes what output behavior is incentivized.
-

Chapter 586. **28. Choosing the Right Adaptation Method**

Different problems require different methods.

Chapter 587. **Problem: prompt misunderstanding**

Use:

- better zero-shot prompt,
- few-shot examples.

Chapter 588. **Problem: missing domain knowledge**

Use:

- RAG.

Chapter 589. **Problem: needs persistent task specialization**

Use:

- SFT,
- LoRA.

Chapter 590. Problem: memory-limited fine-tuning

Use:

- QLoRA.

Chapter 591. Problem: answers are correct but not preferred style

Use:

- DPO.

Chapter 592. Problem: need optimize complex reward

Use:

- RLHF / RLOO.

Chapter 593. 29. Budget Ladder

The chapter frames adaptation by budget and risk.

Method	Cost	Risk	Persistence
Zero-shot	very low	low	no
Few-shot	low	low-medium	no
RAG	medium	retrieval errors	no model change
SFT	medium-high	overfitting	yes
LoRA	lower than full FT	adapter limits	yes
QLoRA	lower memory	quantization issues	yes
DPO	high	preference $\neq$ truth	yes
RLHF/RLOO	highest	reward hacking	yes

The chapter summary says zero-shot and few-shot are cheap but limited; RAG adds grounding without changing the model; SFT and LoRA create reusable specialization; DPO aligns relative preferences; RL methods are most expressive but expensive and easy to misuse if reward is poor.

Chapter 594. **30. Reward Hacking**

Reward hacking happens when the model learns to exploit the reward function rather than actually improve.

Example:

If reward favors long answers, the model may produce unnecessarily long responses.

If reward favors confident tone, the model may become confidently wrong.

So RL alignment depends heavily on reward quality.

Exam sentence:

RL methods are powerful but risky because the model optimizes the reward it is given, not necessarily the true human goal.

Chapter 595. **31. Important Formula Summary**

Chapter 596. VLM autoregressive policy

$$p_{\theta}(y \mid x, v) = \prod_{t=1}^T p_{\theta}(y_t \mid y_{<t}, x, v)$$

Chapter 597. RAG marginal over retrieved evidence

$$p_{\theta, \eta}(y \mid x, v) = \sum_{z \in Z} p_{\eta}(z \mid q(x, v)) p_{\theta}(y \mid x, v, z)$$

Chapter 598. SFT loss

$$\mathcal{L}_{SFT} = - \sum_t m_t \log p_{\theta}(y_t \mid y_{<t}, x, v)$$

Chapter 599. LoRA update

$$W' = W + BA$$

Chapter 600. LoRA trainable parameters

$$r(d_{in} + d_{out})$$

instead of:

$$d_{in}d_{out}$$

Chapter 601. RL objective

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}}[R(y, x, v)]$$

Chapter 602. RLOO baseline

$$b_i = \frac{1}{K-1} \sum_{j \neq i} R_j$$

Chapter 603. RLOO advantage

$$A_i = R_i - b_i$$

Chapter 604. **32. Most Likely Exam Questions**

Chapter 605. **Question 1: Why is the adaptation ladder not simply a ranking from weak to strong?**

Because the methods change different parts of the system. Zero-shot changes only the prompt, few-shot adds demonstrations to context, RAG adds retrieved evidence, SFT changes the model through labeled examples, LoRA changes the parameterization of updates, DPO changes supervision from gold answers to preferences, and RL methods optimize reward. These methods are often complementary because they operate at different intervention levels.

Chapter 606. Question 2: What is the difference between zero-shot and few-shot prompting?

Zero-shot prompting gives only the instruction and input, relying entirely on pretrained knowledge. Few-shot prompting adds examples inside the context window. Neither changes model parameters. Few-shot behavior changes because the Transformer conditions on the full context, not because it has learned new weights.

Chapter 607. Question 3: Why is RAG useful for VLM adaptation?

RAG is useful when the model lacks domain-specific or current knowledge. It retrieves external documents and conditions the answer on them, changing what evidence the model sees at inference time. This can improve grounding without changing model parameters.

Chapter 608. Question 4: Why might RAG be better than SFT or DPO for missing domain knowledge?

If the failure is missing knowledge, RAG directly supplies relevant evidence. SFT teaches imitation but may not provide explicit facts at inference time. DPO adjusts preferences between answers but does not directly provide a factual evidence base. Therefore, RAG is often the most principled first intervention for knowledge gaps.

Chapter 609. Question 5: What is SFT and why does masking matter?

SFT trains the model on labeled instruction-image-answer triples using next-token maximum likelihood. Masking matters because only certain tokens should contribute to the loss, usually the assistant's answer tokens. This prevents the model from being trained to predict the user prompt, system text, or other context tokens.

Chapter 610. Question 6: What is LoRA?

LoRA is a parameter-efficient fine-tuning method. It freezes a pretrained weight matrix W and learns a low-rank update $\Delta W = BA$, so the effective weight becomes $W' = W + BA$. This greatly reduces trainable parameters while allowing the model to adapt to new tasks.

Chapter 611. Question 7: Why are SFT and LoRA not competing methods at the same level?

SFT defines the objective being optimized: imitate labeled answers using next-token likelihood. LoRA defines the parameterization of the update: only low-rank adapter weights are trained. Therefore, LoRA can be used to perform SFT. One answers “what loss?” and the other answers “which parameters move?”

Chapter 612. Question 8: Why adapt both attention and MLP projections?

Attention projections affect information routing: what the model queries, matches, attends to, and returns to the residual stream. MLP projections affect nonlinear transformation of token representations. Many tasks require both new routing patterns and new feature transformations, so adapting both can be more effective than adapting only one.

Chapter 613. Question 9: What is DPO and what is its limitation?

DPO trains the model using preferred and rejected responses. It increases the relative log-probability of preferred answers compared with rejected ones, usually relative to a frozen reference model. Its limitation is that preference does not necessarily equal factual correctness. If preferences reward style more than truth, DPO may improve tone while reducing grounding.

Chapter 614. Question 10: How is VLM generation interpreted as reinforcement learning?

The state is the prompt, image, and generated prefix. The action is the next token. The policy is the model's next-token distribution. The trajectory is the full generated response. The reward is a scalar score assigned to the answer. RLHF optimizes the policy to maximize expected reward over generated trajectories.

Chapter 615. Question 11: What is RLOO?

RLOO generates multiple completions for the same prompt, scores them, and compares each completion's reward to the average reward of the others. This leave-one-out baseline produces an advantage signal. Completions above the baseline are reinforced, while those below it are discouraged.

Chapter 616. 33. Common Mistakes to Avoid

Chapter 617. Mistake 1: Saying few-shot prompting updates weights

It does not. It changes context only.

Chapter 618. Mistake 2: Treating RAG as training

RAG usually changes inference-time evidence, not model parameters.

Chapter 619. Mistake 3: Confusing LoRA with SFT

SFT is an objective. LoRA is a parameter-efficient update mechanism.

Chapter 620. Mistake 4: Thinking DPO guarantees truth

DPO optimizes preference, not necessarily factual correctness.

Chapter 621. Mistake 5: Ignoring loss masking in SFT

Training should usually apply loss only to assistant response tokens.

Chapter 622. **Mistake 6: Thinking RLHF is unrelated to language modeling**

RLHF still uses autoregressive generation. It just treats token generation as policy optimization.

Chapter 623. **34. One-Page Open-Note Summary**

Chapter 9 explains VLM adaptation as a ladder of interventions applied to an autoregressive policy $p_{\theta}(y \mid x, v) = \prod_t p_{\theta}(y_t \mid y_{<t}, x, v)$. Zero-shot prompting changes only the instruction. Few-shot prompting adds examples in the context and changes behavior without updating parameters. RAG retrieves external documents and conditions generation on evidence, making it useful when the main problem is missing knowledge or grounding. SFT trains on labeled instruction-image-answer triples using next-token maximum likelihood, often with a loss mask so only assistant tokens count. LoRA is a parameter-efficient fine-tuning method that freezes pretrained weights and learns low-rank updates $W' = W + BA$; QLoRA adds quantization to reduce memory further. SFT defines the objective, while LoRA defines which parameters move. DPO uses preference pairs, increasing the relative probability of preferred responses over rejected ones, but preference does not guarantee factual correctness. RLHF treats generation as reinforcement learning: the state is prompt/image/generated prefix, the action is the next token, the trajectory is the full answer, and the reward scores the response. RLOO compares several completions for the same prompt using a leave-one-out reward baseline. The key lesson is that different adaptation methods act on different layers: context, evidence, parameters, preferences, or rewards.

Chapter 624. **35. Very Short Final Revision**

Know these:

- VLM policy: $p_{\theta}(y \mid x, v) = \prod_t p_{\theta}(y_t \mid y_{<t}, x, v)$.
- Zero-shot = instruction only, no examples, no weight update.
- Few-shot = examples in context, no weight update.
- RAG = retrieve evidence, condition answer on documents.
- SFT = train on labeled answers with next-token loss.
- Masking = usually train only assistant response tokens.
- LoRA = freeze W , train low-rank update BA .

- QLoRA = quantized base model + LoRA adapters.
- SFT = objective; LoRA = parameterization.
- DPO = preferred vs rejected answer alignment.
- Preference is not the same as factual correctness.
- RLHF = token generation as policy optimization.
- RLOO = compare multiple completions using leave-one-out baseline.
- Adaptation methods are complementary, not simply weak-to-strong.

Chapter 625. Chapter 10 — Quantization

Chapter 10 is about **quantization**, which means reducing the numerical precision of a neural network to make it smaller, faster, and cheaper to run.

The core idea is:

Quantization trades numerical precision for efficiency. The challenge is to reduce memory, latency, bandwidth, and energy while preserving model quality.

The professor's summary describes quantization as a numerical approximation problem, not just a deployment trick. The main question is where and how the approximation error should be absorbed so the model remains useful under low precision.

Chapter 626. 1. What Is Quantization?

Neural networks usually store weights and activations in floating-point precision.

Common formats:

Format	Bits	Example
FP32	32-bit	full precision
FP16 / BF16	16-bit	mixed precision training/inference
INT8	8-bit	common inference quantization
INT4	4-bit	aggressive LLM/VLM compression
Binary	1-bit	extreme quantization
Ternary	2 or few values	−1, 0, +1 style

Quantization converts high-precision values into a smaller set of discrete values.

For example, instead of storing:

0.123847

we may store an integer code such as:

12

plus a scale factor that maps the integer back to an approximate real value.

Chapter 627. 2. Why Quantization Matters

Large models are expensive because they require:

- huge memory,
- high bandwidth,
- many matrix multiplications,
- high energy,
- expensive GPUs,
- long inference latency.

Quantization helps by reducing:

1. **memory footprint,**
2. **memory bandwidth,**
3. **inference latency,**
4. **energy consumption,**
5. **deployment cost.**

Example:

A model with 1 billion parameters:

	Precision	Bytes per parameter	Approx storage
FP32		4 bytes	4 GB
FP16		2 bytes	2 GB
INT8		1 byte	1 GB
INT4		0.5 byte	0.5 GB

So INT4 can be roughly **8× smaller than FP32** for weights.

But smaller is not automatically better. Quantization introduces approximation error.

Chapter 628. 3. The Main Quantization Tradeoff

Quantization has one central tradeoff:

efficiency vs numerical fidelity

Lower precision gives:

- smaller model,
- faster inference,
- lower memory,
- lower energy.

But it may cause:

- accuracy loss,
- unstable generation,
- distorted embeddings,
- accumulated errors,
- worse long-context behavior,
- artifacts in diffusion,
- worse image-text alignment in VLMs.

The professor's summary says the central question is not simply whether low precision is possible, but which numerical objects are most important to preserve.

Chapter 629. 4. Uniform Quantization

A common quantizer is the uniform quantizer.

A simplified form is:

$$q = \text{clip}\left(\text{round}\left(\frac{x}{S}\right), q_{\min}, q_{\max}\right)$$

Then dequantization gives:

$$\hat{x} = sq$$

where:

Symbol	Meaning
x	original real value
s	scale / step size
q	integer quantized code
$q_{\min}, q_{\max}$	min/max integer codes
$\hat{x}$	reconstructed approximate value

The scale s controls the spacing between representable values.

Chapter 630. 5. Scale and Range Tradeoff

This is one of the most important concepts.

The scale s controls two things at the same time:

1. the spacing between quantization levels,
2. the real-valued range covered by integer codes.

The professor's sample question says this creates a structural tension: if s is small, resolution is fine but range is narrow; if s is large, range is wide but resolution is coarse.

Chapter 631. If scale is too small

Quantization levels are close together.

Good:

- low rounding error,
- fine precision.

Bad:

- small representable range,
 - more clipping.
-

Chapter 632. If scale is too large

Representable range is wide.

Good:

- less clipping,
- can cover outliers.

Bad:

- quantization grid is coarse,
 - higher rounding error.
-

Chapter 633. Exam sentence

In uniform quantization, the scale controls both precision and range. A small scale improves local resolution but increases clipping risk, while a large scale reduces clipping but increases rounding error.

Chapter 634. 6. Rounding Error and Clipping Error

Quantization error mainly comes from two sources.

Chapter 635. 6.1 Rounding Error

Rounding error happens because real values must be rounded to the nearest representable level.

Example:

If quantization step is:

$$s = 0.1$$

then:

$$0.34 \rightarrow 0.3$$

or:

$$0.34 \rightarrow 0.4$$

depending on rounding.

Error:

$$|x - \hat{x}|$$

This error exists even if the value is inside the allowed range.

Chapter 636. 6.2 Clipping Error

Clipping error happens when a value is outside the representable range.

Example:

If allowed range is:

$$[-1,1]$$

but:

$$x = 3.2$$

then it is clipped to:

$$1$$

This can cause large error.

Chapter 637. Important comparison

Error type	Cause	Fix
Rounding error	grid too coarse	smaller scale / more bits
Clipping error	range too narrow	larger scale / better clipping
Outlier problem	few extreme values stretch scale	per-channel/group scaling, smoothing, clipping

Chapter 638. 7. What Can Be Quantized?

Different tensors can be quantized.

Chapter 639. 7.1 Weight Quantization

Weights are quantized.

This reduces model storage.

Common in LLM deployment.

Chapter 640. 7.2 Activation Quantization

Activations are quantized during inference.

This reduces memory movement and can speed computation.

But activations are harder because they depend on input and may contain outliers.

Chapter 641. 7.3 Weight + Activation Quantization

Both weights and activations are quantized.

This can give stronger hardware speedups, but is harder to preserve accuracy.

Chapter 642. 7.4 KV-Cache Quantization

For autoregressive Transformers, the key-value cache can become very large during long-context decoding.

Quantizing KV cache reduces memory in long generation.

Chapter 643. **8. Quantization Granularity**

Quantization can use different scale granularities.

Chapter 644. **8.1 Per-Tensor Quantization**

One scale for the whole tensor.

Simple but less flexible.

Problem:

One outlier can force a large scale for everything.

Chapter 645. **8.2 Per-Channel Quantization**

One scale per output channel.

Better for weights because different channels can have different ranges.

Chapter 646. **8.3 Group-Wise Quantization**

One scale per group of weights.

Common for LLMs.

It balances:

- compression,
 - accuracy,
 - metadata overhead.
-

Chapter 647. Comparison

Granularity	Scales	Pros	Cons
Per-tensor	one scale	simple	sensitive to outliers
Per-channel	scale per channel	better accuracy	more metadata
Group-wise	scale per group	good balance	more complex

The professor’s summary says quantization design includes whether to use per-tensor, per-channel, or group-wise scales, and whether some layers/operators should remain in higher precision.

Chapter 648. 9. Quantization Phases

The chapter organizes quantization into three phases:

1. **Pre-quantization**
2. **During-training quantization**
3. **Post-training quantization**

The professor’s summary says these are different times at which we handle the same approximation problem.

Chapter 649. 10. Phase 1 — Pre-Quantization

Pre-quantization is the analysis and preparation phase.

The goal is to understand:

- weight ranges,
 - activation ranges,
 - outliers,
 - sensitive layers,
 - suitable bit-widths,
 - scaling granularity,
 - which layers should stay high precision.
-

Chapter 650. Calibration

Calibration means running representative data through the model to estimate activation distributions.

For example:

- min/max activation ranges,
- percentile ranges,
- outlier channels,
- layer sensitivity.

The professor's summary says calibration is best viewed as distribution estimation, not just mechanical preprocessing.

Chapter 651. Why Calibration Matters

If calibration data is not representative, the quantization range may be wrong.

Example:

If calibration data has small activations but real test data has large activations, the model may clip heavily at inference.

Exam sentence:

Calibration estimates the numerical distributions that quantization must preserve.

Chapter 652. 11. Layer Sensitivity

Some layers are more sensitive to quantization than others.

Sensitive layers may include:

- first layer,
- last layer,
- attention projections,
- normalization-related operations,
- output head,
- layers with strong outliers,

- layers important for cross-modal alignment.

A common strategy is **mixed precision**:

Quantize most of the model, but keep sensitive layers in higher precision.

Chapter 653. 12. Curvature and Hessian Sensitivity

A deeper idea in the chapter is that not all weight errors matter equally.

Suppose quantization perturbs a weight:

$$w \rightarrow w + \delta$$

The loss increase can be approximated by Taylor expansion.

Near a trained solution, the first derivative is often small, so the second derivative matters:

$$\Delta L \approx \frac{1}{2} H \delta^2$$

In multiple dimensions:

$$\Delta L \approx \frac{1}{2} \delta^T H \delta$$

where:

Symbol	Meaning
δ	quantization perturbation
H	Hessian / curvature matrix
ΔL	approximate loss increase

The professor's summary says curvature predicts quantization damage: the diagonal Hessian measures individual parameter sensitivity, while off-diagonal terms measure coupling between parameters.

Chapter 654. Intuition

If a parameter lies in a flat loss region, quantization error may not hurt much.

If a parameter lies in a sharp loss region, small quantization error can hurt a lot.

So curvature-aware quantization tries to protect sensitive directions.

Chapter 655. 13. Phase 2 — Quantization-Aware Training, QAT

Quantization-Aware Training means training the model while simulating quantization during the forward pass.

The model experiences quantization effects during training.

The professor's summary says QAT learns a model that is already robust to quantization distortions, instead of first learning a floating-point model and compressing it later.

Chapter 656. QAT Objective

A generic QAT objective is:

$$\min_{\theta} \mathbb{E}_{(x,y)} [\mathcal{L}(f_{Q(\theta)}(x), y)]$$

Meaning:

Train the model under fake-quantized behavior.

Chapter 657. Fake Quantization

During training, fake quantization simulates low precision:

1. quantize value,
2. dequantize back to float,

3. continue computation in training framework.

This lets training see quantization noise while still using floating-point hardware and gradients.

Chapter 658. **14. The Problem with Gradients Through Quantization**

Quantization includes operations like:

$$\text{round}(x)$$

and:

$$\text{clip}(x)$$

These are bad for gradients.

Chapter 659. **Rounding**

Rounding has zero derivative almost everywhere.

So if we backpropagate through exact rounding, learning stalls.

Chapter 660. **Clipping**

Clipping has zero gradient outside the clipping range.

So values outside the range may stop learning.

Chapter 661. **15. Straight-Through Estimator — STE**

The **Straight-Through Estimator** solves this problem approximately.

Forward pass:

use quantized values.

Backward pass:

pretend quantization is identity, or use a simple surrogate gradient.

The professor's summary says STE separates forward fidelity from backward usability: the forward pass exposes the network to quantization distortion, while the backward pass uses a surrogate derivative so optimization can proceed.

Chapter 662. Exam sentence

STE is conceptually necessary for QAT because true quantization operations have unhelpful gradients, so the backward pass uses a surrogate gradient while the forward pass still simulates quantized behavior.

Chapter 663. 16. PACT

PACT stands for **Parameterized Clipping Activation**.

It makes the clipping threshold learnable.

Instead of manually choosing activation clipping value α , PACT learns it during training.

The idea:

$$x_{clipped} = \min(\max(x, 0), \alpha)$$

where:

$$\alpha$$

is trainable.

The professor's summary says PACT changes clipping from a manual engineering choice into a learned parameter that balances preserving useful signal with narrowing the quantization range.

Chapter 664. Why PACT Helps

If α is too small:

- too much clipping,
- information loss.

If α is too large:

- range too wide,
- coarse quantization.

Learning α lets the model find a better tradeoff.

Chapter 665. 17. LSQ

LSQ stands for **Learned Step Size Quantization**.

LSQ learns the quantization step size:

s

instead of fixing it manually.

The professor's summary says LSQ makes the quantizer grid itself part of the optimization problem.

Chapter 666. PACT vs LSQ

Method	Learns
PACT	clipping threshold
LSQ	quantization step size

Both show that good quantization often comes from learning numerical decisions that were previously hand-designed.

The professor's sample question says PACT and LSQ are examples of learning previously fixed numerical decisions.

Chapter 667. **18. Binary and Ternary Quantization**

These are extreme quantization methods.

Chapter 668. **Binary quantization**

Weights or activations use only two values, often:

$$\{-1, +1\}$$

Very efficient, but accuracy can suffer.

Chapter 669. **Ternary quantization**

Uses three values:

$$\{-1, 0, +1\}$$

The zero value adds sparsity.

The professor's summary says binary and ternary methods make the approximation problem stark: with an extremely small representational alphabet, training must reorganize the network strongly enough to survive compression.

Chapter 670. **19. Phase 3 — Post-Training Quantization, PTQ**

Post-Training Quantization means quantizing a pretrained model without full retraining.

This is attractive because it is cheaper than QAT.

PTQ usually uses:

- calibration data,
- layer reconstruction,
- rounding methods,
- curvature information,
- activation smoothing.

Chapter 671. PTQ vs QAT

Feature	PTQ	QAT
When applied after training		during training/fine-tuning
Cost	lower	higher
Data needed	calibration data	training/fine-tuning data
Accuracy	may be lower	usually better
Complexity	simpler	more complex
Best for	quick deployment	high-quality low precision

Chapter 672. 20. RTN — Round-To-Nearest

The simplest PTQ method is **round-to-nearest**.

For each weight:

$$q = \text{round}(x/s)$$

It treats each weight independently.

The professor's summary says RTN is the natural reference point, exposing what advanced PTQ methods try to fix: lack of data awareness, layerwise reconstruction, and sensitivity information.

Chapter 673. Limitation of RTN

RTN minimizes scalar weight error, but not necessarily model output error.

A small weight error in a sensitive location may hurt more than a larger error elsewhere.

This motivates more advanced PTQ.

Chapter 674. 21. AdaRound

AdaRound improves on naive rounding.

Instead of automatically rounding each weight to nearest, it learns whether to round up or down based on layer output reconstruction.

The key idea:

The best rounding choice is the one that preserves the layer's function, not necessarily the one closest to the original weight.

The professor's summary says AdaRound shifts the viewpoint from "approximate each weight well" to "preserve the function of the layer well."

Chapter 675. **22. GPTQ**

GPTQ is a second-order PTQ method used for large Transformers.

It uses approximate Hessian / inverse-curvature information.

The idea:

1. quantize part of the weight matrix,
2. measure induced error,
3. update remaining weights to compensate,
4. repeat.

The professor's summary says GPTQ uses Hessian information to predict which quantization perturbations are expensive and updates remaining unquantized weights to reduce downstream distortion.

Chapter 676. **Why GPTQ Uses Curvature**

Training asks:

How do I reduce loss?

PTQ asks:

If quantization forces a perturbation, how harmful is it?

Curvature answers the second question.

High-curvature directions are sensitive.

Low-curvature directions are safer to perturb.

Chapter 677. **23. SmoothQuant**

SmoothQuant is designed for Transformer activation outliers.

Problem:

LLMs often have activation outliers that are hard to quantize.

SmoothQuant uses an equivalent rescaling trick.

It shifts some difficulty from activations into weights.

Conceptually:

Instead of directly quantizing troublesome activations, rescale the computation so activations become easier to quantize while preserving the floating-point function.

The professor's summary says SmoothQuant migrates part of the numerical difficulty from activations into weights without changing the floating-point function of the layer.

Chapter 678. **Why This Helps**

Activations vary depending on input and can have large outliers.

Weights are fixed and easier to analyze.

So moving difficulty from activations into weights can make quantization easier.

Chapter 679. **24. AWQ — Activation-Aware Weight Quantization**

AWQ is a weight-only PTQ method.

Its main idea:

Not all weight channels are equally important. Activations reveal which weight channels matter most.

AWQ protects important channels based on activation information.

The professor's summary says AWQ is activation-aware weight-only PTQ: activations reveal which weight channels matter most, so those channels receive special protection.

Chapter 680. **25. OmniQuant**

OmniQuant is a PTQ method that makes calibration more learnable.

It includes:

1. **Learnable Weight Clipping, LWC**
2. **Learnable Equivalent Transformations, LET**

The professor's summary says OmniQuant sits between fixed-parameter PTQ and full QAT: it does not retrain the network end to end, but it also does not accept fixed clipping/scaling rules as final.

Chapter 681. **26. I-BERT**

I-BERT focuses on fully integer Transformer inference.

Important point:

Quantizing only matrix multiplications is not enough.

A real Transformer also has operations like:

- GELU,
- Softmax,
- LayerNorm.

If these remain floating point, the computation graph is not fully quantized.

The professor's summary says the real deployment goal is a fully quantized computation graph, not merely quantized weight matrices.

Chapter 682. Exam sentence

I-BERT reframes quantization as a property of the entire executed graph, including nonlinear operators, not just the stored parameters.

Chapter 683. 27. Quantization in Autoregressive LLMs

Autoregressive LLMs are difficult to quantize because of:

1. activation outliers,
2. deep residual connections,
3. attention sensitivity,
4. long-context KV-cache accumulation,
5. error propagation during generation.

The professor's summary says autoregressive LLM quantization is difficult because of activation outliers, residual propagation, attention sensitivity, and KV-cache accumulation during long-context decoding.

Chapter 684. Why Errors Accumulate

In autoregressive generation, each token depends on previous tokens.

If quantization slightly changes one token probability, the model may generate a different token.

That different token changes the future context.

So small numerical errors can change the whole generated sequence.

Chapter 685. 28. Quantization in Diffusion Models

Diffusion models are iterative.

They apply the model many times across denoising timesteps.

Quantization errors can compound across steps.

The professor's summary says in diffusion models, small errors can compound across denoising timesteps, and calibration may need to be timestep-aware because activation distributions evolve through the reverse process.

Chapter 686. **Why Timestep-Aware Calibration Matters**

At early reverse steps, inputs are noisy.

At later reverse steps, inputs are cleaner.

Activation distributions change across timesteps.

So calibration data should cover different noise levels.

Otherwise quantization may work at some timesteps but fail at others.

Chapter 687. **29. Quantization in Diffusion Transformers**

Diffusion Transformers combine two challenges:

1. Transformer outlier problems,
2. diffusion timestep dependence.

So quantization must handle:

- attention sensitivity,
- MLP outliers,
- timestep-conditioned activations,
- repeated denoising steps.

The professor's summary says diffusion transformers contain both Transformer outlier issues and timestep dependence simultaneously.

Chapter 688. **30. Quantization in Flow Matching**

Flow matching learns a vector field:

$$v_{\theta}(x, t)$$

Quantization perturbs this vector field.

Because sampling integrates the vector field over time:

$$\frac{dx}{dt} = v_{\theta}(x, t)$$

small velocity errors can alter the trajectory.

The professor's summary says in flow matching, quantization perturbs a time-dependent vector field and can therefore alter the integrated trajectory of the solver itself.

Chapter 689. **31. Quantization in CLIP and VLMs**

CLIP-like models are hard to quantize because we care about embedding geometry.

A CLIP model must preserve:

- image embedding directions,
- text embedding directions,
- cosine similarities,
- cross-modal alignment,
- retrieval ranking.

A small distortion in embeddings may hurt retrieval or zero-shot classification.

The professor's summary says CLIP-like dual encoders are hard to quantize because the goal is not only local task accuracy but preservation of shared image-text embedding geometry.

Chapter 690. **32. Quantization in Fusion VLMs**

Fusion VLMs are even more complex because they contain:

- visual encoder,

- projector,
- fusion/cross-attention blocks,
- language decoder.

These components may not have equal sensitivity.

The professor’s summary says fusion-style multimodal Transformers are harder because visual encoder, projector, fusion blocks, and language decoder do not have equal sensitivities.

Chapter 691. Example

If the visual encoder is quantized too aggressively, image details may be lost.

If the projector is distorted, image features may not align with language tokens.

If the language decoder is distorted, generation quality may degrade.

So multimodal quantization must be component-aware.

Chapter 692. 33. Why Quantization Is Modality-Dependent

Different model families fail differently under quantization.

Model type	Main quantization issue
LLM	activation outliers, KV cache, long-context errors
Diffusion	timestep error accumulation
Flow matching	trajectory distortion
CLIP	embedding geometry distortion
Fusion VLM	unequal component sensitivity
ViT	attention/MLP sensitivity and patch-token errors

The professor’s summary says quantization is deeply modality dependent, especially when errors propagate through iterative, cross-modal, or long-context structures.

Chapter 693. **34. Quantization Hierarchy**

The chapter ends with a useful hierarchy.

Chapter 694. **Layer 1: Numerical**

- levels,
- scales,
- ranges,
- clipping,
- rounding.

Chapter 695. **Layer 2: Structural**

- which tensors,
- which layers,
- which operations,
- which channels are sensitive.

Chapter 696. **Layer 3: Algorithmic**

- calibration,
- QAT,
- PTQ,
- reconstruction,
- curvature,
- equivalent transformations.

Chapter 697. **Layer 4: Systems**

- deployable low-precision graph,
- hardware support,
- latency,
- bandwidth,
- energy.

The professor's summary says quantization connects these layers into one picture: deciding when, where, and how to absorb approximation error so the model remains functionally useful under severe numerical constraints.

Chapter 698. **35. Important Formula Summary**

Chapter 699. Uniform quantization

$$q = \text{clip}\left(\text{round}\left(\frac{x}{S}\right), q_{\min}, q_{\max}\right)$$
$$\hat{x} = sq$$

Chapter 700. Rounding error

$$e = x - \hat{x}$$

Chapter 701. Curvature approximation, 1D

$$\Delta L \approx \frac{1}{2} H \delta^2$$

Chapter 702. Curvature approximation, multi-dimensional

$$\Delta L \approx \frac{1}{2} \delta^T H \delta$$

Chapter 703. LoRA connection to quantization / QLoRA

$$W' = W + BA$$

where W may be quantized and BA is trainable.

Chapter 704. **36. Most Likely Exam Questions**

Chapter 705. Question 1: What is quantization?

Quantization is the process of approximating high-precision weights, activations, or other tensors using lower-precision discrete values. It reduces memory footprint, bandwidth, latency, and energy, but introduces numerical error. The main challenge is preserving model behavior while using fewer bits.

Chapter 706. Question 2: What is the scale-range tradeoff in uniform quantization?

In uniform quantization, the scale controls both the spacing between representable values and the real-valued range covered by the integer codes. A small scale gives fine resolution and low rounding error but increases clipping risk. A large scale covers a wider range and reduces clipping but makes the grid coarse, increasing rounding error. This is the central precision-range tension.

Chapter 707. Question 3: What are rounding error and clipping error?

Rounding error occurs when a real value inside the representable range is rounded to the nearest quantization level. Clipping error occurs when a value lies outside the representable range and is clipped to the minimum or maximum code. Rounding error is caused by coarse grid resolution, while clipping error is caused by insufficient range.

Chapter 708. Question 4: What is calibration?

Calibration is the process of running representative data through the model to estimate activation ranges, outliers, and layer sensitivity before quantization. It is a form of distribution estimation. Good calibration helps choose scales, clipping thresholds, and granularity.

Chapter 709. Question 5: Why is curvature useful for quantization?

Curvature estimates how harmful a quantization perturbation will be. Near a trained optimum, first-order gradients may be small, so second-order Hessian terms dominate the loss increase. High-curvature directions are sensitive to perturbation and should be protected, while low-curvature directions can tolerate more quantization error.

Chapter 710. Question 6: What is QAT and why is STE needed?

Quantization-Aware Training trains the model while simulating quantized weights and/or activations during the forward pass. STE is needed because quantization operations such as rounding and clipping have unhelpful gradients. The forward pass uses fake-quantized values, while the backward pass uses a surrogate gradient so training can proceed.

Chapter 711. Question 7: What are PACT and LSQ?

PACT learns the activation clipping threshold, turning clipping into a trainable decision. LSQ learns the quantization step size, turning the grid spacing into a trainable parameter. Both methods show that good quantization often comes from learning numerical choices that were previously fixed manually.

Chapter 712. Question 8: What is PTQ?

Post-Training Quantization quantizes a pretrained model without full retraining. It is cheaper than QAT and usually uses calibration data. Simple PTQ uses round-to-nearest, while more advanced PTQ methods use layer reconstruction, activation-aware scaling, or second-order information.

Chapter 713. Question 9: What is the limitation of round-to-nearest?

Round-to-nearest treats each weight independently and minimizes scalar weight error. However, the best rounding decision is not always the one closest to the original weight, because what matters is preserving model or layer output. This motivates methods such as AdaRound and GPTQ.

Chapter 714. Question 10: What is GPTQ?

GPTQ is a second-order post-training quantization method for large Transformers. It uses approximate Hessian or inverse-curvature information to quantize weights while compensating remaining weights for the induced error. Its goal is to reduce the functional damage caused by quantization.

Chapter 715. Question 11: What is SmoothQuant?

SmoothQuant is a Transformer quantization method that handles activation outliers by rescaling the computation. It migrates some quantization difficulty from activations into weights without changing the original floating-point function. This makes activations easier to quantize.

Chapter 716. Question 12: Why is quantization difficult for diffusion models?

Diffusion models apply the denoising network repeatedly over many timesteps. Small quantization errors can accumulate across the reverse trajectory. Also, activation distributions change with timestep, so calibration may need to be timestep-aware.

Chapter 717. Question 13: Why is quantization difficult for CLIP-like VLMs?

CLIP-like models rely on preserving a shared image-text embedding geometry. Quantization must preserve not only classification accuracy but also cosine similarities, retrieval rankings, and cross-modal alignment. Small embedding distortions can harm zero-shot classification and retrieval.

Chapter 718. Question 14: Why is fully quantizing a Transformer more than quantizing matrix multiplications?

A Transformer includes nonlinear and normalization operations such as GELU, Softmax, and LayerNorm. If these remain floating point, the computation graph is not fully quantized. A deployable low-precision model should consider the entire executed graph, not only weight matrices.

Chapter 719. 37. Common Mistakes to Avoid

Chapter 720. Mistake 1: Thinking quantization is only compression

Quantization reduces memory, but it also affects bandwidth, latency, energy, and hardware execution.

Chapter 721. Mistake 2: Ignoring activation quantization

Weight quantization reduces storage, but activation quantization is often needed for real speedups.

Chapter 722. Mistake 3: Thinking lower bit always means better

Lower bits improve efficiency but may destroy accuracy or generation quality.

Chapter 723. Mistake 4: Thinking all layers are equally sensitive

Some layers, channels, or operations are much more fragile under quantization.

Chapter 724. Mistake 5: Thinking PTQ and QAT are the same

PTQ quantizes after training. QAT trains while simulating quantization.

Chapter 725. Mistake 6: Thinking STE gives the true gradient

STE gives a surrogate gradient, not the real derivative of rounding.

Chapter 726. Mistake 7: Thinking CLIP quantization only needs classification accuracy

CLIP needs preservation of embedding geometry and cross-modal similarity.

Chapter 727. **38. One-Page Open-Note Summary**

Quantization approximates high-precision model values using lower-precision discrete codes to reduce memory, bandwidth, latency, and energy. Uniform quantization uses a scale s to map real values to integer codes and back. The central tradeoff is precision versus range: a small scale gives fine resolution but more clipping, while a large scale covers outliers but increases rounding error. Quantization error comes mainly from rounding and clipping. Quantization can target weights, activations, KV-cache, or the full computation graph, and can use per-tensor, per-channel, or group-wise scales. Pre-quantization involves calibration and sensitivity analysis. Curvature/Hessian information helps predict which weight perturbations are harmful. QAT trains under simulated quantized behavior and uses STE because rounding and clipping are not gradient-friendly. PACT learns clipping thresholds, while LSQ learns step sizes. PTQ quantizes a pretrained model using calibration, with RTN as the simplest baseline. AdaRound learns rounding decisions based on layer reconstruction. GPTQ uses second-order information to compensate quantization errors. SmoothQuant handles activation outliers by rescaling difficulty from activations into weights. AWQ protects important channels based on activation information. OmniQuant learns clipping and equivalent transformations in PTQ. I-BERT highlights that full deployment requires quantizing nonlinear operations, not only matrix multiplications. Quantization is especially challenging for LLMs, diffusion models, flow matching, CLIP, and fusion VLMs because errors propagate through long-context, iterative, vector-field, or cross-modal structures.

Chapter 728. **39. Very Short Final Revision**

Know these:

- Quantization = lower precision approximation.
- Goal = reduce memory, bandwidth, latency, and energy.
- Main risk = numerical error and quality loss.
- Uniform quantization uses scale s .
- Small scale = fine precision but clipping.
- Large scale = wide range but rounding error.
- Rounding error = nearest level approximation.
- Clipping error = value outside range.
- Calibration = estimate activation/range distributions.
- QAT = train with fake quantization.
- STE = surrogate gradient through rounding/clipping.
- PACT = learn clipping threshold.
- LSQ = learn step size.
- PTQ = quantize after training.
- RTN = round-to-nearest baseline.
- AdaRound = data-aware rounding.

- GPTQ = Hessian/curvature-aware PTQ.
- SmoothQuant = move difficulty from activations to weights.
- AWQ = activation-aware weight protection.
- I-BERT = quantize full Transformer graph.
- Diffusion quantization needs timestep awareness.
- CLIP/VLM quantization must preserve embedding geometry and cross-modal alignment.